

MCS-207 Database Management Systems

Solved question bank · every question from 10 term-end papers, December 2021 to June 2026

Transactions, ACID, concurrency and recovery

Q1. What is a transaction ? Explain properties of a transaction with an example.

[4-10 marks · J23 Q1(c), J24 Q4(a), D24 Q1(c), J25 Q4(a), D25 Q1(e), J26 Q3(c)]

A **transaction** is a logical unit of work on a database — a sequence of read and write operations that must be executed completely or not at all. It begins with BEGIN TRANSACTION and ends with COMMIT (make permanent) or ROLLBACK/ABORT (undo everything).

Example — transfer Rs. 500 from account A (balance 2000) to account B (balance 1000):

```
T1: READ(A)           /* A = 2000 */
    A = A - 500
    WRITE(A)           /* A = 1500 */
    READ(B)            /* B = 1000 */
    B = B + 500
    WRITE(B)           /* B = 1500 */
    COMMIT
```

A transaction must satisfy the four **ACID** properties:

Property	Meaning	In the example
Atomicity	All operations happen or none do	If the system crashes after WRITE(A), the debit of 500 is undone; money cannot vanish
Consistency	A transaction takes the database from one consistent state to another	$A+B = 3000$ before and after
Isolation	Concurrent transactions do not see each other's intermediate state	Another transaction must not read $A = 1500$ while B is still 1000
Durability	Once committed, effects survive any later failure	After COMMIT, $A = 1500$ and $B = 1500$ persist even if power fails

Atomicity and durability are enforced by the recovery manager using the log; isolation is enforced by the concurrency control manager using locks; consistency is enforced jointly with integrity constraints.

Two transactions are said to be **concurrent** when their execution intervals overlap in time — one has begun but not yet committed or aborted when the other begins, so their operations may be interleaved by the scheduler.

Q2. Explain ACID properties of a transaction with the help of an example.

[4-6 marks · J22 Q2(c), D22 Q4(a)]

The four properties of a transaction are **Atomicity, Consistency, Isolation and Durability (ACID)**.

Take T1: transfer Rs. 500 from A (2000) to B (1000) — READ(A); A=A-500; WRITE(A); READ(B); B=B+500; WRITE(B); COMMIT.

- **Atomicity** — "all or nothing". If the machine fails between WRITE(A) and WRITE(B), the recovery manager rolls A back to 2000. A half-done transfer is never left behind.
- **Consistency** — the transaction preserves every integrity constraint and application invariant. Here the invariant $A+B = 3000$ holds both before T1 starts and after it commits.
- **Isolation** — the net effect of concurrent transactions equals some serial order. T2 reading A and B must not observe $A = 1500$ with $B = 1000$ (total 2500).
- **Durability** — after COMMIT the new values 1500 and 1500 are written to stable storage (or recoverable from the log), so they survive a crash.

Q3. Explain the term transaction in the context of DBMS, with the help of an example. What are the problems associated when concurrent transactions are executed ? Explain any one of these problems with the help of an example. [10 marks · D21 Q1(c)]

A **transaction** is a unit of program execution that accesses and possibly updates the database, and that the DBMS treats as atomic. It is delimited by BEGIN TRANSACTION ... COMMIT / ROLLBACK, and it must satisfy the ACID properties.

Example: T1 withdraws Rs. 500 from account A — READ(A); A = A - 500; WRITE(A); COMMIT. Either both the arithmetic and the WRITE take effect, or neither does.

When two or more transactions are executed concurrently and the scheduler interleaves their operations without control, four problems can arise:

1. **Lost update problem** — two transactions read the same item and the second write overwrites the first.
2. **Dirty read (uncommitted dependency) problem** — a transaction reads a value written by another transaction that later aborts.
3. **Unrepeatable read (inconsistent analysis) problem** — a transaction reads the same item twice and gets different values because another transaction updated it in between.
4. **Phantom read problem** — a transaction re-executes a range query and finds new rows inserted by another transaction.

Lost update, explained. Let $X = 1000$. T1 adds 100, T2 adds 200.

Time	T1	T2	X in database
t1	READ(X) → 1000		1000
t2		READ(X) → 1000	1000
t3	$X = X + 100$ (local 1100)		1000
t4		$X = X + 200$ (local 1200)	1000
t5	WRITE(X)		1100
t6		WRITE(X)	1200
t7	COMMIT	COMMIT	1200

The correct final value is 1300. T1's update of +100 has been **lost** because T2 computed from a stale copy of X and overwrote T1's write. Locking (T2 must wait for T1's exclusive lock on X) prevents this.

Q4. What are the problems of concurrent transactions ? Explain with the help of an example of each. [9–10 marks · J22 Q1(c), D23 Q3(b)]

Concurrent execution is **needed** to increase throughput (CPU and I/O overlap), reduce average response time for short transactions, and make full use of multi-user systems. Uncontrolled interleaving, however, causes these anomalies.

(i) Lost update. X = 1000; T1 adds 100, T2 adds 200.

T1	T2	X
READ(X) → 1000		1000
	READ(X) → 1000	1000
WRITE(X) = 1100		1100
	WRITE(X) = 1200	1200

Correct answer 1300; T1's update is lost.

(ii) Dirty read / uncommitted dependency. X = 1000.

T1	T2	X
READ(X); X = X + 500; WRITE(X)		1500
	READ(X) → 1500 (dirty)	1500
ROLLBACK		1000

T2 has computed with 1500, a value that never existed in any committed state.

(iii) Unrepeatable read. T1 reads X twice to verify a total.

T1	T2	X
READ(X) → 1000		1000
	READ(X); X = 2000; WRITE(X); COMMIT	2000
READ(X) → 2000		2000

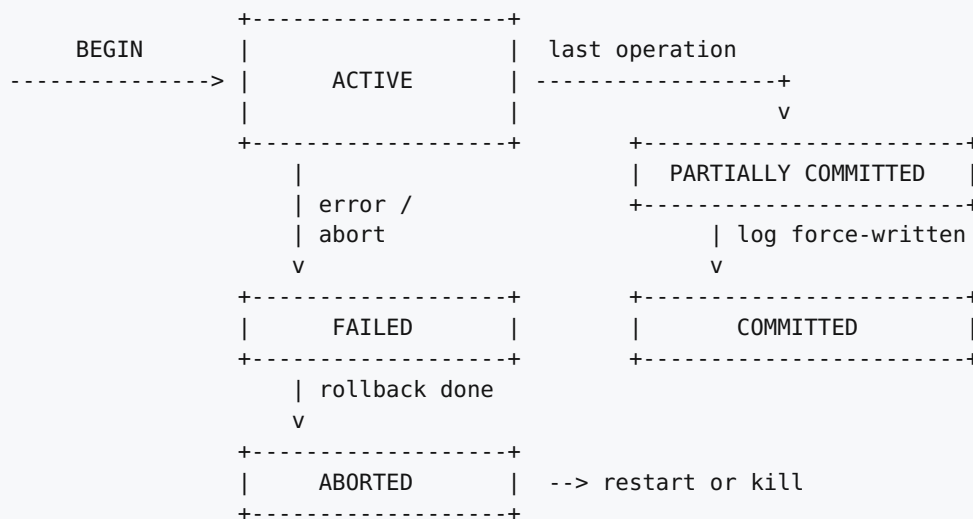
The same read in the same transaction returns two different values, so T1's analysis is inconsistent.

(iv) Phantom read. T1 executes `SELECT COUNT(*) FROM Account WHERE branch='Delhi'` and gets 10; T2 inserts a new Delhi account and commits; T1 repeats the query and gets 11 — a "phantom" row has appeared.

All four are prevented by a serialisable schedule, obtained in practice by strict two-phase locking.

Q5. Explain the states of a transaction execution with the help of a diagram. Also define the lost update and unrepeatable reads problems of concurrent execution of transactions with the help of an example of each. [10 marks · D24 Q4(a)]

A transaction passes through five states:



- **Active** — the initial state; the transaction is executing its read/write operations.
- **Partially committed** — the final statement has executed, but the log records are not yet forced to stable storage.
- **Committed** — the log has been written out; the changes are permanent.
- **Failed** — normal execution can no longer proceed (constraint violation, deadlock victim, system error).
- **Aborted** — the database has been restored to its state before the transaction began (rolled back).

Lost update problem. Two transactions read the same data item and the later write overwrites the earlier one, so one update disappears. With $X = 500$, T1 adding 100 and T2 adding 50: both read 500, T1 writes 600, T2 writes 550. The correct value is 650; T1's +100 is lost.

Unrepeatable read problem. A transaction reads the same data item twice and obtains different values, because another transaction updated and committed the item in between. T1 reads balance = 500; T2 then sets balance = 900 and commits; T1 reads balance again and gets 900. T1 can no longer reproduce its own earlier read, so any computation spanning the two reads is inconsistent.

Q6. What is a transaction schedule in the context of concurrent transactions ? Explain the following anomalies/problems of concurrent transactions with the help of an example of each: (i) Lost update anomaly (ii) Unrepeatable read anomaly (iii) Dirty read anomaly. [10 marks · D25 Q4(a)]

A **schedule** is a chronological ordering of the operations (READ, WRITE, COMMIT, ABORT) of a set of concurrent transactions, such that the relative order of the operations *within* each individual transaction is preserved. A schedule is **serial** if each transaction runs to completion before the next begins, and **serialisable** if its effect is equivalent to some serial schedule. Non-serialisable schedules produce the anomalies below.

(i) Lost update anomaly — $X = 100$.

T1	T2	X
READ(X) → 100		100
	READ(X) → 100	100
$X = X + 50$; WRITE(X)		150
	$X = X + 20$; WRITE(X)	120

Correct value 170. T1's update is lost because T2 wrote from a value read before T1's write.

(ii) Unrepeatable read anomaly — $X = 100$.

T1	T2	X
READ(X) → 100		100
	WRITE(X) = 400; COMMIT	400
READ(X) → 400		400

T1's two reads of the same item disagree.

(iii) Dirty read anomaly — $X = 100$.

T1	T2	X
WRITE(X) = 600		600
	READ(X) → 600	600
ROLLBACK		100

T2 read 600, a value from an uncommitted transaction that was later undone, so T2's result is based on data that never existed in the database.

Q7. Consider the following three transactions TA (READ A; A=A+100; WRITE A), TB (READ B; B=B-100; WRITE B), TC (READ A; A=A+200; WRITE A), executed concurrently. Show the execution schedules for: (i) a schedule serializable with transaction sequence TA, TC, TB; (ii) a schedule which results in the lost update problem. [10 marks · J25 Q1(c)]

Let the initial values be $A = 1000$ and $B = 500$.

(i) Schedule serialisable with the order TA, TC, TB. TA and TC both touch A, so their operations must not interleave; TB touches only B, so it may be interleaved freely without affecting serialisability.

Time	TA	TC	TB	A	B
t1	READ(A) → 1000			1000	500
t2	A = A + 100			1000	500
t3	WRITE(A)			1100	500
t4	COMMIT			1100	500
t5		READ(A) → 1100		1100	500
t6		A = A + 200	READ(B) → 500	1100	500
t7		WRITE(A)	B = B - 100	1300	500
t8		COMMIT	WRITE(B)	1300	400
t9			COMMIT	1300	400

Final $A = 1300$, $B = 400$ — identical to the serial order TA, TC, TB, so the schedule is serialisable (conflict-equivalent to it).

(ii) Schedule showing the lost update problem. Interleave TA and TC so that both read A before either writes it.

Time	TA	TC	A
t1	READ(A) → 1000		1000
t2		READ(A) → 1000	1000
t3	A = A + 100 (local 1100)		1000
t4		A = A + 200 (local 1200)	1000
t5	WRITE(A)		1100
t6		WRITE(A)	1200
t7	COMMIT	COMMIT	1200

The correct final value is 1300. The update made by TA (+100) has been **lost**, because TC overwrote A using a value it had read before TA's write. Exclusive locking of A for the whole of TA would force TC to wait and remove the anomaly.

Q8. Define a serializable schedule. Determine whether the given Schedule A (T1: Read(X), Write(Y), Commit interleaved with T2: Read(X), Write(Y), Commit) is serializable or not.

[5 marks · D23 Q1(b)]

A schedule of n concurrent transactions is **serialisable** if it is equivalent in its effect on the database to *some* serial schedule of the same n transactions. The usual test is **conflict serialisability**: build a precedence graph with one node per transaction and an edge $T_i \rightarrow T_j$ whenever T_i performs an operation before a *conflicting* operation of T_j (two operations conflict if they touch the same item and at least one is a WRITE). The schedule is conflict-serialisable iff the graph is **acyclic**; a topological sort then gives the equivalent serial order.

Schedule A: T1: Read(X), Write(Y), Commit; T2: Read(X), Write(Y), Commit, interleaved.

- Read(X) by T1 and Read(X) by T2 — both reads, **no conflict**.
- Write(Y) by T1 and Write(Y) by T2 — write-write on the same item, **conflict**. Whichever of them occurs first, say T1's, gives the single edge $T1 \rightarrow T2$.

The precedence graph has nodes {T1, T2} and the single edge $T1 \rightarrow T2$, which is **acyclic**. Hence Schedule A is **serialisable**, equivalent to the serial schedule T1 followed by T2. The final value of Y is the one written by the later Write, exactly as in that serial order.

Q9. Differentiate between serial schedule and serializable schedule. How can it be assured that a schedule is serializable or not ? Describe the role of locks in assuring serializability.

[8 marks · J26 Q4(a)]

Serial schedule	Serialisable schedule
Transactions execute one after another; no interleaving at all	Operations of different transactions are interleaved
Always correct, but gives no concurrency	Correct <i>and</i> concurrent
Throughput is low; CPU idles during I/O	High throughput, good resource utilisation
n transactions admit $n!$ serial schedules	Equivalent in effect to at least one of those $n!$ serial schedules
Trivially free of all concurrency anomalies	Free of anomalies because of that equivalence

Testing serialisability. Draw the **precedence (serialisability) graph**: one node per transaction, and an edge $T_i \rightarrow T_j$ if T_i executes an operation before a conflicting operation of T_j (same data item, at least one WRITE: read-write, write-read, write-write). If the graph is **acyclic** the schedule is conflict-serialisable and a topological sort yields the equivalent serial order; a **cycle** means it is not serialisable. View serialisability is a weaker, more general but NP-hard test.

Role of locks. Testing a schedule after the fact is useless at run time, so the DBMS *generates* only serialisable schedules by locking. Before reading an item a transaction takes a **shared (S)** lock, before writing it an **exclusive (X)** lock; the compatibility matrix allows S with S only.

	S requested	X requested
S held	granted	wait
X held	wait	wait

Two-phase locking (2PL) guarantees serialisability: every transaction has a **growing phase** in which it only acquires locks and a **shrinking phase** in which it only releases them — no lock may be acquired after the first release. The point of the last acquisition is the transaction's *lock point*, and ordering transactions by lock point gives the equivalent serial order. **Strict 2PL** holds all exclusive locks until commit, which additionally prevents dirty reads and cascading rollbacks.

Q10. Serialisable schedule (short note). [4-5 marks · J23 Q4(a)(i), J24 Q5(iii)]

A schedule is **serialisable** if its effect on the database is the same as that of some serial execution of the same transactions. It is the correctness criterion for concurrency control: it permits interleaving (for throughput) while guaranteeing the Isolation property of ACID.

Two forms are used. **Conflict serialisability** — the schedule can be transformed into a serial schedule by swapping only non-conflicting adjacent operations; tested by checking that the precedence graph (edge $T_i \rightarrow T_j$ on each conflicting pair, T_i first) is acyclic. **View serialisability** — a weaker condition requiring that each transaction read the same values and the final write of each item be by the same transaction as in some serial schedule.

Example: $S = R1(A), W1(A), R2(A), W2(A), R1(B), W1(B)$ gives edges $T1 \rightarrow T2$ only, so it is serialisable and equivalent to $T1, T2$. If instead $T2$ also wrote B before $T1$ read it, a cycle $T1 \rightarrow T2 \rightarrow T1$ would appear and the schedule would not be serialisable. Two-phase locking produces only serialisable schedules.

Q11. Explain the concept of locking in RDBMS. How does locking solve the problems of concurrent transactions ? Explain with the help of an example. [7 marks · J22 Q4(c)]

A **lock** is a variable associated with a data item that records which transactions may currently access it. A transaction must request a lock from the lock manager before accessing the item and release it afterwards. Two modes are used:

- **Shared lock, S (read lock)** — many transactions may hold S on the same item simultaneously; the holder may only read.
- **Exclusive lock, X (write lock)** — only one transaction may hold X, and no S may coexist with it; the holder may read and write.

Compatibility: S/S compatible; S/X, X/S, X/X incompatible, so the requester waits.

How locking removes the anomalies. Because conflicting accesses are serialised by the lock manager, the interleavings that produce lost updates, dirty reads and unrepeatable reads cannot

occur. Under **strict 2PL** all X locks are held until commit, so no other transaction can read or overwrite an uncommitted value.

Example — the lost update of Q3, now with locking. X = 1000; T1 adds 100, T2 adds 200.

Time	T1	T2	X
t1	X-lock(X) granted; READ(X) → 1000		1000
t2		X-lock(X) requested → WAIT	1000
t3	X = X + 100; WRITE(X)	waiting	1100
t4	COMMIT; unlock(X)		1100
t5		lock granted; READ(X) → 1100	1100
t6		X = X + 200; WRITE(X); COMMIT	1300

The final value is the correct 1300. The cost of locking is reduced concurrency and the possibility of **deadlock**, which is handled by prevention schemes (wait-die, wound-wait) or by wait-for-graph detection and victim rollback.

Q12. What is a lock ? Why are locks used ? Explain the two phase locking protocol with the help of an example. [7-10 marks · D23 Q3(a), J25 Q4(b)]

A **lock** is a control variable attached to a data item that regulates concurrent access to it. Locks are needed because unrestricted interleaving of transactions produces the lost update, dirty read and unrepeatable read anomalies; by making conflicting operations wait, the lock manager forces the schedule to be serialisable and so enforces the Isolation property.

Types of locks

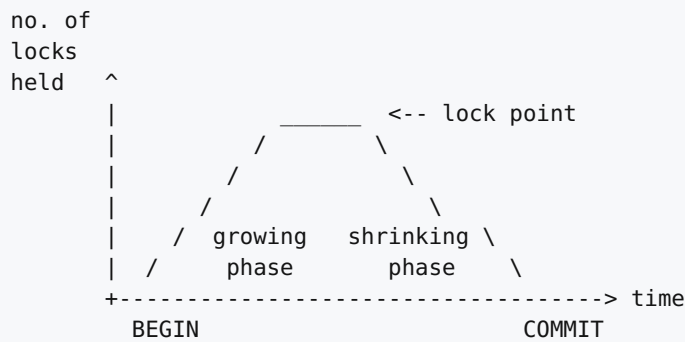
Lock	Granted to	Permits	Compatible with
Shared (S) / read lock	many transactions at once	READ only	other S locks
Exclusive (X) / write lock	one transaction only	READ and WRITE	nothing

(Refinements: update locks, and intention locks IS/IX/SIX for multi-granularity locking over database → file → page → record.)

Two-phase locking (2PL). Each transaction obeys two phases:

1. **Growing (expanding) phase** — the transaction may acquire locks and upgrade S to X, but may not release any lock.
2. **Shrinking (contracting) phase** — the transaction may release locks (and downgrade), but may not acquire any new lock.

The instant of the last lock acquisition is the **lock point**. 2PL guarantees conflict serialisability: the serial order equivalent to the schedule is the order of the transactions' lock points.



Example. T1 transfers 500 from A to B under 2PL:

```
X-lock(A)           -- growing
READ(A); A = A - 500; WRITE(A)
X-lock(B)           -- still growing (lock point here)
READ(B); B = B + 500; WRITE(B)
unlock(A)           -- shrinking begins
unlock(B)
COMMIT
```

If a concurrent T2 requests X-lock(A) at any point before T1's unlock(A), it waits, so T2 can neither overwrite nor read A's uncommitted value. **Strict 2PL** (hold all X locks until commit) and **rigorous 2PL** (hold all locks until commit) are the forms used in practice, because basic 2PL still permits cascading rollback. 2PL does not prevent deadlock: if T1 locks A then wants B while T2 locks B then wants A, both wait forever and one must be rolled back.

Q13. Two-phase locking / Locking in transaction (short note).

[3–4 marks · D21 Q5(d), D22 Q1(c)(ii), J23 Q4(a)(ii)]

Locking is the standard mechanism for concurrency control. Before reading an item a transaction acquires a shared (S) lock, before writing it an exclusive (X) lock; S is compatible only with S, so conflicting transactions are made to wait and anomalies such as lost update and dirty read cannot arise.

Two-phase locking (2PL) restricts *when* locks may be taken. A transaction has a **growing phase** in which it only acquires locks and a **shrinking phase** in which it only releases them; once it has released any lock it may never acquire another. The last acquisition is its lock point, and the order of lock points is the equivalent serial order — which is why 2PL guarantees a serialisable schedule.

Example: X-lock(A); read A; write A; X-lock(B); read B; write B; unlock(A); unlock(B); commit. Legal under 2PL. Releasing A before locking B would violate it. **Strict 2PL** retains all exclusive locks until commit, which also eliminates dirty reads and cascading rollbacks. 2PL does not prevent deadlock, which is handled separately.

Q14. What is database recovery ? Explain the log based recovery with the help of an example.

[6–8 marks · D22 Q4(b), D24 Q4(b), J26 Q4(b), J22 Q3(b)]

Database recovery is the process of restoring the database to the most recent consistent state after a failure — a transaction failure (logical error, deadlock abort), a system crash (power or software failure, losing main memory), or a media failure (disk loss). It enforces Atomicity (undo partial work) and Durability (redo committed work).

The log. The DBMS maintains a sequential **log** (journal) on stable storage. The main record types are:

```
<Ti, start>
<Ti, X, old_value, new_value>    -- update record
<Ti, commit>
<Ti, abort>
<checkpoint, L>                -- L = list of active transactions
```

The **write-ahead logging (WAL)** rule is that the log record for an update must reach stable storage *before* the corresponding data page, and all log records of a transaction must be forced out before its commit record is acknowledged.

Recovery actions. On restart the log is scanned from the last checkpoint:

- **UNDO (backward recovery)** — for every T_i with $\langle T_i, \text{start} \rangle$ but no $\langle T_i, \text{commit} \rangle$, scan backwards and restore each X to its `old_value`; write $\langle T_i, \text{abort} \rangle$.
- **REDO (forward recovery)** — for every T_i with $\langle T_i, \text{commit} \rangle$, scan forwards and reapply each `new_value`.

Example. A (1000) and B (2000). T_1 transfers 500 from A to B; T_2 adds 300 to B. The log at the time of the crash:

```
<T1, start>
<T1, A, 1000, 500>
<T1, B, 2000, 2500>
<T1, commit>
<T2, start>
<T2, B, 2500, 2800>
*** CRASH ***
```

Recovery: T_1 has a commit record → **REDO T_1** , setting $A = 500$ and $B = 2500$. T_2 has no commit record → **UNDO T_2** , restoring B to 2500. The database comes back as $A = 500$, $B = 2500$ — exactly the state after T_1 only, which is consistent. A **checkpoint** record written periodically (flush log buffers and dirty pages, then write $\langle \text{checkpoint}, L \rangle$) means the redo scan need only start at the last checkpoint instead of at the beginning of the log.

Q15. What is the purpose of log-file in the context of database recovery ? Explain with the help of an example. [5 marks · D25 Q1(f)]

The **log file** (journal) is a sequential file on stable storage in which the DBMS records every change made to the database *before* that change is applied to the data files (the write-ahead logging rule). Its purpose is to provide the information needed to undo the effects of incomplete transactions and redo the effects of committed ones, thereby guaranteeing Atomicity and Durability. It also supports media recovery when combined with an archive dump.

Record types: $\langle T_i, \text{start} \rangle$, the update record $\langle T_i, X, \text{old_value}, \text{new_value} \rangle$, $\langle T_i, \text{commit} \rangle$, $\langle T_i, \text{abort} \rangle$ and $\langle \text{checkpoint}, L \rangle$.

Example — $A = 1000$, T_1 debits 200:

```
<T1, start>
<T1, A, 1000, 800>
<T1, commit>
```

If the system crashes *after* the commit record is on disk but before the data page was flushed, the recovery manager **redoes** T_1 using `new_value` and sets $A = 800$. If the crash happens with only

the first two records present, there is no commit record, so the manager **undoes** T1 using `old_value` and restores `A = 1000`. Either way the database returns to a consistent state, which is impossible without the log.

Q16. What is Data Recovery ? How can it be performed ? Explain with the help of an example.

[6 marks · D21 Q4(a), D22 Q1(c)(iv)]

Data (database) recovery is the restoration of the database to the latest consistent state after a transaction failure, system crash or media failure, so that no committed update is lost and no uncommitted update remains.

How it is performed. Three mechanisms are combined:

1. **Logging** — every update is first recorded as `<Ti, X, old, new>` in the log on stable storage (write-ahead logging).
2. **Checkpointing** — periodically the log buffer and all dirty pages are flushed and a `<checkpoint, L>` record is written, bounding the work done at restart.
3. **Restart (undo/redo)** — the log is read from the last checkpoint; transactions with a commit record are **redone** (forward recovery, apply new values), transactions without one are **undone** (backward recovery, restore old values). For media failure, the most recent **archive dump** is reloaded and the log is rolled forward over it.

Other techniques: **shadow paging** (keep a shadow page table and switch atomically at commit) and **deferred update** (write to the database only after commit, so only redo is ever needed).

Example — `X = 500, Y = 700`:

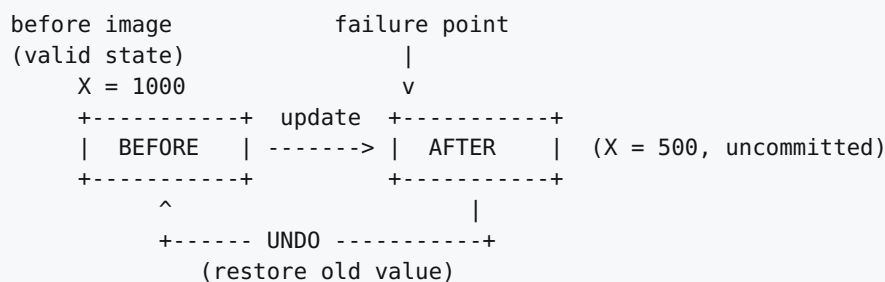
```
<T1, start> <T1, X, 500, 400> <T1, commit> <T2, start> <T2, Y, 700, 900> *crash*
```

T1 is redone (`X = 400`); T2 is undone (`Y = 700`). The recovered state is `X = 400, Y = 700`.

Q17. Explain the concept of backward recovery and forward recovery with the help of a diagram for each. [6 marks · J24 Q4(b), J25 Q4(c), J23 Q4(a)(iii)]

Recovery is needed because failures (transaction aborts, system crashes, disk loss) can leave the database with partially applied transactions, violating Atomicity, or with committed updates present only in a lost memory buffer, violating Durability. Two complementary techniques are used.

Backward recovery (UNDO / rollback) — applied to transactions that had started but not committed. The log is scanned **backwards** and each updated item is restored to its `old_value`, taking the database from the invalid "after" image back to the valid "before" image.



Forward recovery (REDO / roll-forward) — applied to committed transactions whose updates may not yet have reached the disk, and to media failure. The most recent backup/after-image is taken and the log is scanned **forwards**, reapplying each `new_value`.

last backup / checkpoint	required current state
+-----+ REDO new_values +-----+	
BACKUP or -----> CURRENT	
CHECKPOINT from the log (valid)	
+-----+	+-----+

Backward recovery	Forward recovery
Uses old_value (before image)	Uses new_value (after image)
Log scanned backwards	Log scanned forwards
Applied to uncommitted/aborted transactions	Applied to committed transactions and after a media failure
Enforces Atomicity	Enforces Durability
Result: the transaction's effects are erased	Result: the transaction's effects are reinstated

Q18. Explain the use/role of checkpoint in database recovery.

[2-5 marks · D22 Q1(c)(iii), J23 Q4(a)(iv), J25 Q5(e), J26 Q5(b)]

A **checkpoint** is a point of synchronisation between the database buffers and the log, taken periodically (say every few minutes or every N log records). Taking a checkpoint involves three steps: (1) suspend the start of new updates and force all log records in main memory out to the log file; (2) force all modified (dirty) buffer pages out to the database on disk; (3) write a record `<checkpoint, L>` to the log, where L is the list of transactions active at that instant, and flush it.

Role in recovery. Without checkpoints, restart would have to scan the entire log from its beginning, and would redo updates that are already safely on disk. The checkpoint record guarantees that every change made by a transaction that committed *before* the checkpoint is already in the database. Therefore the restart algorithm needs to scan the log only from the **last checkpoint record** onwards: transactions that committed before it need no redo at all; transactions in L and those started after it are redone if they have a commit record and undone if they do not.

Example — log: ... `<T1,commit>` `<checkpoint,{T2}>` `<T2,X,10,20>` `<T2,commit>` `<T3,start>` `<T3,Y,5,9>` then a crash. T1 is ignored (its work is on disk), T2 is redone ($X = 20$), T3 is undone ($Y = 5$). The benefit is a much shorter restart time; the cost is a brief pause and extra I/O when the checkpoint is taken.

Q19. Deadlock (J26: the wound-wait scheme for deadlocks). [4-5 marks · J22 Q5(d), J26 Q5(a)]

A **deadlock** is a state in which two or more transactions each hold a lock that another needs and each waits for the other, so none can proceed. Example: T1 holds X-lock(A) and requests B; T2 holds X-lock(B) and requests A. The **wait-for graph** (node per transaction, edge $T_i \rightarrow T_j$ if T_i waits for a lock held by T_j) then contains a cycle $T_1 \rightarrow T_2 \rightarrow T_1$. Detection consists of periodically looking for such a cycle and rolling back a victim; prevention uses ordered locking or timestamp-based schemes; a cheap alternative is lock **timeout**.

Wound-wait is a timestamp-based prevention scheme. Each transaction gets a timestamp TS at start; a smaller TS means older. When T_i requests an item locked by T_j :

- If **$TS(T_i) < TS(T_j)$** — T_i is older — T_i **wounds** T_j : T_j is pre-emptively rolled back (and restarted later with its *original* timestamp) so T_i gets the lock.
- If **$TS(T_i) > TS(T_j)$** — T_i is younger — T_i **waits**.

Example: $TS(T1) = 10$, $TS(T2) = 20$. If T1 requests an item locked by T2, T2 is aborted and its locks released. If T2 requests an item locked by T1, T2 waits. Since waiting always goes from younger to older, no cycle can form, so deadlock is impossible; keeping the original timestamp on restart prevents starvation. (The dual scheme, **wait-die**, makes the older transaction wait and kills the younger one instead of pre-empting.)

Q20. Explain the following terms in the context of DBMS, with the help of an example for each:

(i) Transaction (ii) Locking (iii) Checkpoint (iv) Recovery (v) Query cost. [10 marks · D22 Q1(c)]

(i) Transaction — a logical unit of work treated atomically by the DBMS, delimited by BEGIN ... COMMIT/ROLLBACK and obeying the ACID properties. Example: `READ(A); A = A - 500; WRITE(A); READ(B); B = B + 500; WRITE(B); COMMIT` — a fund transfer that must happen entirely or not at all.

(ii) Locking — the concurrency-control mechanism by which a transaction reserves a data item before accessing it. A shared (S) lock permits reading and is compatible with other S locks; an exclusive (X) lock permits writing and is compatible with nothing. Example: T1 holds X-lock(A), so T2's request for A waits until T1 commits — which is what prevents the lost update anomaly.

(iii) Checkpoint — a synchronisation point at which the log buffer and all dirty data pages are forced to disk and a `<checkpoint, L>` record is written. Example: after `<checkpoint, {T2}>`, restart need not redo any transaction that committed earlier, so the recovery scan starts there instead of at the start of the log.

(iv) Recovery — restoring the database to its latest consistent state after a failure, using the log: **redo** every transaction with a commit record using `new_value`, **undo** every transaction without one using `old_value`. Example: log `<T1,X,10,20> <T1,commit> <T2,Y,5,8>` then a crash → X becomes 20, Y is restored to 5.

(v) Query cost — the measure the optimiser uses to compare evaluation plans, stated in **block transfers plus disk seeks** (CPU time is usually ignored). Example: a linear search for `SELECT * FROM Account WHERE balance = 5000` on an unsorted file of $b_r = 1000$ blocks costs 1000 block transfers + 1 seek, whereas a B^+ -tree index of height 3 on `balance` costs about 4 block transfers + 4 seeks — which is why the optimiser chooses the index.

Q21. Explain the following in the context of RDBMS, with the help of an example: (i) Serialisable schedule (ii) Two-phase locking (iii) Backward recovery (iv) Checkpoint (v) Authorisation.

[15 marks · J23 Q4(a)]

(i) Serialisable schedule. An interleaved schedule whose effect on the database equals that of some serial execution of the same transactions; it is the correctness criterion for concurrency control. Tested by the precedence graph — edge $T_i \rightarrow T_j$ for each conflicting pair with T_i first — which must be acyclic. Example: $S = R1(A), W1(A), R2(A), W2(A)$ gives only $T1 \rightarrow T2$, so S is serialisable and equivalent to T1 then T2.

(ii) Two-phase locking. A protocol in which every transaction first acquires all the locks it needs (**growing phase**) and only then begins to release them (**shrinking phase**); no lock may be acquired after the first release. Ordering transactions by their lock points gives the equivalent serial order, so 2PL guarantees serialisability. Example: `X-lock(A); write A; X-lock(B); write B; unlock(A); unlock(B); commit` is legal; releasing A before locking B is not. Strict 2PL keeps all X locks until commit.

(iii) Backward recovery. Undoing the work of transactions that did not commit, by scanning the log backwards and restoring each item to its `old_value`, so the database returns to its before-

image. Example: log `<T2, Y, 700, 900>` with no `<T2, commit>` → Y is reset to 700. It enforces Atomicity, as opposed to forward recovery (redo with `new_value`), which enforces Durability.

(iv) Checkpoint. A periodic synchronisation point at which all log records and all dirty buffer pages are forced to disk and a `<checkpoint, L>` record is written, L being the transactions then active. It shortens restart because the recovery scan may begin at the last checkpoint instead of the start of the log. Example: with `... <T1,commit> <checkpoint,{T2}> ...`, T1 is never redone.

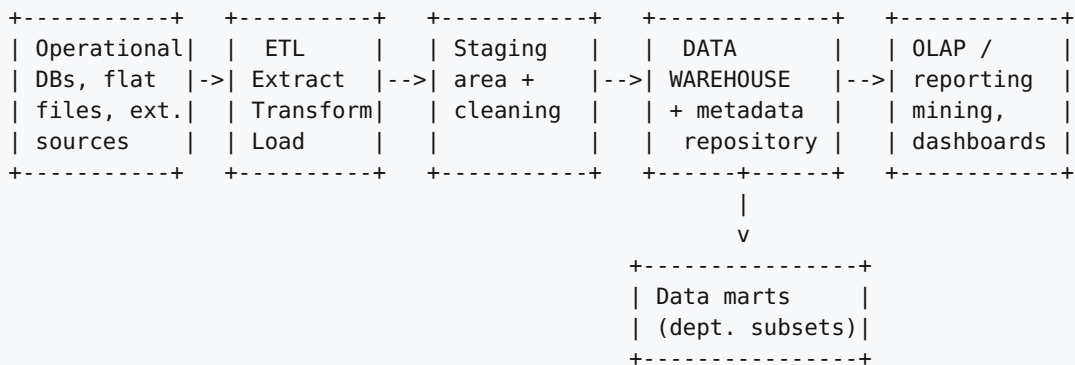
(v) Authorisation. The granting of specific privileges on specific database objects to specific users, enforced by the DBMS after authentication. SQL privileges include SELECT (read), INSERT, UPDATE, DELETE, REFERENCES and EXECUTE, granted with GRANT and withdrawn with REVOKE, optionally with WITH GRANT OPTION for delegation; views are used to restrict access to a subset of rows or columns.

```
GRANT SELECT, UPDATE (balance) ON Account TO clerk1;
REVOKE UPDATE ON Account FROM clerk1;
```

Data warehouse, data mining, object-relational, NoSQL and other advanced topics

Q1. Explain the different components and processes of a data warehouse with the help of a diagram. [5 marks · D23 Q1(f)]

A **data warehouse** is a subject-oriented, integrated, time-variant and non-volatile collection of data organised to support management decision-making.



Components. (1) *Data sources* — operational OLTP databases, legacy files, external feeds. (2) *ETL tools* — extract, transform, load. (3) *Staging area* where cleaning, de-duplication and integration happen. (4) The *warehouse database* itself, held in a star or snowflake schema of fact and dimension tables. (5) The *metadata repository*, describing source-to-target mappings, refresh history and business definitions. (6) *Data marts*, subject subsets for single departments. (7) *Front-end tools* — OLAP servers (ROLAP/MOLAP), query and reporting tools, data mining.

Processes. Extraction from heterogeneous sources → transformation (format conversion, unit harmonisation, derivation, surrogate-key assignment) → cleaning → loading and periodic refresh → indexing and aggregation (materialised summary tables) → querying through OLAP operations such as roll-up, drill-down, slice, dice and pivot.

Q2. Data Warehouse and its characteristics (short note). [4–5 marks · D21 Q5(b), J24 Q5(iv), D24 Q5(b)]

A **data warehouse** is a central repository of integrated historical data drawn from several operational sources and organised for analysis and decision support rather than for transaction processing. W. H. Inmon's definition gives its four characteristics:

- **Subject-oriented** — organised around business subjects (sales, customer, product) rather than around application functions (order entry, billing).
- **Integrated** — data from heterogeneous sources is made consistent in naming, units, encoding and keys; e.g. gender coded M/F in one source and 0/1 in another is stored one way.
- **Time-variant** — every record carries a time element and history is retained, typically 5–10 years, so trends can be analysed; an OLTP database keeps only the current value.
- **Non-volatile** — data is loaded and read, never updated or deleted by users; refreshes append new snapshots.

Further properties: it is **read-mostly** and query-intensive, denormalised into star/snowflake schemas of fact and dimension tables, heavily indexed and aggregated, and large (hundreds of GB to PB). Example: a retail warehouse with a Sales fact table keyed by time, store, product and customer supports "compare quarterly sales by region over the last three years", a query no OLTP system could answer efficiently.

Q3. Explain the concept of multi-dimensional data in a data warehouse, with the help of an example. [2½–6 marks · D22 Q5(b), D24 Q1(d)(ii)]

In the **multi-dimensional data model** the data is viewed as a *data cube*: numeric **measures** (facts) indexed by several **dimensions**, each dimension having a hierarchy of levels. This matches the way analysts think — "sales by product by region by month" — and lets the OLAP engine answer aggregate queries by indexing rather than joining.

- **Fact (measure)** — the quantity analysed: `sales_amount`, `units_sold`, additive over all dimensions.
- **Dimension** — an axis of analysis with descriptive attributes and a hierarchy: Time (day → month → quarter → year), Product (item → sub-category → category), Store (store → city → region → country), Customer.

Example — a three-dimensional cube Sales(Time, Product, Store):

Store \ Product (Q1 2025)	Laptop	Printer	Scanner
Delhi	400	120	60
Mumbai	350	90	45
Chennai	280	110	30

Each cell is `sales_amount` for one (Time, Product, Store) combination. A cube with n dimensions has 2^n possible aggregation sub-cubes.

OLAP operations on the cube: *roll-up* (aggregate from month to quarter), *drill-down* (quarter back to month), *slice* (fix Time = Q1 2025, giving the table above), *dice* (select a sub-cube on two or more dimensions), and *pivot/rotate* (swap the axes of the view). Physically the cube is implemented as a star schema (ROLAP) or as a stored multi-dimensional array (MOLAP).

Q4. Explain the star schema (and snowflake schema) of a data warehouse with the help of an example. [5 marks · J25 Q5(c), D25 Q1(g)]

Star schema — one large central **fact table** holding the measures and the foreign keys to several **dimension tables**, each dimension kept denormalised in a single table. The diagram resembles a star.



Here **SALES_FACT** has the composite key (`time_key`, `product_key`, `store_key`, `customer_key`) and the additive measures `units_sold`, `sales_amount`. Queries need only one join per dimension, so they are fast and simple.

Snowflake schema — the same fact table, but each dimension is **normalised** into a hierarchy of tables. Normalising **Product_Dim**:

```

Product_Dim(product_key, name, brand, subcat_key)
  -> Subcategory(subcat_key, subcat_name, categ_key)
    -> Category(categ_key, categ_name)
  
```

Star schema	Snowflake schema
Dimensions denormalised, one table each	Dimensions normalised into several tables
Some redundancy, more storage	Less redundancy, less storage
Fewer joins → faster queries	More joins → slower queries
Simple for users and tools	More complex to navigate

Q5. Explain the ETL process of a data warehouse. [2 marks · J25 Q1(e)(ii)]

ETL is the three-stage process that populates a data warehouse from its operational sources.

- **Extract** — read the required data from heterogeneous sources (OLTP databases, flat files, external feeds), either as a full extract or, on refresh, as a change-data-capture delta.
- **Transform** — in the staging area, clean and reconcile the data: resolve format and unit differences, standardise codes, remove duplicates, handle missing values, derive calculated columns, assign surrogate keys and build aggregates, so that the result conforms to the warehouse schema.
- **Load** — write the conformed dimension rows and fact rows into the warehouse (initial bulk load, then periodic incremental refresh), and rebuild indexes and materialised aggregates.

Example: daily sales from two branch systems storing amounts in paise and rupees are extracted, converted to a common rupee measure with a surrogate `time_key`, and appended to `SALES_FACT`.

Q6. Data warehousing life cycle (short note). [5 marks · J26 Q5(d)]

The **data warehouse life cycle** (Kimball's business-dimensional lifecycle) is the sequence of phases through which a warehouse is planned, built and maintained:

1. **Project planning and requirement definition** — scope, justification, staffing; interview business users to capture the decision questions the warehouse must answer.
2. **Dimensional modelling** — identify the business processes, choose the grain of each fact table, then the dimensions and the measures; produce the star/snowflake schema and the bus matrix of conformed dimensions.
3. **Physical design** — indexing, partitioning, aggregation/materialised views, storage sizing.
4. **ETL design and development** — source-to-target mapping, extraction, cleaning, transformation, surrogate keys, initial load and incremental refresh jobs.
5. **Technical architecture and product selection** — back-room and front-room architecture, OLAP server (ROLAP/MOLAP), metadata repository.
6. **BI application development** — standard reports, dashboards, ad-hoc query and data mining tools.
7. **Deployment** — user training, documentation, go-live.
8. **Maintenance and growth** — monitoring performance, refreshing data, adding new subject areas; the cycle iterates for each new data mart.

The phases overlap and iterate; the three tracks (data, technology, BI application) run in parallel between requirements and deployment.

Q7. Explain the terms Data Mining and Data Warehousing. How is data warehousing related to database management system ? [8 marks · J22 Q1(d), D22 Q1(d)(ii)]

Data warehousing is the process of building and maintaining a subject-oriented, integrated, time-variant, non-volatile repository of historical data, extracted from several operational sources via ETL and organised as fact and dimension tables for analysis and reporting.

Data mining is the process of discovering previously unknown, valid and useful patterns in large volumes of data by applying statistical and machine-learning algorithms — classification, clustering, association-rule mining, regression, outlier and sequence analysis. It is the analysis step of the wider KDD (Knowledge Discovery in Databases) process.

Data warehousing	Data mining
Stores and organises integrated historical data	Analyses data to extract patterns
Process of consolidation; output is a database	Process of discovery; output is knowledge (rules, models, clusters)
User drives it with known queries (OLAP, reports)	Algorithm finds relationships the user did not know to ask for
Answers "what happened"	Answers "why it happened / what will happen"
Example: monthly sales by region cube	Example: "customers who buy bread and butter also buy jam (support 5%, confidence 70%)"

Relationship with a DBMS. A data warehouse is a database and is implemented on a DBMS (usually a relational one), so it inherits the DBMS's storage management, indexing, query processing, transaction and security services. It differs in purpose and tuning: a conventional DBMS supports **OLTP** — many short read/write transactions, current data, fully normalised schema, concurrency and recovery critical; a warehouse supports **OLAP** — few long read-only queries scanning millions of rows, historical data, denormalised star schema, heavy indexing and pre-aggregation, bulk periodic loads instead of continuous updates. The warehouse takes its data from the operational DBMSs, and modern DBMS products add warehouse-specific features (bitmap indexes, materialised views, partitioning, CUBE/ROLLUP operators, columnar storage).

Q8. Compare and contrast operational data and data of a data warehouse / dimensional table and fact table. [4 marks each · J23 Q5(c), J23 Q5(d)]

(a) Operational data vs data warehouse data

Operational (OLTP) data	Data warehouse data
Supports day-to-day transactions	Supports analysis and decision-making
Current, overwritten on change	Historical, several years retained, non-volatile
Application-oriented	Subject-oriented and integrated
Normalised (3NF), many small tables	Denormalised star/snowflake, few large tables
Read/write; short transactions, high concurrency	Read-mostly; long complex aggregate queries
Detailed records only	Detailed plus pre-computed summaries
Size: MB to GB	Size: GB to PB
Example: UPDATE Account SET balance=... WHERE acc_no=...	Example: average balance per branch per quarter, last 5 years

(b) Dimension table vs fact table

Dimension table	Fact table
Holds descriptive, textual attributes — the context of analysis	Holds numeric measures — the subject of analysis
Small: hundreds to a few million rows	Large: millions to billions of rows
Single-attribute surrogate primary key (product_key)	Composite primary key made of the dimension foreign keys
Denormalised, wide, many attributes	Narrow, few columns, mostly keys plus measures
Grows slowly; rows change (slowly changing dimensions)	Grows continuously by insertion; rows rarely updated
Supplies the WHERE, GROUP BY and label columns	Supplies the SUM/AVG/COUNT columns
Example: Product(product_key, name, brand, category)	Example: Sales_Fact(time_key, product_key, store_key, units_sold, amount)

Q9. Explain classification in the context of data mining.

[2½–6 marks · D22 Q5(c), J23 Q1(d)(iii)+(iv), D24 Q1(d)(iii), J25 Q5(d), D25 Q5(b)]

Classification is a **supervised** data-mining task: given a training set of tuples whose class label is known, build a model (classifier) that predicts the class label of new, unlabelled tuples. It works in two steps — a *learning* step in which the model is induced from the labelled training data, and a *classification* step in which the model is first validated on a held-out test set (measuring accuracy, precision, recall) and then applied to unseen data.

Example — a bank classifies loan applicants as `risk = low` or `high` from training data:

age	income	credit_rating	risk (class)
35	high	excellent	low
22	low	fair	high
48	medium	excellent	low

An induced decision tree might be: `if income = low then high else if credit_rating = fair then high else low`. A new applicant is then routed down the tree to get a predicted class.

Common algorithms: **decision tree induction** (ID3, C4.5, CART, splitting on information gain or Gini index), **naive Bayes**, **k-nearest neighbour**, **support vector machines**, **neural networks** and **rule-based classifiers**. Business uses: credit-risk scoring, fraud detection, churn prediction, medical diagnosis, spam filtering.

Clustering, by contrast, is **unsupervised**: there are no class labels, and the algorithm groups tuples so that intra-cluster similarity is high and inter-cluster similarity is low — e.g. k-means partitioning customers into 4 spending segments, or hierarchical and density-based (DBSCAN) methods. It is used for customer segmentation, outlier detection and as a preprocessing step.

Q10. Compare and contrast classification and clustering in the context of data mining.

[4 marks · J23 Q5(b)]

Classification	Clustering
Supervised learning	Unsupervised learning
Training data carries a known class label	No labels; structure is discovered
Goal: predict the class of a new tuple	Goal: group similar tuples into natural clusters
Number and meaning of classes fixed in advance	Number of groups usually unknown (or a parameter, k)
Needs a separate training and test phase	Single pass over the whole data set
Accuracy measured against known labels (accuracy, precision, recall, confusion matrix)	Quality measured internally (intra-cluster distance, silhouette coefficient)
Algorithms: decision tree (C4.5), naive Bayes, k-NN, SVM, neural network	Algorithms: k-means, k-medoids, hierarchical (AGNES/DIANA), DBSCAN
Example: label a loan applicant as low-risk or high-risk	Example: segment customers into 4 groups by purchase behaviour

Both are predictive/descriptive mining tasks operating on the same kind of tuple data, both depend on a distance or similarity notion, and clustering is often used first to suggest the classes that classification will later predict.

Q11. What is data mining ? How is data mining useful for an organisation ? How is data mining different from knowledge discovery ? [4 marks · J25 Q1(d)]

Data mining is the automated extraction of previously unknown, valid, non-trivial and potentially useful patterns from large data sets, using statistical and machine-learning techniques: classification, clustering, association-rule mining, regression, outlier detection and sequential-pattern analysis.

Usefulness to an organisation — market-basket analysis for store layout and cross-selling; customer segmentation and targeted campaigns; churn prediction and retention; credit scoring and fraud detection; demand forecasting and inventory optimisation; preventive maintenance. In each case it converts data the organisation already owns into a decision it could not otherwise justify.

Data mining vs knowledge discovery (KDD). KDD is the complete process of turning raw data into knowledge, and data mining is only one of its steps. The KDD steps are: data **cleaning** → **integration** → **selection** → **transformation** → **data mining** (apply the algorithms) → **pattern evaluation** (test interestingness) → **knowledge presentation** (visualisation, reports). So KDD is the superset; data mining is the pattern-extraction core within it.

Q12. Data Mining (short note, general). [4-5 marks · D21 Q5(a), D24 Q5(d)]

Data mining is the discovery of hidden, valid and useful patterns in large volumes of data, typically held in a data warehouse. It is the core step of the KDD process (clean → integrate → select → transform → **mine** → evaluate → present).

Principal tasks:

- **Association rule mining** — find co-occurrence rules $X \Rightarrow Y$ with support and confidence; Apriori, FP-growth. Example: {bread, butter} $\Rightarrow$ {jam}, support 5%, confidence 70%.
- **Classification** — supervised prediction of a discrete label; decision trees, naive Bayes, SVM. Example: label a transaction fraudulent / genuine.
- **Clustering** — unsupervised grouping by similarity; k-means, hierarchical, DBSCAN. Example: four customer segments.
- **Regression / prediction** — estimate a continuous value such as next quarter's sales.
- **Outlier and sequence analysis** — detect anomalies and time-ordered patterns.

Applications: retail market-basket analysis, banking credit scoring and fraud detection, telecom churn prediction, medical diagnosis, web usage mining. Issues: data quality, scalability to very large data sets, high dimensionality, over-fitting, interestingness of the patterns found, and **privacy** of the individuals in the data.

Q13. Complex data types in the context of object-relational database systems (short note).

[2-5 marks · D22 Q5(a)(i), J23 Q1(d)(i), J25 Q1(e)(i), D25 Q5(a)]

The relational model requires every attribute value to be **atomic** (first normal form). Many applications — CAD, multimedia, GIS, documents — need attributes that are structured, set-valued or very large. **Complex data types** are the object-relational extension that allows such non-atomic attribute values while keeping SQL and relational storage.

The main kinds are:

- **Composite / structured types** — a value with named sub-fields, e.g. `address(street, city, pin)`, accessed with dot notation.
- **Collection types** — multi-valued attributes: `ARRAY`, `MULTISET`, set and list types, e.g. a book's authors as an ordered array.
- **Large objects** — `BLOB` for binary (image, video) and `CLOB` for character data (a document).
- **Reference types** — `REF(type)`, a pointer to another row object, giving a navigable alternative to a foreign-key join.
- **User-defined types with methods** — a type plus the operations that apply to it, giving encapsulation.

```
CREATE TYPE Address AS (street VARCHAR(40), city VARCHAR(20), pin CHAR(6));
CREATE TYPE Book AS (
    title    VARCHAR(100),
    authors  VARCHAR(40) ARRAY[5],
    pub_addr Address,
    cover    BLOB(2M));
CREATE TABLE library OF Book (PRIMARY KEY (title));
SELECT title, authors[1], pub_addr.city FROM library;
```

Benefit: the structure of the data is kept in the database instead of being flattened into extra tables or hidden in application code.

Q14. Explain the following in the context of object oriented and object relational DBMS: (i) Complex data types (ii) Type inheritance (iii) Object definition language. [8 marks · D22 Q5(a)]

(i) Complex data types. Non-atomic attribute values, which relax 1NF: structured types (`address(street, city, pin)`), collection types (`ARRAY`, `MULTISET`), large objects (`BLOB`, `CLOB`), reference types (`REF`) and user-defined types carrying methods.

```
CREATE TYPE Address AS (street VARCHAR(40), city VARCHAR(20), pin CHAR(6));
CREATE TYPE Person AS (name VARCHAR(40), addr Address, phones VARCHAR(15) ARRAY[3]);
```

(ii) Type inheritance. A sub-type inherits all the attributes and methods of its super-type and may add its own or override a method; instances of the sub-type are substitutable for the super-type. SQL uses `UNDER`, and table inheritance extends this to tables so that a query on the super-table sees sub-table rows too. This models generalisation/specialisation from the E-R model directly.

```
CREATE TYPE Person AS (name VARCHAR(40), dob DATE) NOT FINAL;
CREATE TYPE Student UNDER Person (enrol_no CHAR(9), programme VARCHAR(10));
CREATE TYPE Teacher UNDER Person (emp_id CHAR(6), salary NUMERIC(10,2));
```

`Student` has name, dob, enrol_no and programme. Multiple inheritance (a `Teaching_Assistant` that is both) is allowed in some systems and requires conflict resolution for identically named attributes.

(iii) Object Definition Language (ODL). The ODMG standard, specification-level language for describing an object database **schema** independently of any programming language — the object-oriented counterpart of SQL's DDL. It declares interfaces/classes with their **attributes**, **relationships** (with inverse declarations, so referential integrity is bidirectional), **methods** (signatures only) and **extent/key** clauses; it is then mapped to C++, Java or Smalltalk bindings, while queries are written in the companion **OQL**.

```
class Student (extent students key enrol_no) {
    attribute string enrol_no;
    attribute string name;
    relationship set<Course> takes inverse Course::taken_by;
    short total_credits();
};
```

Q15. Explain the features of object-oriented database management system. How is it different from a relational database management system ?

[7-8 marks · D21 Q1(d), J22 Q3(c), D22 Q1(d)(i), J23 Q5(a)]

An **OODBMS** stores data as **objects**, as in an object-oriented programming language, and provides persistence, concurrency, recovery and querying for them. Its features:

- **Object and object identity (OID)** — every object has a system-generated immutable identity independent of its attribute values, so two objects with identical values are still distinct.
- **Complex objects** — attributes may be structured, set-valued, or references to other objects, nested to any depth.
- **Encapsulation** — state plus the methods that operate on it are stored together; access is through the interface.
- **Class and type hierarchy with inheritance** — single and multiple inheritance, overriding, polymorphism and late binding.
- **Persistence by reachability** — objects reachable from a persistent root survive program termination; there is no impedance mismatch between the program's data structures and the stored ones.
- **Versioning, long transactions and schema evolution** — needed by design applications.
- **Navigational access** — pointer chasing through OIDs, plus declarative OQL.

RDBMS	OODBMS
Data in flat tables of rows and columns	Data as objects with state and behaviour
Attribute values must be atomic (1NF)	Complex, nested, multi-valued attributes allowed
Identity is the value of the primary key	System-generated object identifier (OID)
Relationships through foreign keys, resolved by joins	Relationships through direct object references, resolved by navigation
No inheritance in the basic model	Class hierarchy with inheritance and polymorphism
Behaviour lives in the application (or in stored procedures)	Behaviour encapsulated with the data as methods
Declarative SQL, strong optimiser, mature standards	OQL plus language bindings; weaker optimisation, less standardised
Impedance mismatch with OO host languages	Seamless with the host language
Best for structured business data, high-volume OLTP	Best for CAD/CAM, multimedia, GIS, engineering design

An **object-relational DBMS** (Oracle, PostgreSQL, SQL:1999 onwards) is the compromise: the relational engine and SQL are kept, and complex types, inheritance, reference types and user-defined functions are added on top.

Q16. Justify the need for object oriented databases over relational databases. Explain complex data types, inheritance, object identity and reference types with respect to object relational database systems. [10 marks · D23 Q4(a)]

Need. Relational databases assume atomic attribute values, so any naturally structured or multi-valued item must be flattened into extra tables and reassembled by joins. For CAD drawings, multimedia, GIS maps, documents, bills of material and other recursively nested data, this means (a) expensive multi-way and recursive joins, (b) loss of the semantics of the structure, (c) an **impedance mismatch** between the host language's objects and the stored rows, forcing hand-written mapping code, and (d) no way to store behaviour with the data or to express generalisation hierarchies. There is also no built-in notion of identity independent of key values, and no support for versioning or long design transactions. Object-oriented and object-relational systems remove these limitations.

(i) Complex data types. Non-atomic attribute values — structured types, collections (ARRAY, MULTISET), large objects (BLOB, CLOB) and user-defined types with methods.

```
CREATE TYPE Address AS (street VARCHAR(40), city VARCHAR(20), pin CHAR(6));
CREATE TYPE Book AS (title VARCHAR(100), authors VARCHAR(40) ARRAY[5], addr Address);
```

(ii) Inheritance. A sub-type inherits the attributes and methods of its super-type, may add more and may override methods; sub-type instances are substitutable. Table inheritance extends this so a query on the super-table returns sub-table rows too.

```
CREATE TYPE Person AS (name VARCHAR(40), dob DATE) NOT FINAL;
CREATE TYPE Student UNDER Person (enrol_no CHAR(9));
CREATE TABLE people OF Person;
CREATE TABLE students OF Student UNDER people;
```

(iii) Object identity. Each row object gets a system-generated, immutable **OID** that is unique and independent of its attribute values, so identity (same object) is distinguished from equality (same values). Declared in SQL with `REF IS ... SYSTEM GENERATED`; it survives updates to every attribute, unlike a primary key.

(iv) Reference types. `REF(T)` is an attribute whose value is the OID of a row object of type `T` — a typed, system-maintained pointer. It replaces a foreign key plus join with direct navigation, dereferenced with `->` or `DEREF`.

```
CREATE TABLE department OF Dept (REF IS dept_oid SYSTEM GENERATED);
CREATE TYPE Emp AS (name VARCHAR(40), works_in REF(Dept) SCOPE department);
SELECT e.name, e.works_in->dname FROM employee e;
```

Q17. What are the limitations of Relational Databases ? Explain the need for object oriented Databases. [5 marks · J24 Q1(d), D24 Q1(d)(i)]

Limitations / weaknesses of the relational model

- Attribute values must be **atomic** (1NF), so structured and multi-valued data must be split into extra tables.
- Reconstructing an object requires **many joins**, which is expensive for deeply nested data.
- **No inheritance** or generalisation hierarchy in the basic model.
- **No encapsulated behaviour** — operations live in the application, not with the data.
- **Impedance mismatch** with object-oriented host languages; a mapping layer must be written and maintained.
- **Identity is value-based** (the primary key), so an object's identity changes if its key changes.
- Poor support for **large unstructured objects** (images, video), **spatial and temporal** data, recursive structures (bill of materials) and **versioning / long transactions**.
- Limited type extensibility — only the vendor's built-in types.

Need for object-oriented databases. Applications such as CAD/CAM, GIS, multimedia, document management, engineering design and scientific data have exactly the characteristics the relational model handles badly. An OODBMS stores objects directly with complex nested state, system-generated OIDs, class hierarchies with inheritance and polymorphism, encapsulated methods, navigational access by reference instead of joins, and persistence that matches the programming language's own data structures — eliminating the mapping code and making retrieval of a complex object a pointer traversal rather than a join tree. The pragmatic middle ground, the **object-relational** DBMS, adds these features to SQL rather than abandoning it.

Q18. Object-relational database system (short note). [5 marks · D24 Q5(a)]

An **object-relational DBMS (ORDBMS)** keeps the relational model, SQL and the mature relational query optimiser, and extends them with object-oriented features. It was standardised in **SQL:1999** and is what Oracle, PostgreSQL, DB2 and SQL Server implement, so it is the practical answer to the relational model's weakness on complex data without the cost of abandoning SQL.

Features added over a pure RDBMS:

- **User-defined and complex types** — structured types, `ARRAY` and `MULTISET` collections, `BLOB`/`CLOB`, created with `CREATE TYPE`.

- **Type and table inheritance** with `UNDER`, substitutability and method overriding.
- **Object identity** — row objects with system-generated OIDs, and **reference types** `REF(T)` for navigation instead of joins.
- **Methods and user-defined functions**, including table functions and user-defined aggregates and operators.
- **Extensible indexing** so a user-defined type can have its own access method (R-tree for spatial data).

```
CREATE TYPE Address AS (street VARCHAR(40), city VARCHAR(20), pin CHAR(6));
CREATE TYPE Emp AS (name VARCHAR(40), addr Address, skills VARCHAR(20) ARRAY[5]) NOT FINAL;
CREATE TABLE employee OF Emp (PRIMARY KEY (name));
SELECT name, addr.city FROM employee WHERE 'SQL' IN (UNNEST(skills));
```

Advantage: backward-compatible with existing relational applications and tools. Drawback: more complex type system, and queries over complex types are harder to optimise than pure relational ones.

Q19. NoSQL databases (short note). [4–5 marks · D21 Q5(c), J23 Q5(e), D25 Q5(c)]

NoSQL ("not only SQL") databases are non-relational stores designed for very large, rapidly changing, often semi-structured web-scale data. They drop the fixed schema, full normalisation and (usually) full ACID in exchange for **horizontal scalability** across commodity servers, high availability and flexible data models. By the **CAP theorem** a partitioned distributed store must choose between consistency and availability; most NoSQL systems choose availability with **BASE** semantics (Basically Available, Soft state, Eventually consistent).

Four families:

Type	Model	Examples	Suited to
Key-value	opaque value under a key	Redis, DynamoDB	caches, sessions, profiles
Document	self-describing JSON/BSON documents	MongoDB, CouchDB	content, catalogues, event data
Column-family	rows with sparse, grouped columns	Cassandra, HBase	time series, very wide sparse data
Graph	nodes + typed edges with properties	Neo4j, JanusGraph	networks, recommendations, fraud rings

Graph-based NoSQL stores data as **nodes** (entities with properties), **edges** (typed, directed, property-carrying relationships) and uses **index-free adjacency**, so traversing a relationship is a pointer hop rather than a join. Queries are written in a traversal language such as Cypher: `MATCH (a:Person {name:'Ravi'})-[:FRIEND*2]->(f) RETURN f.name` finds friends-of-friends. It is the right choice when relationships themselves are the data — social networks, recommendation engines, fraud detection, network topology — queries that would need many self-joins in SQL.

RDBMS	NoSQL
Fixed schema, normalised tables	Schema-less / flexible schema
SQL, joins, strong ACID transactions	API or dialect per product, few or no joins, BASE/eventual consistency
Scales vertically (bigger server)	Scales horizontally (sharding, replication)
Structured business data, complex queries	Semi-structured, huge volume, simple access patterns

Q20. What are the features of 'Document-based' NoSQL database ? Explain with the help of an example. [5 marks · D25 Q1(h)]

A **document-based NoSQL database** (MongoDB, CouchDB) stores data as self-describing **documents** — JSON/BSON or XML trees — grouped into **collections**, which are the analogue of tables but without a fixed schema.

Features:

- **Schema-less collections** — documents in one collection may have different fields, so new attributes need no ALTER TABLE.
- **Self-describing, nested documents** — embedded sub-documents and arrays hold the whole aggregate in one place, so a read needs no join; this is denormalisation by design.
- **Unique document key** (`_id`) and secondary indexes on any field, including fields inside arrays and sub-documents.
- **Rich query API** on document content — field predicates, range queries, projection, aggregation pipelines, map-reduce.
- **Horizontal scale-out** by sharding on a key, with replica sets for availability; atomicity is guaranteed at the single-document level.

Example — a student document:

```
{
  "_id": "2301234567",
  "name": "Ravi Kumar",
  "programme": "MCA",
  "address": { "city": "Delhi", "pin": "110001" },
  "courses": [
    { "code": "MCS-207", "marks": 78 },
    { "code": "MCS-208", "marks": 65 }
  ]
}
```

The nested `address` and the array `courses` would need three normalised tables and two joins in SQL; here one read returns everything. Query: `db.student.find({ "courses.code": "MCS-207" }, { name: 1 })`. A second student document may simply omit `address` or add a `phone` field.

Q21. Multimedia database and the challenges in designing it / applications of multimedia database. [2½–5 marks · D24 Q1(d)(iv), J25 Q5(a)]

A **multimedia database** stores and manages media objects — text, images, audio, video, animation and graphics — together with their metadata, and supports retrieval by content as well as by attribute.

Challenges in design

- **Huge object size** — a video is gigabytes; needs BLOB storage, compression (JPEG/MPEG), hierarchical and tertiary storage.
- **Content-based retrieval** — queries like "find images similar to this one" need feature extraction (colour histogram, texture, shape) and *similarity* search returning ranked approximate matches, not exact matches; this requires multi-dimensional indexes (R-tree, KD-tree) rather than B-trees.
- **Real-time / continuous delivery** — audio and video must be streamed at a guaranteed rate with bounded jitter, so the DBMS needs QoS-aware buffering and admission control.
- **Temporal and spatial synchronisation** — audio with video, subtitles with frames.
- **Heterogeneity and format dependence** — many formats and resolutions to be integrated.
- **Imprecise semantics** — the gap between low-level features and the user's meaning ("a happy crowd").
- **Updating, versioning, indexing cost, copyright and access control** on large media.

Applications — digital libraries and e-learning repositories; video-on-demand and news archives; medical imaging (PACS: X-ray, MRI, CT with patient records); GIS and satellite imagery; biometric identification (fingerprint, face, iris); CAD/CAM and product catalogues; video surveillance and forensic retrieval; social-media and entertainment content platforms.

Q22. Geographic Information Systems (short note). [2–5 marks · D24 Q5(c), J25 Q1(e)(iii)]

A **Geographic Information System (GIS)** is a system for capturing, storing, querying, analysing and displaying **spatially referenced** data — data tied to locations on the earth's surface. A spatial DBMS underpins it (Oracle Spatial, PostGIS).

- **Data models.** *Vector* — points (a well), lines (a road), polygons (a district) with coordinates; *raster* — a grid of cells with a value per cell (satellite image, elevation). Data is organised in thematic **layers** (roads, soil, population, land use) that are overlaid.
- **Attribute data** — ordinary relational tuples joined to the geometry: `District(d_id, name, population, boundary POLYGON)`.
- **Spatial data types and operators** — area, length, distance, centroid; topological predicates `contains`, `overlaps`, `touches`, `within`; set operations `intersection` and `union`; nearest-neighbour and range queries. Example: `SELECT name FROM hospital h, city c WHERE ST_Within(h.loc, c.area) AND c.name='Delhi';`
- **Spatial indexing** — R-trees, quad-trees, grid files, since B-trees cannot index two-dimensional extents.
- **Analysis** — buffer, overlay, network/shortest-path, interpolation, terrain and visibility analysis.

Applications: urban and infrastructure planning, utility and telecom network management, vehicle navigation and logistics, disaster management, epidemiology, agriculture and forestry, defence, and location-based services. Challenges: volume of data, the cost of spatial joins, coordinate-system and projection conversion, and currency of the map data.

Q23. Compare and contrast the features of the following with the help of an example: (a) OODBMS and RDBMS (b) classification and clustering (c) dimensional table and fact table (d) operational data and data of a data warehouse (e) NoSQL database and RDBMS. [20 marks · J23 Q5]

(a) OODBMS vs RDBMS

RDBMS	OODBMS
Data in flat relations; values atomic (1NF)	Data as objects with nested, multi-valued state
Identity = primary key value	System-generated object identifier (OID)
Relationships via foreign keys + joins	Relationships via object references, traversed by navigation
No inheritance, no methods	Class hierarchy, inheritance, polymorphism, encapsulated methods
SQL, mature optimiser, standardised	OQL + language bindings, weaker optimisation
Impedance mismatch with OO languages	Seamless persistence of program objects
Business/OLTP data	CAD, multimedia, GIS, design data

Example: a bill-of-materials part containing sub-parts needs a recursive join in SQL; in an OODBMS `part.subparts` is a set-valued attribute traversed directly.

(b) Classification vs clustering

Classification	Clustering
Supervised; training data labelled	Unsupervised; no labels
Predicts a known class for a new tuple	Discovers natural groups
Classes fixed in advance	Groups (and often their number) unknown
Train/test phases; accuracy measurable	One pass; quality measured internally
C4.5 decision tree, naive Bayes, k-NN, SVM	k-means, hierarchical, DBSCAN

Example: classification labels a loan applicant `low-risk` from past labelled applicants; clustering splits the same customer base into four unlabelled spending segments.

(c) Dimension table vs fact table

Dimension table	Fact table
Descriptive textual attributes (context)	Numeric measures (the subject)
Small, denormalised, wide	Very large, narrow
Single surrogate primary key	Composite key of the dimension FKs
Changes slowly	Grows by continuous insertion
Used in WHERE / GROUP BY	Used in SUM / AVG / COUNT

Example: `Product(product_key, name, brand, category)` vs `Sales_Fact(time_key, product_key, store_key, units_sold, amount)`.

(d) Operational data vs data warehouse data

Operational	Warehouse
Runs the business, day to day	Supports decisions
Current, overwritten	Historical, non-volatile, time-stamped
Application-oriented, normalised	Subject-oriented, integrated, denormalised
Short read/write transactions, high concurrency	Long read-only aggregate queries
Detail only, MB-GB	Detail + summaries, GB-PB

Example: `UPDATE Account SET balance = 5200 WHERE acc_no = 'A101'` vs "average balance per branch per quarter for the last five years".

(e) NoSQL vs RDBMS

RDBMS	NoSQL
Fixed, pre-declared schema	Schema-less / flexible
Normalised tables, joins	Denormalised aggregates, no joins
Full ACID transactions	BASE, eventual consistency (CAP trade-off)
Vertical scaling	Horizontal scaling by sharding and replication
SQL, one standard	Product-specific APIs; key-value, document, column-family, graph
Complex ad-hoc queries, strong integrity	Huge volume, simple access patterns, high write rate

Example: a student with nested address and a variable list of courses is one MongoDB document (`db.student.find({"courses.code":"MCS-207"})`), but three normalised tables and two joins in SQL.

Q24. Explain the basic features of: (i) Complex data type in context to DBMS (ii) Data warehouse (iii) Classification in the context of data mining (iv) Clustering in the context of data mining. [10 marks · J23 Q1(d)]

(i) Complex data type. An attribute whose value is not atomic, allowed by object-relational DBMSs to escape the 1NF restriction: structured types (`Address(street, city, pin)`), collections (`ARRAY`, `MULTISET`), large objects (`BLOB`, `CLOB`), reference types `REF(T)`, and user-defined types with methods. Declared with `CREATE TYPE Book AS (title VARCHAR(100), authors VARCHAR(40) ARRAY[5], cover BLOB(2M))`; and accessed with dot and subscript notation. Benefit: the structure stays in the database, so nested data needs no decomposition into extra tables and no joins to reassemble.

(ii) Data warehouse. A **subject-oriented, integrated, time-variant, non-volatile** repository of historical data, built from several operational sources by ETL and organised for analysis. Architecture: sources → ETL → staging/cleaning → warehouse (star or snowflake schema of fact and dimension tables) + metadata → data marts → OLAP, reporting and mining tools. It is read-mostly, denormalised, heavily indexed and pre-aggregated, and supports OLAP operations roll-up, drill-down, slice, dice and pivot on a multi-dimensional cube.

(iii) Classification. A **supervised** mining task: induce a model from tuples whose class label is known, then use it to predict the label of new tuples. Two phases — learning from the training set, then testing on held-out data for accuracy before deployment. Algorithms: decision tree (C4.5, CART), naive Bayes, k-NN, SVM, neural networks. Example: from past applicants labelled low-/high-risk, a decision tree `if income = low then high else low` classifies a new loan applicant. Used for credit scoring, fraud detection, churn prediction, diagnosis.

(iv) Clustering. An **unsupervised** mining task: partition tuples into groups so that similarity within a cluster is maximised and between clusters minimised, using a distance measure; there are no class labels. Methods: partitioning (k-means, k-medoids), hierarchical (agglomerative/divisive), density-based (DBSCAN), grid-based. Example: k-means with $k = 4$ on (annual spend, visit frequency) yields four customer segments the business did not know existed. Used for customer segmentation, outlier/fraud detection, document grouping and as preprocessing for classification.

Q25. Explain the following: (i) Weakness of RDBMS (ii) Multi-dimensional data model of a data warehouse (iii) Classification in the context of data mining (iv) Applications of multimedia database. [10 marks ($2\frac{1}{2} \times 4$) · D24 Q1(d)]

(i) Weakness of RDBMS. Attribute values must be atomic, so structured, multi-valued and nested data must be split across tables and rebuilt by expensive joins; there is no inheritance, no encapsulated behaviour, and identity is value-based (the primary key) rather than intrinsic. Poor support for large unstructured objects (image, video), spatial, temporal and recursive data, versioning and long design transactions, plus an impedance mismatch with object-oriented host languages that forces hand-written mapping code. Type extensibility is limited to the vendor's built-in types.

(ii) Multi-dimensional data model. Data viewed as a **cube**: numeric **measures** (sales_amount, units_sold) indexed by several **dimensions** (Time, Product, Store, Customer), each with a hierarchy (day $\rightarrow$ month $\rightarrow$ quarter $\rightarrow$ year). A cell holds the measure for one combination of dimension values. OLAP operations navigate the cube: **roll-up** (aggregate up a hierarchy), **drill-down** (the reverse), **slice** (fix one dimension), **dice** (sub-cube on several dimensions), **pivot** (rotate the axes). It is implemented relationally as a star/snowflake schema (ROLAP) or as a stored array (MOLAP).

(iii) Classification. Supervised prediction of a discrete class label. A model is learned from labelled training tuples, validated on a test set (accuracy, precision, recall) and then applied to unlabelled data. Algorithms: decision trees (C4.5, CART, splitting by information gain or Gini index), naive Bayes, k-NN, SVM, neural networks. Example: a bank learns if income = low then risk = high from past applicants and scores new ones. Uses: credit risk, fraud detection, churn prediction, medical diagnosis, spam filtering.

(iv) Applications of multimedia database. Digital libraries and e-learning content repositories; video-on-demand, news and broadcast archives; medical imaging systems (PACS) holding X-ray, CT and MRI with patient records; GIS and remote-sensing imagery; biometric identification from fingerprint, face and iris images; CAD/CAM drawings and illustrated product catalogues; video surveillance and forensic search; entertainment and social-media platforms. All need content-based similarity retrieval, BLOB storage with compression, multi-dimensional indexing and real-time streaming.

Normalisation and functional dependencies

Q1. Consider the relation $R = \{A, B, C, D, E\}$ and the FDs $A \rightarrow BC$, $B \rightarrow E$, $C \rightarrow D$. (i) What is the key of R ? (ii) Decompose R into 2NF and 3NF.

[8 marks · D21 Q4(c); J22 Q2(a) with $R = \{A, B, C, D, E, F\}$ and $A \rightarrow BD$; $B \rightarrow E$; $D \rightarrow CF$]

Version 1: $R(A, B, C, D, E)$, $F = \{A \rightarrow BC, B \rightarrow E, C \rightarrow D\}$

(i) Key. Compute closures. A appears on no right-hand side, so A must be in every candidate key.

```

A+ = A
A→BC  ⇒ A+ = A B C
B→E   ⇒ A+ = A B C E
C→D   ⇒ A+ = A B C D E = R

```

$A^+ = R$, and no proper subset of $\{A\}$ determines R , so **A is the only candidate key** and the primary key. Prime attribute: A. Non-prime: B, C, D, E.

(ii) 1NF — assume all values atomic, so R is in 1NF.

2NF. A partial dependency is a dependency of a non-prime attribute on a *proper subset* of a candidate key. The key here is the single attribute A , which has no proper non-empty subset, so **no partial dependency can exist and R is already in 2NF**. (R is left as $R(A, B, C, D, E)$ with key A .)

3NF. A relation is in 3NF if for every non-trivial FD $X \rightarrow Y$, either X is a superkey or every attribute of Y is prime. Check:

FD	X a superkey?	Y prime?	Verdict
$A \rightarrow BC$	yes (A is the key)	—	OK
$B \rightarrow E$	no, $B^+ = BE \neq R$	E is non-prime	violates 3NF (transitive: $A \rightarrow B \rightarrow E$)
$C \rightarrow D$	no, $C^+ = CD \neq R$	D is non-prime	violates 3NF (transitive: $A \rightarrow C \rightarrow D$)

Decompose by removing each transitive dependency into its own relation:

```

R1(A, B, C)    key A    FD: A→BC
R2(B, E)       key B    FD: B→E
R3(C, D)       key C    FD: C→D

```

Every determinant (A, B, C) is now the key of its relation, so R_1, R_2, R_3 are in 3NF — and in fact in BCNF. The decomposition is lossless ($R_1 \cap R_2 = \{B\}$, a key of R_2 ; $R_1 \cap R_3 = \{C\}$, a key of R_3) and dependency-preserving (all three FDs are retained).

Version 2: $R(A, B, C, D, E, F)$, $F = \{A \rightarrow BD, B \rightarrow E, D \rightarrow CF\}$

(i) Key.

```

A+ = A
A→BD  ⇒ A+ = A B D
B→E   ⇒ A+ = A B D E
D→CF  ⇒ A+ = A B C D E F = R

```

So **A is the only candidate key**.

(ii) Key is a single attribute $\Rightarrow$ no partial dependency $\Rightarrow$ **R is already in 2NF**. 3NF is violated by the transitive dependencies $A \rightarrow B \rightarrow E$ and $A \rightarrow D \rightarrow CF$, since B and D are not superkeys and E, C, F are non-prime. 3NF (and BCNF) decomposition:

```

R1(A, B, D)    key A    FD: A→BD
R2(B, E)       key B    FD: B→E
R3(D, C, F)    key D    FD: D→CF

```

Lossless ($R_1 \cap R_2 = \{B\} = \text{key of } R_2$; $R_1 \cap R_3 = \{D\} = \text{key of } R_3$) and dependency-preserving.

Q2. Given Student(student_id, name, coursecode, coursename, marks) with the stated constraints: (i) List all the functional dependencies in the relation (ii) What are the anomalies in the relation ? Explain (iii) Normalize the relation into 2NF and then 3NF. [15 marks · D22 Q3(a)]

Relation: Student(student_id, name, coursecode, coursename, marks). Assumptions: a student has one name; a course code has one course name; a student may register for many courses and a course has many students; marks are awarded per student per course.

Sample instance:

student_id	name	coursecode	coursename	marks
S01	Ravi	MCS-207	DBMS	78
S01	Ravi	MCS-208	Data Structures	65
S02	Meena	MCS-207	DBMS	82
S03	Anil	MCS-208	Data Structures	55

(i) Functional dependencies

FD1: student_id → name
FD2: coursecode → coursename
FD3: student_id, coursecode → marks
(hence student_id, coursecode → name, coursename, marks)

Candidate key. (student_id, coursecode)⁺ = {student_id, coursecode, name, coursename, marks} = R, and neither attribute alone suffices (student_id⁺ = {student_id, name}; coursecode⁺ = {coursecode, coursename}). So the **primary key is (student_id, coursecode)**; prime attributes student_id, coursecode; non-prime name, coursename, marks.

(ii) Anomalies — all caused by storing student facts and course facts redundantly in every enrolment row.

- **Insertion anomaly.** A new course MCS-209 "Operating Systems" that no student has yet registered for cannot be stored: coursecode is part of the key but student_id would have to be NULL, violating entity integrity. Likewise a newly admitted student with no course yet cannot be recorded.
- **Update anomaly.** "DBMS" appears in every row for MCS-207. Renaming it to "Database Management Systems" requires updating all such rows; if one is missed the database holds two names for one course — an inconsistency.
- **Deletion anomaly.** Deleting the single row (S03, MCS-208, 55) because Anil has withdrawn also destroys the only record that student S03 is named Anil. Deleting every MCS-207 row loses the fact that the course exists.

(iii) Normalisation

1NF — all attribute values are atomic and there is no repeating group, so R is in 1NF.

2NF — 2NF requires that no non-prime attribute be partially dependent on the key. Two partial dependencies exist:

- FD1 student_id → name — a proper subset of the key determines the non-prime name.
- FD2 coursecode → coursename — a proper subset of the key determines the non-prime coursename.

Remove each into its own relation, leaving the fully dependent attribute with the whole key:

STUDENT(student_id, name)	PK: student_id
COURSE (coursecode, coursename)	PK: coursecode
MARKS (student_id, coursecode, marks)	PK: (student_id, coursecode)
	FK: student_id -> STUDENT
	FK: coursecode -> COURSE

STUDENT		COURSE		MARKS		
S01	Ravi	MCS-207	DBMS	S01	MCS-207	78
S02	Meena	MCS-208	Data Structures	S01	MCS-208	65
S03	Anil			S02	MCS-207	82
				S03	MCS-208	55

3NF — check each relation for a transitive dependency (a non-prime attribute determined by a non-superkey):

- STUDENT: only FD is student_id→name, and student_id is the key. In 3NF.
- COURSE: only FD is coursecode→coursename, and coursecode is the key. In 3NF.
- MARKS: only FD is (student_id, coursecode)→marks, the full key. In 3NF.

So the 2NF decomposition is **already in 3NF** (and in BCNF, since every determinant is a candidate key). All three anomalies are gone: a new course is one COURSE row, renaming DBMS touches exactly one row, and deleting a MARKS row leaves the student and the course intact. The decomposition is lossless (STUDENT ∩ MARKS = {student_id}, a key of STUDENT; COURSE ∩ MARKS = {coursecode}, a key of COURSE) and dependency-preserving.

Q3. Given the schema (Customer_ID, Customer_name, Account_No, Balance, type of account) with the stated constraints: (i) List all the FDs (ii) What is the primary key ? (iii) What are the different anomalies in the relation ? Show with the help of example data (iv) Normalize the relation to 2NF and then to 3NF. [17 marks · J23 Q3(a)]

Relation: CustAcc(Customer_ID, Customer_name, Account_No, Balance, acc_type). Assumptions: a customer may hold several accounts and an account may be jointly held by several customers; each customer has one name; each account has one balance and one type.

Instance:

Customer_ID	Customer_name	Account_No	Balance	acc_type
C01	Ravi	A101	5000	Savings
C01	Ravi	A102	12000	Current
C02	Meena	A101	5000	Savings
C03	Anil	A103	800	Savings

(i) Functional dependencies

FD1: Customer_ID	-> Customer_name
FD2: Account_No	-> Balance, acc_type
FD3: Customer_ID, Account_No	-> Customer_name, Balance, acc_type

(ii) Primary key. (Customer_ID, Account_No)⁺ = all five attributes = R; Customer_ID⁺ = {Customer_ID, Customer_name} ≠ R and Account_No⁺ = {Account_No, Balance, acc_type} ≠ R, so neither part alone is a key. **PK = (Customer_ID, Account_No)** — a composite key, needed

because of joint accounts. Prime: Customer_ID, Account_No. Non-prime: Customer_name, Balance, acc_type.

(iii) Anomalies, with the example data

- **Insertion anomaly.** Open account A104 (Savings, 2000) before any holder is assigned: the row would be (NULL, NULL, A104, 2000, Savings) and Customer_ID is part of the primary key, so the insert is rejected. Equally, customer C04 "Sita" who has not yet opened an account cannot be stored.
- **Update anomaly.** A101's balance 5000 is stored twice (rows 1 and 3, because it is a joint account of C01 and C02). A deposit of 1000 must update both rows; updating only row 1 leaves the database claiming A101 holds 6000 for Ravi and 5000 for Meena — an inconsistency for one physical account.
- **Deletion anomaly.** Deleting row 4 because Anil closed A103 also erases the only record that C03 is named Anil. Conversely, deleting C01's two rows loses the existence of account A102 altogether.

(iv) Normalisation

1NF — all values atomic; satisfied.

2NF — remove the partial dependencies FD1 (Customer_ID → Customer_name) and FD2 (Account_No → Balance, acc_type), each of which is on a proper subset of the key:

CUSTOMER(Customer_ID, Customer_name)	PK: Customer_ID
ACCOUNT (Account_No, Balance, acc_type)	PK: Account_No
HOLDS (Customer_ID, Account_No)	PK: (Customer_ID, Account_No)
	FK: Customer_ID -> CUSTOMER
	FK: Account_No -> ACCOUNT

CUSTOMER		ACCOUNT			HOLDS	
C01	Ravi	A101	5000	Savings	C01	A101
C02	Meena	A102	12000	Current	C01	A102
C03	Anil	A103	800	Savings	C02	A101
					C03	A103

3NF — in CUSTOMER the only determinant is Customer_ID (the key); in ACCOUNT it is Account_No (the key); HOLDS is all-key with no non-prime attribute at all. No non-prime attribute is determined by a non-superkey, so there is no transitive dependency and all three relations are in **3NF** (and BCNF). A104 can now be inserted into ACCOUNT alone, a balance change touches one ACCOUNT row, and closing an account deletes HOLDS and ACCOUNT rows without losing customer data.

Q4. Given Student(Enrolmentno, name, coursecode, coursename, grade) with enrolmentno→name, coursecode→coursename, (enrolmentno, coursename)→grade: normalise the relation into second normal form. Also find the primary key of the student relation.

[5 marks · D25 Q1(c)]

Primary key. Test (enrolmentno, coursecode):

```
(enrolmentno, coursecode)+ = enrolmentno, coursecode
  enrolmentno -> name          => + name
  coursecode  -> coursename    => + coursename
  (enrolmentno, coursename) -> grade => + grade
= {enrolmentno, name, coursecode, coursename, grade} = R
```

Neither attribute alone closes to R ($\text{enrolmentno}^+ = \{\text{enrolmentno}, \text{name}\}$; $\text{coursecode}^+ = \{\text{coursecode}, \text{coursename}\}$). So the **primary key is the composite (enrolmentno, coursecode)**. ((enrolmentno, coursename) determines grade but cannot derive coursecode, so it is not a key.)

2NF. Prime attributes are enrolmentno and coursecode; non-prime are name, coursename, grade. Two partial dependencies on proper subsets of the key violate 2NF:

- enrolmentno → name
- coursecode → coursename

Removing them gives:

STUDENT(enrolmentno, name)	PK: enrolmentno
COURSE (coursecode, coursename)	PK: coursecode
GRADE (enrolmentno, coursecode, grade)	PK: (enrolmentno, coursecode)
	FK: enrolmentno -> STUDENT
	FK: coursecode -> COURSE

All three are in 2NF; since every determinant is now the whole key of its relation, they are in 3NF and BCNF as well. The decomposition is lossless and preserves all three FDs (the third becomes (enrolmentno, coursecode)→grade, equivalent given coursecode→coursename).

Q5. Explain 1NF, 2NF and 3NF with the help of appropriate examples.

[2-10 marks · D21 Q1(e), D21 Q3(c), D23 Q2(b), J24 Q5(ii)]

Normalisation is the stepwise decomposition of a relation to remove redundancy and the insertion, update and deletion anomalies it causes, without losing information.

First normal form (1NF). A relation is in 1NF if every attribute is **single-valued and atomic** — no repeating groups, no composite or set-valued attributes. It is the minimum requirement of the relational model.

Not in 1NF:

emp_id	name	phones
E1	Ravi	9810011111, 9810022222

In 1NF (flatten, or better, move the multi-valued attribute to its own relation):

emp_id	name		emp_id	phone
E1	Ravi		E1	9810011111
			E1	9810022222

Second normal form (2NF). A relation is in 2NF if it is in 1NF and **every non-prime attribute is fully functionally dependent on the whole of every candidate key** — i.e. there is no *partial* dependency on a proper subset of a key. (A relation whose key is a single attribute is automatically in 2NF.)

Violation — Enrol(student_id, coursecode, coursename, marks) with key (student_id, coursecode) and FD coursecode → coursename: coursename depends on part of the key, so coursename is repeated in every enrolment of that course and renaming the course means updating many rows.

2NF decomposition:

COURSE(coursecode, coursename)	PK coursecode
ENROL (student_id, coursecode, marks)	PK (student_id, coursecode), FK coursecode

Third normal form (3NF). A relation is in 3NF if it is in 2NF and no non-prime attribute is **transitively dependent** on a candidate key — formally, for every non-trivial FD $X \rightarrow Y$, either X is a superkey or every attribute in Y is prime.

Violation — Student(enrol_no, name, prog_code, prog_fee) with key enrol_no and FDs enrol_no → prog_code, prog_code → prog_fee. Then enrol_no → prog_code → prog_fee is transitive, so prog_fee is repeated for every student in the programme and changing the MCA fee means updating hundreds of rows; a programme with no student enrolled cannot have its fee stored at all.

3NF decomposition:

STUDENT (enrol_no, name, prog_code)	PK enrol_no, FK prog_code
PROGRAMME(prog_code, prog_fee)	PK prog_code

The fee is now stored once, and the original relation is recoverable by a lossless join on prog_code.

Q6. Define the third-normal form with the help of an example. [5 marks · D25 Q2(b)]

A relation R is in **third normal form (3NF)** if it is in 2NF and no **non-prime** attribute is transitively dependent on any candidate key. Formally: for every non-trivial functional dependency $X \rightarrow Y$ in R , at least one of the following holds — (a) X is a superkey of R , or (b) every attribute of Y — X is a prime attribute (part of some candidate key). Equivalently, 3NF forbids a chain $\text{key} \rightarrow X \rightarrow Y$ where X is not a superkey and Y is non-prime.

Example of a violation. Employee(emp_id, emp_name, dept_no, dept_location) with FDs

emp_id → emp_name, dept_no	(emp_id is the only candidate key)
dept_no → dept_location	(dept_no is not a superkey; dept_location is non-prime)

emp_id	emp_name	dept_no	dept_location
E1	Ravi	D10	Delhi
E2	Meena	D10	Delhi
E3	Anil	D20	Mumbai

It is in 2NF (the key is a single attribute, so no partial dependency), but $\text{emp_id} \rightarrow \text{dept_no} \rightarrow \text{dept_location}$ is transitive, so 3NF is violated. The consequences: D10's location is repeated in every employee row (**redundancy**); moving D10 to Noida requires every such row to be changed (**update anomaly**); a department with no employees cannot be recorded (**insertion anomaly**); deleting E3 loses the fact that D20 is in Mumbai (**deletion anomaly**).

3NF decomposition — put the transitively dependent attribute with its determinant:

EMPLOYEE (emp_id, emp_name, dept_no)	PK emp_id, FK dept_no -> DEPARTMENT
DEPARTMENT(dept_no, dept_location)	PK dept_no

Now every determinant (emp_id, dept_no) is the key of its own relation, so both relations are in 3NF; the decomposition is lossless (common attribute dept_no is the key of DEPARTMENT) and preserves both FDs.

Q7. Explain the third normal form and BCNF with the help of an example. What are trivial functional dependencies and transitive functional dependencies ? How are they related to 3NF and BCNF ? [7-10 marks · J24 Q3(b), J26 Q2(a)]

Trivial functional dependency. $X \rightarrow Y$ is **trivial** if $Y \subseteq X$ — it holds in every relation regardless of the data, e.g. $\{\text{enrol_no}, \text{name}\} \rightarrow \text{name}$ or $\text{enrol_no} \rightarrow \text{enrol_no}$. Trivial FDs carry no information, so every normal-form test is stated "for every **non-trivial** FD"; they can never cause a violation.

Transitive functional dependency. If $X \rightarrow Y$ and $Y \rightarrow Z$ hold and Y does not determine X , then $X \rightarrow Z$ holds **transitively** (Armstrong's transitivity rule). Example: $\text{enrol_no} \rightarrow \text{prog_code}$ and $\text{prog_code} \rightarrow \text{prog_fee}$ give $\text{enrol_no} \rightarrow \text{prog_fee}$. Such an indirect dependency of a non-prime attribute on the key is precisely the redundancy that 3NF removes.

Third normal form (3NF). R is in 3NF if it is in 2NF and for every non-trivial FD $X \rightarrow Y$, **either X is a superkey of R, or every attribute of Y – X is prime**. The second alternative is what makes 3NF weaker than BCNF: a dependency whose right side is part of a candidate key is tolerated.

Boyce-Codd normal form (BCNF). R is in BCNF if for every non-trivial FD $X \rightarrow Y$, **X is a superkey of R** — "every determinant is a candidate key". BCNF therefore drops 3NF's escape clause for prime attributes, and $\text{BCNF} = 3\text{NF}$ but not conversely.

Example — violating 3NF. Student(enrol_no, name, prog_code, prog_fee), key enrol_no, FDs $\text{enrol_no} \rightarrow \text{name}$, $\text{prog_code} \rightarrow \text{prog_fee}$. Here prog_code is not a superkey and prog_fee is non-prime, so the transitive dependency violates 3NF. Decompose into Student(enrol_no, name, prog_code) and Programme(prog_code, prog_fee) — both 3NF and BCNF.

Example — 3NF but not BCNF. Teach(student, subject, teacher) with the rules: each teacher teaches exactly one subject, and for a given subject a student is taught by exactly one teacher.

FD1: teacher -> subject
 FD2: student, subject -> teacher
 Candidate keys: (student, subject) and (student, teacher)
 Prime attributes: student, subject, teacher -- all of them

student	subject	teacher
Ravi	DBMS	Sharma
Meena	DBMS	Sharma
Ravi	OS	Gupta

FD1 $\text{teacher} \rightarrow \text{subject}$ has a non-superkey determinant, so BCNF is violated. But 3NF is **satisfied**, because the right-hand side subject is a prime attribute. The cost of the violation is real: "Sharma teaches DBMS" is repeated for every student of Sharma, and a teacher with no students assigned cannot be recorded.

BCNF decomposition: $\text{TS}(\text{teacher}, \text{subject})$ with key teacher , and $\text{ST}(\text{student}, \text{teacher})$ with key $(\text{student}, \text{teacher})$. Both are in BCNF and the join on teacher is lossless — but FD2 $(\text{student}, \text{subject}) \rightarrow \text{teacher}$ is no longer enforceable within a single relation. This illustrates the standard trade-off: **3NF decomposition can always be made both lossless and dependency-preserving; BCNF decomposition can always be made lossless but not always dependency-preserving**, which is why 3NF is usually the practical design target.

Q8. What is meant by lossless join decomposition of a relation ? Explain with the help of an example. [3–6 marks · D21 Q3(a), J23 Q3(b)]

A decomposition of R into $R_1, R_2, \dots, R_n$ is **lossless-join** (non-loss, non-additive) if for every legal instance r of R ,

$$\pi_{R_1}(r) \bowtie \pi_{R_2}(r) \bowtie \dots \bowtie \pi_{R_n}(r) = r$$

i.e. joining the fragments back together reproduces exactly the original tuples — no tuple is lost and, more importantly, no **spurious tuple** is gained. (A lossy decomposition actually produces *extra* tuples, so information about which values belonged together is lost.) For a binary decomposition the test is simple: R_1, R_2 is lossless iff $R_1 \cap R_2 \rightarrow R_1$ or $R_1 \cap R_2 \rightarrow R_2$ is in F^+ — that is, the common attributes form a superkey of at least one fragment.

Lossy example. $\text{Emp}(\text{emp_id}, \text{emp_name}, \text{dept})$ with emp_id the key:

emp_id	emp_name	dept
E1	Ravi	Sales
E2	Ravi	Audit

Decompose on emp_name : $R_1(\text{emp_name}, \text{dept}) = \{(\text{Ravi}, \text{Sales}), (\text{Ravi}, \text{Audit})\}$, $R_2(\text{emp_id}, \text{emp_name}) = \{(\text{E1}, \text{Ravi}), (\text{E2}, \text{Ravi})\}$. The natural join on emp_name gives **four** tuples: $(\text{E1}, \text{Ravi}, \text{Sales}), (\text{E1}, \text{Ravi}, \text{Audit}), (\text{E2}, \text{Ravi}, \text{Sales}), (\text{E2}, \text{Ravi}, \text{Audit})$. Two are spurious; we can no longer tell which employee was in which department. The common attribute emp_name is not a key of either fragment, so the test fails.

Lossless example. Decompose the same relation as $R_1(\text{emp_id}, \text{emp_name})$ and $R_2(\text{emp_id}, \text{dept})$. The common attribute is emp_id , and $\text{emp_id} \rightarrow \text{emp_name}$ (indeed emp_id is a superkey of both fragments), so the test is satisfied. The join on emp_id returns exactly $(\text{E1}, \text{Ravi}, \text{Sales})$ and $(\text{E2}, \text{Ravi}, \text{Audit})$ — the original relation. Lossless join is a mandatory property of every decomposition; dependency preservation is desirable but can be sacrificed (as in BCNF).

Q9. What is the meaning of dependency preserving and lossless decomposition ? Explain with the help of an example of each. [5–8 marks · J22 Q2(b), J26 Q2(b)]

Lossless decomposition. R decomposed into $R_1, \dots, R_n$ is lossless if $\pi_{R_1}(r) \bowtie \dots \bowtie \pi_{R_n}(r) = r$ for every legal instance r — the natural join reconstructs R exactly, adding no spurious tuples. Binary test: the common attributes $R_1 \cap R_2$ must be a superkey of R_1 or of R_2 .

Example. $\text{Emp}(\text{emp_id}, \text{emp_name}, \text{dept})$, key emp_id . The decomposition $R_1(\text{emp_id}, \text{emp_name}), R_2(\text{emp_id}, \text{dept})$ has $R_1 \cap R_2 = \{\text{emp_id}\}$, which is a key of both, so it is **lossless**. The

decomposition $R1(\text{emp_name}, \text{dept})$, $R2(\text{emp_id}, \text{emp_name})$ has $R1 \cap R2 = \{\text{emp_name}\}$, not a key of either; with two employees both called Ravi the join produces four tuples instead of two, so it is **lossy**.

Dependency-preserving decomposition. The decomposition preserves dependencies if the union of the dependencies projected onto the fragments is equivalent to the original set:

$$(F_{R1} \cup F_{R2} \cup \dots \cup F_{Rn})^+ = F^+ \\ \text{where } F_{Ri} = \{ X \rightarrow Y \text{ in } F^+ : X \cup Y \subseteq Ri \}$$

This matters because a dependency that spans two fragments cannot be checked by a local constraint — enforcing it would require a join on every insert.

Example of preservation. $R(A, B, C)$ with $F = \{A \rightarrow B, B \rightarrow C\}$. Decompose into $R1(A, B)$ and $R2(B, C)$. $R1$ enforces $A \rightarrow B$, $R2$ enforces $B \rightarrow C$, and $A \rightarrow C$ follows by transitivity, so F is preserved; the decomposition is also lossless since $R1 \cap R2 = \{B\}$ is the key of $R2$.

Example of non-preservation. $\text{Teach}(\text{student}, \text{subject}, \text{teacher})$ with $F = \{\text{teacher} \rightarrow \text{subject}, (\text{student}, \text{subject}) \rightarrow \text{teacher}\}$, candidate keys $(\text{student}, \text{subject})$ and $(\text{student}, \text{teacher})$. The BCNF decomposition $TS(\text{teacher}, \text{subject})$, $ST(\text{student}, \text{teacher})$ is lossless (common attribute teacher is the key of TS) and $\text{teacher} \rightarrow \text{subject}$ is preserved in TS , but **$(\text{student}, \text{subject}) \rightarrow \text{teacher}$ is lost**: its attributes are split across both fragments, so the rule "a student takes a subject from only one teacher" can no longer be enforced without joining the two relations.

This is the general trade-off: lossless join is always achievable and is mandatory; a lossless **and** dependency-preserving decomposition into 3NF always exists, but into BCNF it may not.

Q10. Explain the desirable properties of decomposition of a relation with the help of an example. [5-7 marks · J24 Q1(b), J25 Q3(a)]

When a relation is decomposed during normalisation, the decomposition should have three properties.

1. Attribute preservation. The union of the fragment schemas must equal the original schema, $R1 \cup R2 \cup \dots \cup Rn = R$, so that no attribute is dropped.

2. Lossless join (mandatory). $\pi_{R1}(r) \bowtie \dots \bowtie \pi_{Rn}(r) = r$ for every legal instance — the join reconstructs the original relation and generates no spurious tuples. For a binary split the test is that $R1 \cap R2$ is a superkey of $R1$ or of $R2$.

Example. $\text{Emp}(\text{emp_id}, \text{name}, \text{dept}) \rightarrow R1(\text{emp_id}, \text{name}), R2(\text{emp_id}, \text{dept})$: the common attribute emp_id is a key of both, so the join is lossless. Splitting instead into $(\text{name}, \text{dept})$ and $(\text{emp_id}, \text{name})$ is lossy — two employees named Ravi produce four joined tuples.

3. Dependency preservation (desirable). Every FD of F must be inferable from the FDs that fall entirely inside one fragment, so that each constraint can be checked locally without a join.

Example. $R(A, B, C)$, $F = \{A \rightarrow B, B \rightarrow C\} \rightarrow R1(A, B), R2(B, C)$ preserves both FDs and is lossless. By contrast the BCNF decomposition of $\text{Teach}(\text{student}, \text{subject}, \text{teacher})$ into $(\text{teacher}, \text{subject})$ and $(\text{student}, \text{teacher})$ is lossless but loses $(\text{student}, \text{subject}) \rightarrow \text{teacher}$.

4. Minimum redundancy / no anomaly — the fragments should be in the highest practical normal form (3NF or BCNF) so that insertion, update and deletion anomalies disappear.

A lossless, dependency-preserving decomposition into **3NF always exists** (synthesis from a canonical cover); into **BCNF a lossless decomposition always exists but a dependency-preserving one may not** — which is why 3NF is the usual design target. The cost of any decomposition is that queries spanning the fragments need joins.

Q11. What is functional dependency ? Explain with the help of an example.

[4–7 marks · D23 Q4(b), J25 Q3(a), D24 Q5(f)]

A **functional dependency** $X \rightarrow Y$ on a relation schema R ($X, Y \subseteq R$) is an integrity constraint stating that for any two tuples t_1, t_2 of any legal instance of R , if $t_1[X] = t_2[X]$ then $t_1[Y] = t_2[Y]$. X is the **determinant**, Y the **dependent**. The FD is a property of the *meaning* of the attributes, not of one particular instance, so it is derived from the application's rules.

Example — `Student(enrol_no, name, prog_code, prog_fee)`:

enrol_no	name	prog_code	prog_fee
2301	Ravi	MCA	60000
2302	Meena	MCA	60000
2303	Anil	BCA	40000

```
enrol_no -> name, prog_code    (each enrolment number belongs to one student)
prog_code -> prog_fee          (each programme has one fee)
enrol_no -> prog_fee           (transitive, derived)
```

`name → enrol_no` does **not** hold: two students may share a name.

Kinds. *Trivial* ($Y \subseteq X$, e.g. `enrol_no → enrol_no`); *full* (no proper subset of X determines Y); *partial* (a proper subset of a key determines a non-prime attribute — breaks 2NF); *transitive* ($X \rightarrow Y \rightarrow Z$ — breaks 3NF).

Armstrong's axioms generate all FDs implied by a set F : **reflexivity** (if $Y \subseteq X$ then $X \rightarrow Y$), **augmentation** ($X \rightarrow Y \vdash XZ \rightarrow YZ$), **transitivity** ($X \rightarrow Y, Y \rightarrow Z \vdash X \rightarrow Z$); the derived rules are union, decomposition and pseudo-transitivity.

Use. FDs define keys (X is a superkey iff $X^+ = R$) and are the basis of every normal form test: 2NF forbids partial FDs, 3NF forbids transitive FDs on non-prime attributes, BCNF requires every determinant to be a candidate key.

Q12. What is multi-valued dependency (MVD) ? Explain with the help of an example. How is MVD different from functional dependency ? Explain how MVDs can be used to decompose a relation to 4NF. [5–8 marks · D22 Q3(b), D23 Q5(ii), D24 Q3(b), J25 Q3(b), D25 Q2(c)]

A **multi-valued dependency** $X \twoheadrightarrow Y$ holds on $R(X, Y, Z)$ if, for every pair of tuples t_1, t_2 with $t_1[X] = t_2[X]$, the relation also contains tuples t_3 and t_4 such that $t_3[X] = t_4[X] = t_1[X]$, $t_3[Y] = t_1[Y]$, $t_3[Z] = t_2[Z]$, $t_4[Y] = t_2[Y]$, $t_4[Z] = t_1[Z]$. In words: for a given X value, the set of Y values is **independent** of the set of Z values, so the relation must contain *every combination* of them. MVDs always come in pairs: $X \twoheadrightarrow Y$ implies $X \twoheadrightarrow Z$ where $Z = R - X - Y$. An MVD is **trivial** if $Y \subseteq X$ or $X \cup Y = R$.

Example. `Course(course, instructor, textbook)` — a course is taught by several instructors and uses several textbooks, and the two sets are independent of each other:

course	instructor	textbook
DBMS	Sharma	Korth
DBMS	Sharma	Elmasri
DBMS	Gupta	Korth
DBMS	Gupta	Elmasri

Here $\text{course} \twoheadrightarrow \text{instructor}$ and $\text{course} \twoheadrightarrow \text{textbook}$. The relation is all-key, so it is in BCNF, yet it is massively redundant: adding a third textbook forces two new rows, one per instructor, and dropping an instructor means deleting one row per textbook. This is exactly the anomaly 4NF removes.

MVD vs FD

Functional dependency $X \rightarrow Y$	Multi-valued dependency $X \twoheadrightarrow Y$
One X value determines one Y value	One X value determines a set of Y values
Constrains pairs of tuples	Requires the <i>existence</i> of additional tuples (tuple-generating)
Basis of 2NF, 3NF, BCNF	Basis of 4NF
Every FD $X \rightarrow Y$ is also an MVD $X \twoheadrightarrow Y$	The converse does not hold
Removed by splitting off the dependent attribute with its determinant	Removed by splitting the two independent multi-valued attributes apart

Decomposition to 4NF. R is in **4NF** if for every non-trivial MVD $X \twoheadrightarrow Y$ in R, X is a **superkey** of R. If a non-trivial MVD $X \twoheadrightarrow Y$ violates this, decompose R into $R_1(X \cup Y)$ and $R_2(X \cup (R - X - Y))$; by Fagin's theorem this decomposition is **lossless** precisely because the MVD holds. Repeat until every fragment is in 4NF.

Applying it to Course: $\text{course} \twoheadrightarrow \text{instructor}$ is non-trivial and course is not a superkey, so

TEACHES(course, instructor)	PK (course, instructor)	2 rows
USES (course, textbook)	PK (course, textbook)	2 rows

Four rows become two plus two; adding a textbook is now one row, and the natural join on `course` reproduces the original four rows exactly. Both fragments are in 4NF because their only MVDs are trivial.

Q13. Explain the update anomaly, insertion anomaly and deletion anomaly in the context of relational database systems with the help of an example.

[2–6 marks · D22 Q1(e), J23 Q3(a)(iii), D24 Q3(a), J26 Q1(h)]

Anomalies are the undesirable side effects of storing logically independent facts in one unnormalised relation, caused by redundancy.

Example — `Enrol(student_id, student_name, coursecode, course_name, marks)` with key `(student_id, coursecode)`, FDs $\text{student_id} \rightarrow \text{student_name}$ and $\text{coursecode} \rightarrow \text{course_name}$:

student_id	student_name	coursecode	coursename	marks
S01	Ravi	MCS-207	DBMS	78
S01	Ravi	MCS-208	Data Structures	65
S02	Meena	MCS-207	DBMS	82
S03	Anil	MCS-209	Operating Systems	55

Insertion anomaly — a fact cannot be inserted because unrelated data is missing. A new course MCS-210 "Compiler Design" that nobody has registered for cannot be stored: student_id is part of the primary key and would have to be NULL, violating entity integrity. Likewise a newly admitted student with no course yet.

Update anomaly — one fact is stored many times, so a change must be applied to many rows and partial application creates inconsistency. "DBMS" appears in rows 1 and 3. Renaming the course to "Database Management Systems" requires both rows to be updated; if only row 1 is changed, MCS-207 has two different names in the same table.

Deletion anomaly — deleting one fact destroys another. Deleting row 4 because Anil has withdrawn also erases the only record that MCS-209 is "Operating Systems" and that S03 is named Anil.

Cure. Normalise to 2NF/3NF: STUDENT(student_id, student_name), COURSE(coursecode, coursename), ENROL(student_id, coursecode, marks). Now a course is inserted independently, a rename touches exactly one row, and deleting an enrolment leaves the student and the course rows intact.

ER modelling and conceptual design

Q1. [Scenario description] Perform the following tasks for the description given above: (i) List all the entities. (ii) List the attributes of the entities. (iii) Identify relationships between/among the entities. (iv) Draw an E-R diagram. (v) List all the constraints, including primary and foreign keys. (vi) Convert the E-R diagram to relations.

[12 marks (2 each) · D21 Q1(a) university, J22 Q1(a) bank with joint accounts, D22 Q1(a) bank with withdrawals, J23 Q1(a) library, D24

Notation used in all six sketches

[ENTITY]	rectangle	((WEAK ENTITY))	double rectangle
<RELSHIP>	diamond	(attribute)	oval, key attribute underlined
1 : N	cardinality label written on the edge		

Scenario A — University: students and programmes (D21 Q1(a))

Assumptions. Every student is enrolled in exactly one programme; a programme has many students. A programme consists of many courses and a course may be offered in several programmes. A student registers for courses and is awarded marks per course.

(i) Entities — STUDENT, PROGRAMME, COURSE.

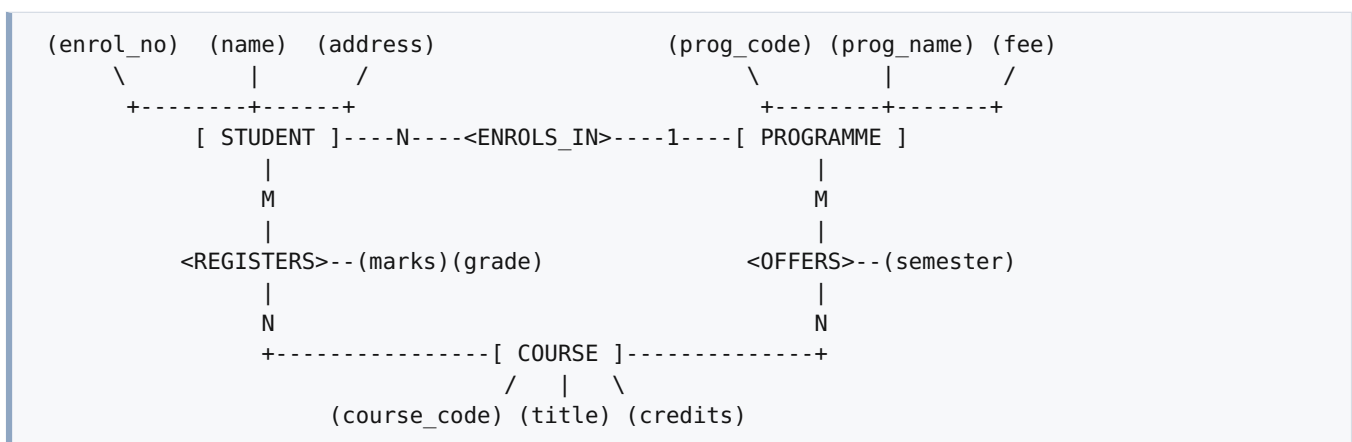
(ii) Attributes

Entity	Attributes (key underlined by __)
STUDENT	<u>enrol_no</u> , name (f_name, l_name), address, phone, date_of_birth, date_of_admission
PROGRAMME	<u>prog_code</u> , prog_name, duration, fee, total_credits
COURSE	<u>course_code</u> , course_title, credits, duration

(iii) Relationships

Relationship	Entities	Cardinality	Participation	Descriptive attribute
ENROLS_IN	STUDENT - PROGRAMME	N : 1	total on STUDENT side	date_of_enrolment
OFFERS	PROGRAMME - COURSE	M : N	partial	semester
REGISTERS	STUDENT - COURSE	M : N	partial	session, marks, grade

(iv) E-R diagram



(v) Constraints

- Primary keys: STUDENT(enrol_no), PROGRAMME(prog_code), COURSE(course_code), REGISTERS(enrol_no, course_code), OFFERS(prog_code, course_code).
- Foreign keys: STUDENT.prog_code → PROGRAMME; REGISTERS.enrol_no → STUDENT; REGISTERS.course_code → COURSE; OFFERS.prog_code → PROGRAMME; OFFERS.course_code → COURSE.
- Total participation: every STUDENT must have a prog_code (NOT NULL). Domain: marks between 0 and 100; grade ∈ {A, B, C, D, F}; fee > 0.

(vi) Relations

PROGRAMME	(prog_code, prog_name, duration, fee, total_credits)
STUDENT	(enrol_no, name, address, phone, date_of_birth, prog_code, date_of_enrolment)
COURSE	(course_code, course_title, credits, duration)
OFFERS	(prog_code, course_code, semester)
REGISTERS	(enrol_no, course_code, session, marks, grade)

Scenario B — Bank: customers, accounts, joint accounts (J22 Q1(a))

Assumptions. A customer may hold several accounts and an account may be held jointly by several customers, so the relationship is many-to-many. Every account belongs to exactly one branch.

(i) Entities — CUSTOMER, ACCOUNT, BRANCH. (TRANSACTION is a weak entity if transactions are recorded.)

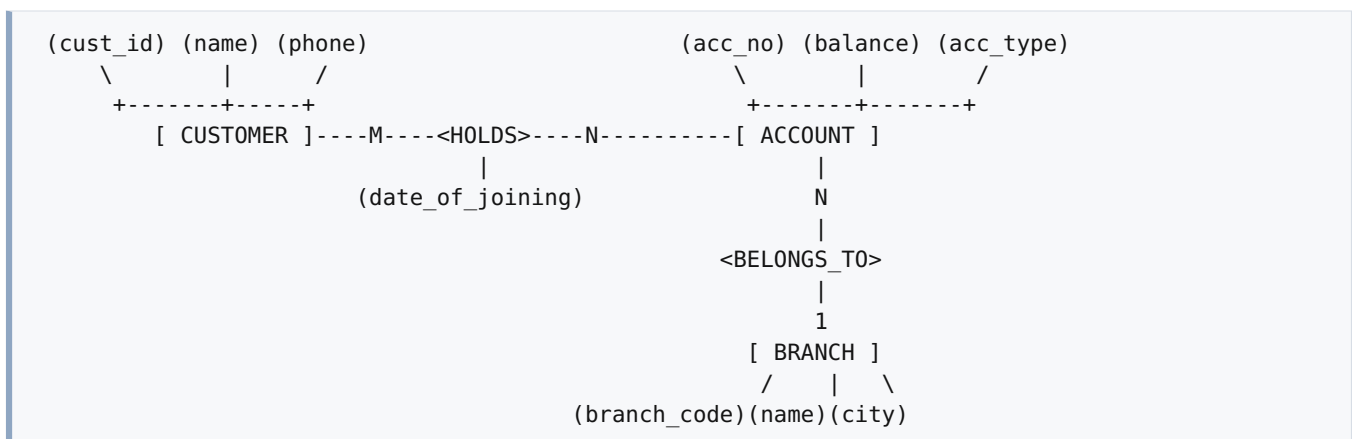
(ii) Attributes

Entity	Attributes
CUSTOMER	cust_id , name, address (street, city, pin), phone, date_of_birth
ACCOUNT	acc_no , acc_type, balance, date_of_opening
BRANCH	branch_code , branch_name, city, IFSC

(iii) Relationships

Relationship	Entities	Cardinality	Note
HOLDS	CUSTOMER - ACCOUNT	M : N	an M:N holds relationship is the joint-account facility; attribute date_of_joining, is_primary_holder
BELONGS_TO	ACCOUNT - BRANCH	N : 1	total on ACCOUNT side

(iv) E-R diagram



(v) Constraints — PKs: CUSTOMER(cust_id), ACCOUNT(acc_no), BRANCH(branch_code), HOLDS(cust_id, acc_no). FKs: ACCOUNT.branch_code → BRANCH; HOLDS.cust_id → CUSTOMER; HOLDS.acc_no → ACCOUNT. Every account must belong to a branch (NOT NULL); every account must have at least one holder; balance ≥ 0; acc_type ∈ {Savings, Current, FD}.

(vi) Relations

CUSTOMER(cust_id, name, street, city, pin, phone, date_of_birth)
BRANCH (branch_code, branch_name, city, IFSC)
ACCOUNT (acc_no, acc_type, balance, date_of_opening, branch_code)
HOLDS (cust_id, acc_no, date_of_joining, is_primary_holder)

Scenario C — Bank: customers, accounts, withdrawals (D22 Q1(a))

Assumptions. A customer may hold many accounts; many withdrawals/deposits are made on an account. A withdrawal has no meaning without its account, so it is a **weak entity** identified by (acc_no, txn_date_time) or by a partial key txn_no.

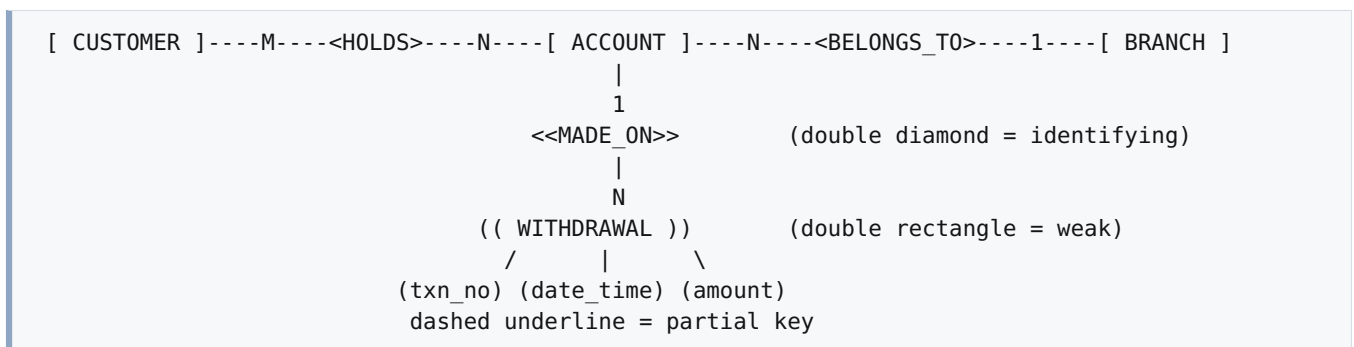
(i) Entities — CUSTOMER, ACCOUNT, BRANCH; **WITHDRAWAL (weak entity)**.

(ii) Attributes

Entity	Attributes
CUSTOMER	cust_id , name, address, phone
ACCOUNT	acc_no , acc_type, balance, date_of_opening
WITHDRAWAL (weak)	txn_no (partial key), date_time, amount, mode
BRANCH	branch_code , branch_name, city

(iii) Relationships — HOLDS (CUSTOMER : ACCOUNT, M:N, joint accounts allowed); **MADE_ON** (WITHDRAWAL : ACCOUNT, N:1, *identifying* relationship, total participation of WITHDRAWAL); BELONGS_TO (ACCOUNT : BRANCH, N:1).

(iv) E-R diagram



(v) Constraints — PKs: CUSTOMER(cust_id), ACCOUNT(acc_no), BRANCH(branch_code), HOLDS(cust_id, acc_no), **WITHDRAWAL(acc_no, txn_no)** — the owner's key plus the partial key. FKs: HOLDS.cust_id, HOLDS.acc_no, WITHDRAWAL.acc_no → ACCOUNT (ON DELETE CASCADE, because a weak entity cannot exist without its owner), ACCOUNT.branch_code → BRANCH. Withdrawal amount > 0 and amount ≤ balance; balance ≥ 0.

(vi) Relations

CUSTOMER	(cust_id, name, address, phone)
BRANCH	(branch_code, branch_name, city)
ACCOUNT	(acc_no, acc_type, balance, date_of_opening, branch_code)
HOLDS	(cust_id, acc_no, date_of_joining)
WITHDRAWAL	(acc_no, txn_no, date_time, amount, mode)

Scenario D — Library: members, books, issue and return (J23 Q1(a))

Assumptions. A member may borrow many books over time and a book title may be borrowed by many members, so ISSUE is M:N with its own attributes; a physical copy can be out with only one member at a time.

(i) Entities — MEMBER, BOOK, AUTHOR (optional), STAFF (optional).

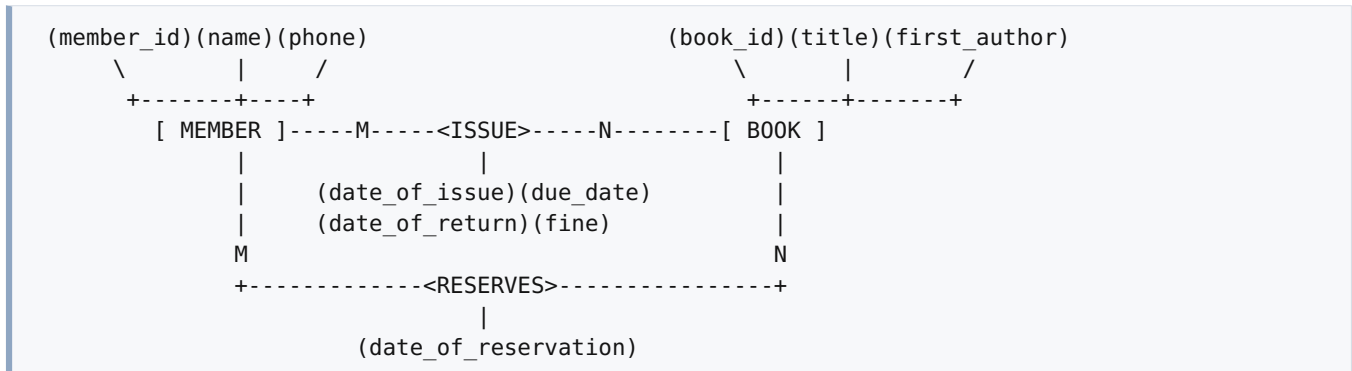
(ii) Attributes

Entity	Attributes
MEMBER	member_id , name, address, phone, membership_type, date_of_joining
BOOK	book_id (accession no), title, first_author, publisher, ISBN, year, price, no_of_copies

(iii) Relationships

Relationship	Entities	Cardinality	Attributes
ISSUE	MEMBER - BOOK	M : N	date_of_issue, due_date, date_of_return, fine
RESERVES	MEMBER - BOOK	M : N	date_of_reservation

(iv) E-R diagram



(v) Constraints — PKs: MEMBER(member_id), BOOK(book_id), ISSUE(member_id, book_id, date_of_issue) — date_of_issue is needed because the same member may borrow the same book again. FKs: ISSUE.member_id → MEMBER, ISSUE.book_id → BOOK, same for RESERVES. due_date > date_of_issue; date_of_return may be NULL (still out); fine ≥ 0; a member may hold at most 4 books at a time (an application constraint, not expressible in the ERD).

(vi) Relations

```

MEMBER (member_id, name, address, phone, membership_type, date_of_joining)
BOOK (book_id, title, first_author, publisher, ISBN, year, price, no_of_copies)
ISSUE (member_id, book_id, date_of_issue, due_date, date_of_return, fine)
RESERVES(member_id, book_id, date_of_reservation)

```

Scenario E — Bookstore: customers, books, copies, authors, orders (D24 Q1(a))

Assumptions. A book is written by one or more authors and an author writes many books (M:N). A book title has many physical **copies**, each identified only within its book, so COPY is a weak entity. A customer places many orders; an order contains many copies/titles (M:N with quantity).

(i) Entities — CUSTOMER, BOOK, AUTHOR, ORDER; **COPY (weak entity, owner BOOK).**

(ii) Attributes

Entity	Attributes
CUSTOMER	cust_no , name, address, phone, email
BOOK	ISBN , title, publisher, year, price, edition
AUTHOR	author_id , author_name, nationality
COPY (weak)	copy_no (partial key), shelf_location, condition, status
ORDER	order_id , order_date, total_amount, status

(iii) Relationships — WRITTEN_BY (BOOK : AUTHOR, M:N, attribute author_position); **HAS_COPY** (BOOK : COPY, 1:N, identifying, total on COPY); PLACES (CUSTOMER : ORDER, 1:N, total on ORDER); CONTAINS (ORDER : BOOK, M:N, attributes quantity, unit_price).

(iv) E-R diagram

```

[ AUTHOR ] ----M-----<WRITTEN_BY>-----N-----[ BOOK ] ----1-----<<HAS_COPY>>-----N-----(( COPY ))
                                     |                                     /      \
                                     M                                     (copy_no)

                                     |
                                     <CONTAINS>--(quantity)(unit_price)
                                     |
                                     N
                                     [ ORDER ] ----N-----<PLACES>-----1-----[ CUSTOMER ]
                                     /      \                                     /      |      \
                                     (order_id)(order_date)                     (cust_no)(name)(phone)
(shelf_location)

```

(v) Constraints — PKs: CUSTOMER(cust_no), BOOK(ISBN), AUTHOR(author_id), ORDER(order_id), **COPY(ISBN, copy_no)**, WRITTEN_BY(ISBN, author_id), CONTAINS(order_id, ISBN). FKs: ORDER.cust_no → CUSTOMER (NOT NULL, total participation); COPY.ISBN → BOOK ON DELETE CASCADE; WRITTEN_BY.ISBN → BOOK, .author_id → AUTHOR; CONTAINS.order_id → ORDER, .ISBN → BOOK. quantity ≥ 1; price > 0; status ∈ {available, sold, damaged}.

(vi) Relations

```

CUSTOMER (cust_no, name, address, phone, email)
AUTHOR   (author_id, author_name, nationality)
BOOK     (ISBN, title, publisher, year, price, edition)
WRITTEN_BY(ISBN, author_id, author_position)
COPY     (ISBN, copy_no, shelf_location, condition, status)
ORDERS   (order_id, order_date, total_amount, status, cust_no)
CONTAINS (order_id, ISBN, quantity, unit_price)

```

Scenario F — Departmental store: items, stock, customers, purchases (J25 Q1(a))

Scenario text. Unique item identifier, item name, price per unit, units in stock; unique customer number, customer phone and address; the list of items purchased by each customer with quantity.

Assumptions. Units in stock is an attribute of ITEM (a single store). A customer buys many items and an item is bought by many customers, so PURCHASES is M:N with quantity and date. (If several purchases of the same item by the same customer must be distinguished, a BILL entity is introduced.)

(i) Entities — ITEM, CUSTOMER. (Optional third entity: PURCHASE/BILL, if each transaction is to be identified separately.)

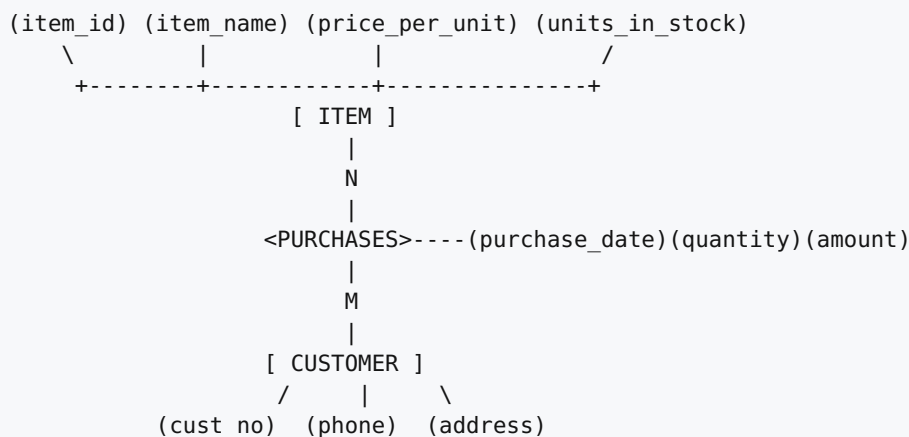
(ii) Attributes

Entity	Attributes
ITEM	item_id , item_name, price_per_unit, units_in_stock
CUSTOMER	cust_no , phone, address (street, city, pin)

(iii) Relationship

Relationship	Entities	Cardinality	Attributes
PURCHASES	CUSTOMER - ITEM	M : N	purchase_date, quantity, amount

(iv) E-R diagram



(v) Constraints — PKs: ITEM(item_id), CUSTOMER(cust_no), PURCHASES(cust_no, item_id, purchase_date). FKs: PURCHASES.cust_no → CUSTOMER, PURCHASES.item_id → ITEM. price_per_unit > 0; units_in_stock ≥ 0; quantity ≥ 1 and quantity ≤ units_in_stock at the time of sale; amount = quantity × price_per_unit (derived).

(vi) Relations

```

ITEM      (item_id, item_name, price_per_unit, units_in_stock)
CUSTOMER  (cust_no, phone, street, city, pin)
PURCHASES(cust_no, item_id, purchase_date, quantity, amount)
  
```

General conversion rules used in all six answers: every strong entity becomes a relation with its own key; a 1:N relationship is implemented by posting the key of the "1" side into the relation of the "N" side; an M:N relationship becomes a separate relation whose key is the union of the two participating keys plus any discriminator, with the relationship's descriptive attributes as non-key columns; a weak entity becomes a relation whose key is the owner's key plus the partial key, with an ON DELETE CASCADE foreign key; a multi-valued attribute becomes a relation of (owner key, value); a composite attribute is flattened into its component columns.

Q2. A bank wants to develop an application to store information about Customer and Bank. Answer the following [Mention all the assumptions]: (i) List all the entities and related attributes (ii) Relationship among all the entities identified in (i) (iii) List all the constraints and draw an E-R diagram as per the above entities, attributes, constraints and relationships. [10 marks · D23 Q1(a)]

Assumptions (stated explicitly, as the question demands)

1. A bank has many branches; a branch belongs to exactly one bank.
2. A customer may hold one or more accounts; an account may be jointly held by more than one customer (M:N).
3. Every account is maintained at exactly one branch.
4. A customer may take zero or more loans; a loan belongs to exactly one customer and one branch.
5. Many transactions are recorded against an account; a transaction cannot exist without its account, so it is a **weak entity**.
6. Each customer, account, loan and branch has a system-generated unique identifier.

(i) Entities and attributes

Entity	Attributes (key in bold)
BANK	bank_code , bank_name, head_office_address
BRANCH	branch_code , branch_name, city, address, phone, bank_code
CUSTOMER	cust_id , name (first, last), address (street, city, pin), phone, date_of_birth, PAN
ACCOUNT	acc_no , acc_type, balance, date_of_opening, interest_rate
LOAN	loan_no , loan_type, principal, rate, tenure, outstanding
TRANSACTION (weak)	txn_no (partial key), date_time, type (Dr/Cr), amount

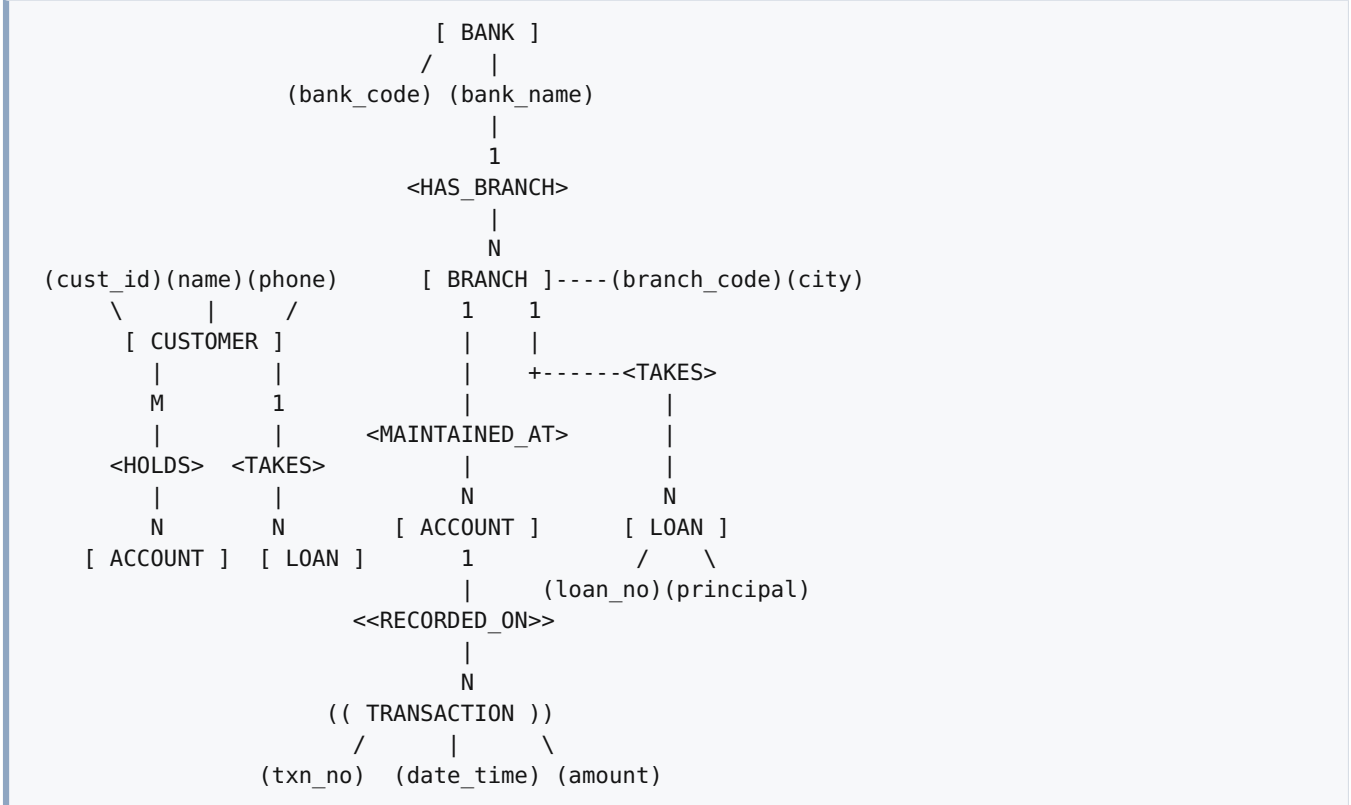
(ii) Relationships

Relationship	Entities	Cardinality	Participation	Attributes
HAS_BRANCH	BANK - BRANCH	1 : N	total on BRANCH	—
HOLDS	CUSTOMER - ACCOUNT	M : N	total on ACCOUNT	date_of_joining
MAINTAINED_AT	ACCOUNT - BRANCH	N : 1	total on ACCOUNT	—
TAKES	CUSTOMER - LOAN	1 : N	total on LOAN	date_sanctioned
RECORDED_ON	TRANSACTION - ACCOUNT	N : 1 (identifying)	total on TRANSACTION	—

(iii) Constraints

- Primary keys: BANK(bank_code), BRANCH(branch_code), CUSTOMER(cust_id), ACCOUNT(acc_no), LOAN(loan_no), HOLDS(cust_id, acc_no), TRANSACTION(acc_no, txn_no).
- Foreign keys: BRANCH.bank_code → BANK; ACCOUNT.branch_code → BRANCH; HOLDS.cust_id → CUSTOMER; HOLDS.acc_no → ACCOUNT; LOAN.cust_id → CUSTOMER; LOAN.branch_code → BRANCH; TRANSACTION.acc_no → ACCOUNT ON DELETE CASCADE.
- Participation: every branch belongs to a bank; every account has at least one holder and exactly one branch (NOT NULL).
- Domain/semantic: balance ≥ 0 for savings; acc_type $\in \{\text{Savings, Current, FD, RD}\}$; amount > 0 ; rate between 0 and 25; a withdrawal may not exceed the balance.

E-R diagram



Resulting relations: BANK(bank_code, bank_name, head_office_address), BRANCH(branch_code, branch_name, city, address, phone, bank_code), CUSTOMER(cust_id, first, last, street, city, pin, phone, dob, PAN), ACCOUNT(acc_no, acc_type, balance, date_of_opening, interest_rate, branch_code), HOLDS(cust_id, acc_no, date_of_joining), LOAN(loan_no, loan_type, principal, rate, tenure, outstanding, cust_id, branch_code), TRANSACTION(acc_no, txn_no, date_time, type, amount).

Q3. Design an ER-diagram for the specifications to maintain any Hospital. Clearly indicate the entities, attributes, primary key, constraints, relationships and cardinality. [10 marks · J24 Q1(f)]

Assumptions. A hospital has many departments; a doctor belongs to exactly one department. A patient may consult many doctors and a doctor treats many patients (M:N). A patient admitted as an in-patient is assigned exactly one bed in one ward at a time. Each admission generates many treatment records, which cannot exist without the admission, so TREATMENT is a weak entity.

Entities, attributes and primary keys

Entity	Attributes	Primary key
DEPARTMENT	dept_id, dept_name, location, hod	dept_id
DOCTOR	doctor_id, name, specialisation, qualification, phone, salary, dept_id	doctor_id
PATIENT	patient_id, name, age, gender, address, phone, blood_group	patient_id
ADMISSION	admission_no, date_of_admission, date_of_discharge, patient_id, ward_no, bed_no	admission_no
WARD	ward_no, ward_type, capacity, floor	ward_no
TREATMENT (weak)	treat_no (partial), date, diagnosis, prescription, doctor_id	(admission_no, treat_no)
BILL	bill_no, bill_date, room_charges, treatment_charges, total	bill_no

Relationships with cardinality

Relationship	Entities	Cardinality	Participation
WORKS_IN	DOCTOR - DEPARTMENT	N : 1	total on DOCTOR
CONSULTS	PATIENT - DOCTOR	M : N (attributes: date, fee)	partial
ADMITTED	PATIENT - ADMISSION	1 : N	total on ADMISSION
ASSIGNED_TO	ADMISSION - WARD	N : 1	total on ADMISSION
HAS_TREATMENT	ADMISSION - TREATMENT	1 : N (identifying)	total on TREATMENT
GENERATES	ADMISSION - BILL	1 : 1	total on BILL

E-R diagram



Constraints. Foreign keys: DOCTOR.dept_id → DEPARTMENT; ADMISSION.patient_id → PATIENT, .ward_no → WARD; TREATMENT.admission_no → ADMISSION ON DELETE CASCADE, .doctor_id → DOCTOR; BILL.admission_no → ADMISSION (UNIQUE, enforcing 1:1); CONSULTS.patient_id → PATIENT, .doctor_id → DOCTOR. Domain: gender ∈ {M, F, O}; age ≥ 0; blood_group ∈ {A+, A-, B+, ...}; date_of_discharge ≥ date_of_admission or NULL. Key constraint: no two current admissions may share the same (ward_no, bed_no).

Relations

DEPARTMENT	(dept_id, dept_name, location, hod)
DOCTOR	(doctor_id, name, specialisation, qualification, phone, salary, dept_id)
PATIENT	(patient_id, name, age, gender, address, phone, blood_group)
WARD	(ward_no, ward_type, capacity, floor)
ADMISSION	(admission_no, date_of_admission, date_of_discharge, patient_id, ward_no, bed_no)
CONSULTS	(patient_id, doctor_id, date, fee)
TREATMENT	(admission_no, treat_no, date, diagnosis, prescription, doctor_id)
BILL	(bill_no, admission_no, bill_date, room_charges, treatment_charges, total)

Q4. Discuss the Entity-Relationship (E-R) Model, focusing on its key components: entities, attributes and relationships. How does it aid in database design ? [10 marks · D25 Q3(b)]

The **E-R model** (Chen, 1976) is a high-level, implementation-independent conceptual data model that describes the miniworld as a collection of **entities**, their **attributes** and the **relationships** among them. It is the standard first step of database design, sitting between the requirements document and the logical (relational) schema.

Entity and entity set. An **entity** is a distinguishable thing in the miniworld — a particular student, a particular account. An **entity set** (entity type) is a collection of entities of the same kind, drawn as a **rectangle**. A **strong entity** has a key of its own; a **weak entity** has none and is identified by an owner entity plus a partial key — drawn as a double rectangle joined by a double diamond (identifying relationship), e.g. a TRANSACTION of an ACCOUNT.

Attributes. Properties of an entity, drawn as **ovals**:

Kind	Meaning	Example
Simple / atomic	indivisible	age
Composite	has sub-parts	name(first, last); address(street, city, pin)
Single-valued	one value per entity	date_of_birth
Multi-valued	a set of values (double oval)	phone_numbers
Derived	computed from others (dashed oval)	age from date_of_birth
Key	uniquely identifies (underlined)	enrol_no
Null-valued	unknown or inapplicable	middle_name

Relationships. A **relationship** is an association among entities; a **relationship set** is drawn as a **diamond**. Its properties are:

- **Degree** — unary (recursive, e.g. EMPLOYEE *supervises* EMPLOYEE), binary, ternary.
- **Cardinality ratio** — 1:1, 1:N, N:1, M:N, labelled on the edges. Example: STUDENT *enrols in* PROGRAMME is N:1; STUDENT *registers for* COURSE is M:N.
- **Participation (existence) constraint** — total (mandatory, double line) or partial (optional).
- **Descriptive attributes** — attributes of the relationship itself, e.g. marks on REGISTERS.
- **Role names** on recursive relationships.

The **Enhanced E-R (EER)** model adds generalisation/specialisation hierarchies with disjoint/overlapping and total/partial constraints, aggregation and categories.

How it aids database design

1. It is a **communication tool**: the diagram is understandable to users and analysts, so requirements can be validated before any code is written.
2. It separates **what** the data means from **how** it will be stored — independence from the DBMS and the physical design.
3. Keys and cardinality/participation constraints captured in the diagram become the **primary keys, foreign keys, NOT NULL and uniqueness constraints** of the schema, so integrity is designed in rather than patched on.
4. It converts **mechanically to relations** by well-defined rules (entity → relation; 1:N → post the key of the 1-side into the N-side; M:N → separate relation; weak entity → owner key + partial key; multi-valued attribute → separate relation), which means the logical design is derived rather than guessed.

5. A correctly drawn ERD yields a schema that is usually already close to 3NF, so **redundancy and the insert/update/delete anomalies are avoided** from the start.
6. It documents the database for maintenance and for later schema evolution.

Q5. Illustrate the conversion of ER diagram to Relational database. How does DBMS differ from RDBMS ? [5 marks · J26 Q1(e)]

Conversion rules, with an illustration. Take the ERD: STUDENT(**enrol_no**, name, {phone}) — N:1 ENROLS_IN → PROGRAMME(**prog_code**, prog_name, fee); STUDENT — M:N REGISTERS(marks) → COURSE(**course_code**, title).

ER construct	Relational rule	Result
Strong entity	one relation, entity key = PK	PROGRAMME(prog_code, prog_name, fee), COURSE(course_code, title)
Composite attribute	one column per component	name → (first_name, last_name)
Multi-valued attribute	separate relation (owner key, value)	ST_PHONE(enrol_no, phone)
Derived attribute	not stored; computed	age from dob
1:N relationship	post the key of the 1-side into the N-side as a FK	STUDENT(enrol_no, first_name, last_name, prog_code)
1:1 relationship	post either key as a FK with UNIQUE	—
M:N relationship	new relation, PK = union of both keys, plus descriptive attributes	REGISTERS(enrol_no, course_code, marks)
Weak entity	relation with PK = owner key + partial key, FK ON DELETE CASCADE	TRANSACTION(acc_no, txn_no, amount)
Generalisation	one table per subclass, or one table with a type discriminator	—

DBMS vs RDBMS

DBMS	RDBMS
Software to store and manage data in any data model (hierarchical, network, file-based, object)	A DBMS whose data model is specifically the relational model — data in tables of rows and columns
Relationships expressed by pointers/parent-child links or not at all	Relationships expressed by primary key / foreign key values
Usually single-user, little or no support for normalisation	Multi-user, supports normalisation, SQL, views
Weak or no enforcement of integrity constraints	Enforces entity, referential and domain integrity; Codd's rules
No distributed support, limited security	Distribution, ACID transactions, GRANT/REVOKE security
Data redundancy higher	Redundancy controlled by normalisation
Examples: IMS, dBASE, Microsoft Access (file-level), XML stores	Examples: Oracle, MySQL, PostgreSQL, SQL Server, DB2

Every RDBMS is a DBMS; the converse is not true.

Q6. Weak entity (D23: weak entity and strong entity) — short note.

[4-5 marks · D21 Q5(e), J22 Q5(b), D23 Q5(i)]

A **strong (regular) entity** type has an attribute or set of attributes of its own that uniquely identifies each of its instances — a **key**. It exists independently of any other entity and is drawn as a single **rectangle** with its key attribute underlined by a solid line. Example: ACCOUNT(acc_no, type, balance) — acc_no identifies the account on its own.

A **weak entity** type has **no key of its own**. Its instances are identified only in combination with an **owner (identifying) entity**, through a mandatory **identifying relationship**, using a **partial key (discriminator)** that distinguishes the weak entities belonging to the same owner. Notation: **double rectangle** for the entity, **double diamond** for the identifying relationship, **dashed underline** for the partial key, and a **double line** on the edge to show total participation.

Example: TRANSACTION(txn_no, date_time, amount) of an ACCOUNT. txn_no 1, 2, 3 ... restarts for every account, so txn_no alone is not unique; the full identifier is (acc_no, txn_no). A transaction cannot exist without its account.

```
[ ACCOUNT ]====1====<<MADE_ON>>====N====(( TRANSACTION ))
(acc_no)                                (txn_no) <- partial key
```

Strong entity	Weak entity
Has its own primary key	Has only a partial key
Single rectangle	Double rectangle
Exists independently	Existence-dependent on the owner
Ordinary relationships	Identifying relationship, total participation
PK = its own key	PK = owner's PK + partial key

On conversion to relations, the weak entity becomes TRANSACTION(acc_no, txn_no, date_time, amount) with primary key (acc_no, txn_no) and a foreign key acc_no → ACCOUNT declared **ON DELETE CASCADE**, because deleting the owner must delete its weak entities. Other examples: DEPENDENT of an EMPLOYEE, ROOM of a BUILDING, ORDER_LINE of an ORDER.

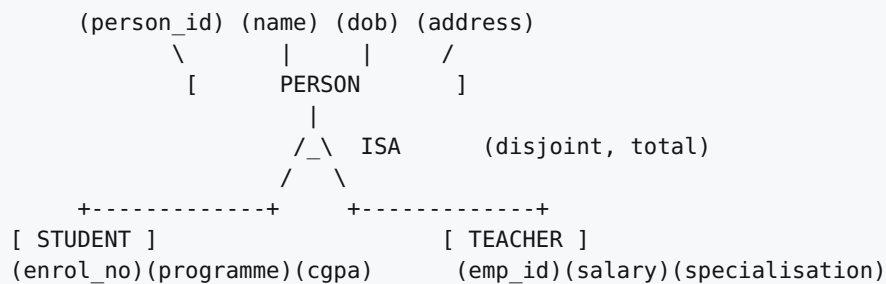
Q7. Explain the concept of generalization and specialization in the context of the E-R model, with the help of an example. / Enhanced E-R model. [5 marks · D22 Q2(c), J25 Q5(g)]

Generalisation and specialisation are the abstraction mechanisms of the **Enhanced E-R (EER)** model, which extends the basic E-R model with class/sub-class hierarchies, aggregation and categories.

Specialisation (top-down) — starting from a higher-level entity set (the **superclass**), identify sub-groupings that have distinguishing attributes or relationships and define them as **subclasses**. Example: from EMPLOYEE, specialise into ENGINEER, MANAGER and CLERK because each has attributes the others do not.

Generalisation (bottom-up) — starting from several entity sets that share attributes, factor the common attributes into a new higher-level superclass. Example: STUDENT and TEACHER both have name, address and date_of_birth, so generalise them into PERSON.

Both describe the same hierarchy, differing only in the direction of design. The subclass **inherits** all attributes, the key and all relationships of the superclass, and adds its own — an **IS-A** relationship, drawn with a triangle labelled ISA (or a circle with subset symbols).



Constraints on a hierarchy. *Disjointness* — **disjoint** (d): an entity may belong to at most one subclass (a person is a student or a teacher, not both); **overlapping** (o): it may belong to several (a person may be both student and teacher). *Completeness* — **total** (double line): every superclass entity must belong to some subclass; **partial**: some may belong to none. A **defining predicate** may be given, e.g. a specialisation of ACCOUNT on acc_type.

Conversion to relations. Either (a) one relation per subclass carrying the inherited key — PERSON(person_id, name, dob, address), STUDENT(person_id, enrol_no, programme, cgpa), TEACHER(person_id, emp_id, salary, specialisation) — or (b) a single relation with all attributes plus a type discriminator and NULLs for inapplicable attributes, which is cheaper to query but wastes space.

Benefits: avoids repeating common attributes, captures the real-world taxonomy, and allows constraints and relationships to be attached at the right level (e.g. only STUDENT participates in REGISTERS).

SQL, DDL and DML

Q1. Write the SQL commands for the following queries on the given relations. (4-5 queries, 2 marks each.) [8-10 marks · D21 Q1(b), J22 Q1(b), D22 Q1(b), J23 Q1(b), D23 Q1(c), D24 Q1(b), J25 Q1(b), D25 Q2(a)]

Every query ever set falls into one of five patterns.

The five patterns

```

-- A. Simple WHERE filter
SELECT <columns> FROM <table> WHERE <condition>;

-- B. ORDER BY ("in the order of", "alphabetical order", "increasing order")
SELECT <columns> FROM <table> ORDER BY <column> ASC;

-- C. Aggregate over the whole table
SELECT COUNT(*) FROM <table>;                                -- "number of"
SELECT SUM(<col>) FROM <table> WHERE <cond>;                  -- "total of"
SELECT AVG(<col>) FROM <table>;                                -- "average"
SELECT <col> FROM <table>
  WHERE <col2> = (SELECT MAX(<col2>) FROM <table>);             -- "highest"

-- D. GROUP BY ("in each ...", "to each ...")
SELECT <group_col>, COUNT(*) FROM <table> GROUP BY <group_col>;

-- E. Join across two or three relations (the join condition is compulsory)
SELECT a.<cols>, b.<cols>
  FROM A a, B b
  WHERE a.<key> = b.<key> AND <extra filter>;

```

Worked queries, session by session

D21 Q1(b) — Customer(cust_id, name, address), Account(acc_no, branch_code, balance, cust_id), Branch(branch_code, branch_name).

```

-- (i) List all the account numbers in the order of 'name'.      [Pattern B + E]
SELECT a.acc_no, c.name
  FROM Account a, Customer c
  WHERE a.cust_id = c.cust_id
  ORDER BY c.name ASC;

-- (ii) Find the account-number which has the highest balance.   [Pattern C]
SELECT acc_no
  FROM Account
  WHERE balance = (SELECT MAX(balance) FROM Account);

-- (iii) List the branch-code, account-number, name and balance of each account. [Pattern E]
SELECT a.branch_code, a.acc_no, c.name, a.balance
  FROM Account a, Customer c
  WHERE a.cust_id = c.cust_id;

-- (iv) Find the number of accounts in each branch.              [Pattern D]
SELECT branch_code, COUNT(*) AS no_of_accounts
  FROM Account
  GROUP BY branch_code;

```

J22 Q1(b) — Student(enrol_no, name), Course(course_code, title, duration), Registration(enrol_no, course_code).

```

-- (i) Course_code and title of courses whose duration is more than one month. [A]
SELECT course_code, title FROM Course WHERE duration > 1;

-- (ii) Title of the course taken by the student whose enrol_no is 'S01'. [E]
SELECT c.title
FROM Course c, Registration r
WHERE c.course_code = r.course_code
AND r.enrol_no = 'S01';

-- (iii) Count the number of all the courses. [C]
SELECT COUNT(*) AS total_courses FROM Course;

-- (iv) List the enrol_no, name, title, duration for all the students. [E]
SELECT s.enrol_no, s.name, c.title, c.duration
FROM Student s, Registration r, Course c
WHERE s.enrol_no = r.enrol_no
AND r.course_code = c.course_code;

```

D22 Q1(b) — Student(id, name, p_code), Programme(p_code, title, credits).

```

-- (i) id and name of all the students of the programme whose p_code is 'MCA'. [A]
SELECT id, name FROM Student WHERE p_code = 'MCA';

-- (ii) Find the programmes which have more than 80 credits. [A]
SELECT p_code, title FROM Programme WHERE credits > 80;

-- (iii) Find the number of students in each programme. [D]
SELECT p_code, COUNT(*) AS no_of_students FROM Student GROUP BY p_code;

-- (iv) List id, name, p_code, title of all the students of the programme 'CIT'. [E]
SELECT s.id, s.name, s.p_code, p.title
FROM Student s, Programme p
WHERE s.p_code = p.p_code
AND s.p_code = 'CIT';

```

J23 Q1(b) — Hospital(Hospital_ID, Hospital_name, city), Doctor(Doctor_ID, Name, specialization, Hospital_ID).

```

-- (i) Names of all hospitals in alphabetical order of hospital name. [B]
SELECT Hospital_name FROM Hospital ORDER BY Hospital_name ASC;

-- (ii) Number of doctors working in the hospital whose Hospital_ID = 'H01'. [C]
SELECT COUNT(*) AS no_of_doctors FROM Doctor WHERE Hospital_ID = 'H01';

-- (iii) Hospital_ID, Hospital_name, Doctor_ID, Name, specialization of all hospitals. [E]
SELECT h.Hospital_ID, h.Hospital_name, d.Doctor_ID, d.Name, d.specialization
FROM Hospital h, Doctor d
WHERE h.Hospital_ID = d.Hospital_ID;

-- (iv) Name of all the doctors whose specialization is 'Physician'. [A]
SELECT Name FROM Doctor WHERE specialization = 'Physician';

```

D23 Q1(c) — Student(st_id, name, programme_code, prof_id), Programme(programme_code, title, fee), Professor(prof_id, prof_name).

```
-- (i) Name of all students of the programme whose programme_code is 'MCA'.      [A]
SELECT name FROM Student WHERE programme_code = 'MCA';

-- (ii) List all the programmes in the increasing order of programme fee.      [B]
SELECT programme_code, title, fee FROM Programme ORDER BY fee ASC;

-- (iii) Find the total number of programmes of the university.                [C]
SELECT COUNT(*) AS total_programmes FROM Programme;

-- (iv) List st_id, name, prof_name for all the students.                      [E]
SELECT s.st_id, s.name, p.prof_name
  FROM Student s, Professor p
 WHERE s.prof_id = p.prof_id;
```

D24 Q1(b) — Customer(customerno, name, address), Account(accountno, balance, branch),
 Accountholder(customerno, accountno).

```
-- (i) Names of all the customers of the bank, in the order of names.          [B]
SELECT name FROM Customer ORDER BY name ASC;

-- (ii) All the account numbers for which balance is less than Rs. 2,000.      [A]
SELECT accountno FROM Account WHERE balance < 2000;

-- (iii) All the account numbers of the customer whose name is 'XYZ'. [E, 3-relation join]
SELECT a.accountno
  FROM Customer c, Accountholder ah, Account a
 WHERE c.customerno = ah.customerno
    AND ah.accountno = a.accountno
    AND c.name = 'XYZ';

-- (iv) The accountno, customerno and balance of all the accounts.             [E]
SELECT a.accountno, ah.customerno, a.balance
  FROM Account a, Accountholder ah
 WHERE a.accountno = ah.accountno;
```

J25 Q1(b) — Member(memberID, name, phone), Library(book_id, Title, first_author),
 Issue(memberID, book_id, date_of_issue).

```
-- (i) Phone number of member(s) whose name is 'Rakesh Jain'.                [A]
SELECT phone FROM Member WHERE name = 'Rakesh Jain';

-- (ii) Title of all the books issued to the member whose ID is 'M001'.      [E]
SELECT l.Title
  FROM Library l, Issue i
 WHERE l.book_id = i.book_id
    AND i.memberID = 'M001';

-- (iii) Count the number of books issued to each member.                    [D]
SELECT memberID, COUNT(*) AS books_issued FROM Issue GROUP BY memberID;

-- (iv) List all the books in the alphabetical order of book title.           [B]
SELECT book_id, Title, first_author FROM Library ORDER BY Title ASC;
```

D25 Q2(a) — Customer(customercode, name, pincode, phone), Orders(order_id, customercode, price).

```
-- (i) List all the customers whose Pincode is 110001. [A]
SELECT * FROM Customer WHERE pincode = 110001;

-- (ii) Find the total number of customers. [C]
SELECT COUNT(*) AS total_customers FROM Customer;

-- (iii) Total of all the prices of the customer whose CustomerCode is 'C001'. [C]
SELECT SUM(price) AS total_price FROM Orders WHERE customercode = 'C001';

-- (iv) List the customercode, name, order-id of all the customers. [E]
SELECT c.customercode, c.name, o.order_id
FROM Customer c, Orders o
WHERE c.customercode = o.customercode;

-- (v) Find the average price of all the orders. [C]
SELECT AVG(price) AS average_price FROM Orders;
```

Q2. What is SQL ? Explain the CREATE TABLE command of SQL with the help of an example.
What is the purpose of Data Definition Language (DDL) in SQL ?

[4-6 marks · D21 Q2(c), J25 Q3(c), D25 Q1(d)]

SQL (Structured Query Language) is the standard non-procedural, declarative language of relational database systems. The user states *what* result is wanted and the DBMS's optimiser decides *how* to obtain it. Its important features and sub-languages:

- **DDL** — CREATE, ALTER, DROP, TRUNCATE, RENAME: define and change the schema.
- **DML** — SELECT, INSERT, UPDATE, DELETE: manipulate the data.
- **DCL** — GRANT, REVOKE: authorisation.
- **TCL** — COMMIT, ROLLBACK, SAVEPOINT: transaction control.
- Set-oriented (operates on whole relations, not records), closed under its operations (a query returns a relation), supports views, constraints, aggregate functions, grouping, nested queries, joins and embedding in host languages; it is an ANSI/ISO standard, so it is portable across products.

Purpose of DDL. DDL statements define the **structure** of the database — tables, their columns and domains, keys, constraints, views, indexes and schemas — and every definition is recorded in the **data dictionary (system catalogue)**, which the DBMS and the optimiser then consult. DDL changes the *schema*; DML changes the *data*. DDL statements are normally auto-committed.

CREATE TABLE. Syntax:

```
CREATE TABLE <table_name> (
    <column> <data_type> [ (size) ] [ column_constraint ],
    ...
    [ table_constraint ]
);
```

Column constraints: NOT NULL, UNIQUE, PRIMARY KEY, DEFAULT, CHECK, REFERENCES.
Table constraints: PRIMARY KEY(...), FOREIGN KEY(...) REFERENCES ..., UNIQUE(...), CHECK(...).

Example:

```

CREATE TABLE programme (
    prog_code CHAR(6) PRIMARY KEY,
    prog_name VARCHAR(40) NOT NULL,
    fee NUMBER(8,2) CHECK (fee > 0)
);

CREATE TABLE student (
    enrol_no CHAR(9) PRIMARY KEY,
    name VARCHAR(40) NOT NULL,
    dob DATE,
    prog_code CHAR(6) NOT NULL,
    fee_paid NUMBER(8,2) DEFAULT 0,
    CONSTRAINT fk_prog FOREIGN KEY (prog_code)
        REFERENCES programme(prog_code) ON DELETE RESTRICT
);

```

This single statement creates the relation, fixes the domain of every attribute, and declares the entity integrity (PRIMARY KEY, NOT NULL), domain integrity (CHECK, data types) and referential integrity (FOREIGN KEY) constraints that the DBMS will enforce on every subsequent insert and update.

Q3. List and explain any three data definition language (DDL) commands of SQL.

[6 marks · D23 Q4(c)]

DDL commands define and modify the schema of the database; their effect is recorded in the data dictionary.

1. CREATE TABLE — defines a new relation: its columns, their data types and its integrity constraints.

```

CREATE TABLE student (
    enrol_no CHAR(9) PRIMARY KEY,
    name VARCHAR(40) NOT NULL,
    prog_code CHAR(6),
    fee NUMBER(8,2) CHECK (fee > 0),
    FOREIGN KEY (prog_code) REFERENCES programme(prog_code)
);

```

2. ALTER TABLE — changes the definition of an existing relation: add, modify, rename or drop a column, and add or drop a constraint. Existing data is retained; adding a column fills it with NULLs.

```

ALTER TABLE student ADD (phone VARCHAR(15));
ALTER TABLE student MODIFY (name VARCHAR(60));
ALTER TABLE student ADD CONSTRAINT uq_phone UNIQUE (phone);
ALTER TABLE student DROP COLUMN phone;

```

3. DROP TABLE — removes the relation, its data, its indexes and its constraints from the database permanently; it cannot be rolled back.

```

DROP TABLE student;           -- fails if other tables reference it
DROP TABLE student CASCADE CONSTRAINTS; -- also drops the referencing FKs

```

(A fourth DDL command is **TRUNCATE TABLE student**, which removes all rows but keeps the empty table structure. It differs from the DML `DELETE FROM student`: TRUNCATE is a DDL operation, is auto-committed, cannot be rolled back and does not fire row triggers, whereas DELETE logs each row and can be rolled back. **RENAME** is also a DDL command.)

Q4. Explain any FOUR (4) commands of DML with suitable example for each command.

[8 marks · J24 Q2(b)]

DML commands manipulate the *data* inside the relations, leaving the schema unchanged; they operate inside a transaction and can be rolled back.

1. INSERT — adds one or more new tuples to a relation.

```
INSERT INTO student (enrol_no, name, prog_code, fee)
VALUES ('230001234', 'Ravi Kumar', 'MCA', 60000);

-- insert many rows from another relation
INSERT INTO alumni (enrol_no, name)
SELECT enrol_no, name FROM student WHERE status = 'passed';
```

2. SELECT — retrieves tuples; the only read operation, and the richest command (projection, selection, joins, grouping, ordering, sub-queries).

```
SELECT prog_code, COUNT(*) AS strength, AVG(fee) AS avg_fee
FROM student
WHERE fee > 0
GROUP BY prog_code
HAVING COUNT(*) > 10
ORDER BY strength DESC;
```

3. UPDATE — modifies the values of attributes in existing tuples that satisfy the WHERE condition. Omitting WHERE updates every row.

```
UPDATE student
SET fee = fee * 1.10
WHERE prog_code = 'MCA';
```

4. DELETE — removes the tuples satisfying the WHERE condition. The table structure is unaffected; DELETE FROM student; empties the table.

```
DELETE FROM student
WHERE enrol_no = '230001234';
```

All four are followed by **COMMIT** to make the change permanent or **ROLLBACK** to undo it, and each is checked against the entity, referential and domain integrity constraints before being applied — e.g. the DELETE above is rejected if a `result` row still references that enrol_no under ON DELETE RESTRICT.

Q5. Explain the following authorisations on data items in a database system: (i) READ (ii) INSERT (iii) UPDATE. [6 marks · D25 Q4(b)]

Authorisation is the set of privileges the DBMS grants to a user or role on a specified database object, enforced after the user has been authenticated. Privileges are conferred with `GRANT` and withdrawn with `REVOKE`; a privilege may be granted at relation level or restricted to named attributes, and views are used to restrict access to a subset of rows.

(i) READ authorisation — allows the user to *retrieve* data but not to modify it in any way. In SQL this is the `SELECT` privilege. It can be narrowed to specific columns by granting `SELECT` on a view instead of the base table.

```
GRANT SELECT ON Account TO auditor;
CREATE VIEW acc_public AS SELECT acc_no, branch_code FROM Account; -- hides balance
GRANT SELECT ON acc_public TO enquiry_clerk;
```

(ii) INSERT authorisation — allows the user to add *new* tuples to the relation, but not to read or change existing ones. It may be restricted to a column list, in which case the remaining columns take their default values (and must therefore permit NULL or have a DEFAULT).

```
GRANT INSERT ON Account TO new_account_clerk;
GRANT INSERT (acc_no, acc_type, branch_code) ON Account TO new_account_clerk;
```

(iii) UPDATE authorisation — allows the user to modify the values of existing tuples, without the right to insert or delete them. It is the privilege most often restricted to particular attributes, so that a clerk may change an address but not a balance.

```
GRANT UPDATE (address, phone) ON Customer TO front_desk;
GRANT UPDATE (balance) ON Account TO teller;
REVOKE UPDATE ON Account FROM teller;
```

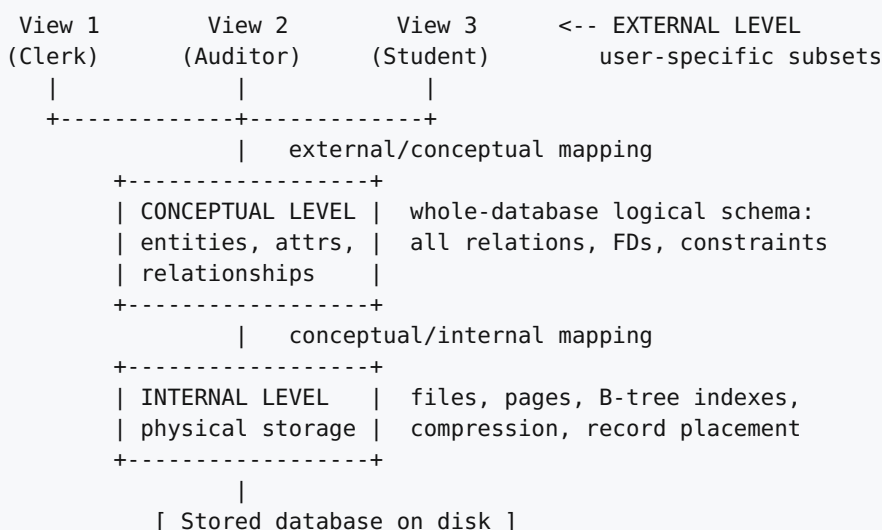
The other privileges in the same family are **DELETE** (remove tuples), **REFERENCES** (create a foreign key pointing at the relation), **EXECUTE** (run a procedure) and the DDL privileges **INDEX**, **ALTER** and **DROP** (called *resource*, *alteration* and *drop* authorisation). **WITH GRANT OPTION** lets the grantee pass a privilege on, which creates an authorisation graph; a **REVOKE** then cascades to every privilege that depended on the revoked one. Granting privileges to **roles** rather than to individual users keeps the scheme manageable, and all privilege checks are recorded in the data dictionary.

DBMS architecture, data independence and fundamentals

Q1. Explain the three level/three layer DBMS architecture with the help of a diagram. How does the three level architecture relate to data independence ?

[6-10 marks · asked 4x: D21 Q3(b) 8, J22 Q3(a) 6, J23 Q2(a)(i) 2, D24 Q2(c) 6]

The ANSI/SPARC three-level architecture separates the user's view of data from its storage.



- **External level** hides the rest of the database from a user; e.g. the view `STUDENT PUBLIC(enrol no, name, programme)` hides fee and marks.

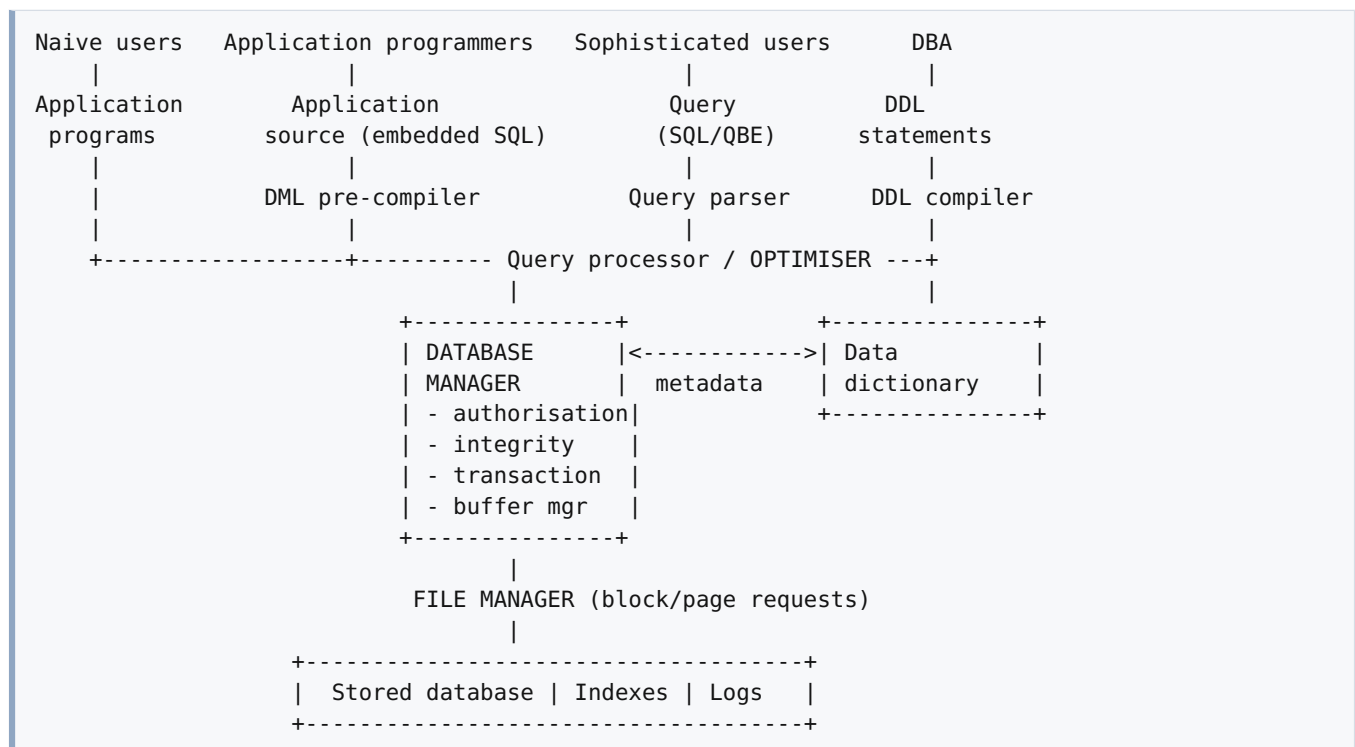
- **Conceptual level** describes what data is stored and how it relates, without storage detail:
STUDENT(enrol_no, name, programme, fee).
- **Internal level** says the relation is a hashed file on enrol_no with a secondary B-tree on name.

Relation to data independence. The two mappings are the insulation layers. **Logical data independence** comes from the external/conceptual mapping: adding attribute email to STUDENT, or splitting the relation in two, only requires the mapping to be redefined, so existing views and application programs still compile. **Physical data independence** comes from the conceptual/internal mapping: replacing a heap file by a B-tree index, moving the file to a new disk, or re-ordering records changes only the internal schema; the conceptual schema and every application remain untouched.

Q2. Explain the physical DBMS architecture with the help of a neat diagram.

[10 marks · asked 1x: J24 Q2(a)]

Physical (component) architecture shows the modules a DBMS uses to turn user requests into disk accesses.



Roles: the **DDL compiler** converts schema definitions into data-dictionary entries; the **DML pre-compiler** extracts embedded SQL from host programs; the **query parser and optimiser** translate SQL into a relational-algebra evaluation plan of least estimated cost; the **database manager** is the central interface that applies authorisation checks, integrity checks, concurrency control (locks) and recovery (log writes), and manages the buffer pool; the **file manager** maps logical records to physical blocks and allocates space; the **data dictionary** stores schemas, indexes, users and privileges.

Q3. Briefly explain the roles of Database Manager in the context of physical DBMS architecture.

[5 marks · asked 2×: J25 Q2(a), D25 Q1(a)]

The database manager is the control module that sits between the query processor and the file manager; every request for stored data passes through it. Its roles are:

- **Authorisation control** — checks the user's privileges in the data dictionary before any access, rejecting unauthorised `SELECT/UPDATE`.
- **Integrity checking** — enforces domain, entity and referential constraints, e.g. refusing an `ACCOUNT` row whose `cust_id` has no matching `CUSTOMER`.
- **Transaction management and concurrency control** — begins/commits transactions, grants and releases shared and exclusive locks so that concurrent schedules stay serialisable.
- **Recovery management** — writes `<Ti, X, old, new>` log records before updates and performs undo/redo after a failure.
- **Buffer and command processing** — moves pages between disk and the buffer pool, calls the file manager, and passes back the requested records.

Q4. Explain the concept of physical data independence and logical data independence with the help of an example of each. [4–6 marks · asked 2×: J22 Q4(b) 6, D22 Q2(a) 4]

Data independence is the capacity to change a schema at one level without altering the schema at the next higher level or the application programs.

Physical data independence — the internal (storage) schema can change without changing the conceptual schema. Example: `EMPLOYEE(emp_no, name, dept, salary)` is stored as an unordered heap file. The DBA creates a B-tree index on `emp_no` and moves the file to an SSD. Queries such as `SELECT name FROM EMPLOYEE WHERE emp_no = 'E10'`; run faster but are not rewritten; only the conceptual/internal mapping changes.

Logical data independence — the conceptual schema can change without changing external views or applications. Example: attribute `email` is added to `EMPLOYEE`, or `EMPLOYEE` is split into `EMP_BASIC` and `EMP_PAY`. The view `V_EMP(emp_no, name)` is redefined over the new schema, so the payroll report using `V_EMP` continues to work unchanged. Logical independence is harder to achieve than physical independence.

Q5. Explain the following in the context of DBMS with the help of an example: (i) The conceptual level in three level database architecture (ii) Database Administrator (iii) File Manager (iv) Data dictionary (v) Hierarchical model. [10 marks · asked 1×: J23 Q2(a)]

(i) Conceptual level. The community logical view of the entire database: all entities, attributes, relationships and constraints, with no storage detail and no user-specific restriction. Example: `STUDENT(enrol_no, name, prog_code)`, `PROGRAMME(prog_code, title, fee)` with `prog_code` a foreign key. It is defined by the DBA using DDL.

(ii) Database Administrator (DBA). The person responsible for the database: defines the conceptual and internal schemas, grants and revokes privileges (`GRANT SELECT ON student TO clerk;`), tunes performance by creating indexes, plans backup and recovery, and resolves conflicting user requirements.

(iii) File Manager. The DBMS module that manages disk space and the data structures used to represent stored information. It translates a request for a logical record into a request for specific

blocks/pages, maintains free-space lists and passes blocks to the buffer. Example: fetching the block holding `enrol_no='S01'`.

(iv) Data dictionary. The system catalogue holding metadata: relation and attribute names, data types, constraints, index definitions, views, users and privileges, and statistics used by the optimiser. Example: Oracle's `USER_TABLES`, `USER_TAB_COLUMNS`.

(v) Hierarchical model. An early model in which records form a tree: each child record has exactly one parent, navigated by pointers. Example: a `DEPARTMENT` root with `EMPLOYEE` children. It represents 1:N well but needs record duplication for M:N relationships and offers no declarative query language.

Q6. Data dictionary (short note). [2–5 marks · asked 2×: J23 Q2(a)(iv), D23 Q5(iii)]

A data dictionary (system catalogue, metadata repository) is the part of the database that stores data **about** the data. It records relation names, attribute names with domains and lengths, primary/foreign key and check constraints, index and view definitions, storage structures, user accounts and their privileges, and statistics such as number of tuples and distinct values. It is itself stored as relations and queried with SQL, for example `SELECT column_name, data_type FROM USER_TAB_COLUMNS WHERE table_name='STUDENT'`; . It is used by the DDL compiler when schemas are defined, by the database manager for authorisation and integrity checks, and by the query optimiser for cost estimation. An *active* dictionary is updated automatically by the DBMS; a *passive* one is maintained separately. Benefits: documentation, impact analysis, consistency of naming and reduced redundancy.

Q7. Compare and contrast the traditional file based system with the database approach / describe file based system and list the limitations of it.

[5–6 marks · asked 3×: D21 Q2(a) 6 ("List and explain any six advantages of database management system"), J24 Q1(a) 5, J26 Q1(a) 5]

In a file-based system each application owns its private files and embeds the record structure in its own code.

File-based system	Database approach (DBMS)
Data duplicated across application files	Single integrated store, redundancy controlled
Inconsistency: an address changed in one file only	Consistency enforced by one copy plus constraints
Structure hard-coded in programs — no data independence	Logical and physical data independence via three-level architecture
No integrity or constraint enforcement beyond program code	Declarative domain, entity and referential constraints
Security per file at OS level only	Per-user, per-table, per-column <code>GRANT/REVOKE</code> , views
No concurrency control; concurrent updates corrupt data	Locking/2PL gives serialisable concurrent access
No general recovery; restore whole file from tape	Log-based recovery, checkpoints, rollback
Queries need a new program each time	Ad-hoc querying with SQL

Six advantages of a DBMS: control of redundancy, data consistency, data sharing among many users, integrity enforcement, security and authorisation, backup and recovery — plus data independence and ad-hoc query support.

Q8. Discuss the role of data models in developing DBMS. Compare conceptual models with object based models. [5 marks · asked 1x: J26 Q1(b)]

A data model is a set of concepts for describing data, its relationships, semantics and constraints. It is the design language of a DBMS: the **conceptual/high-level model** (ER, EER) captures user requirements independently of any product; the **representational/implementation model** (relational, network, hierarchical) is what the DBMS actually supports and the target of ER-to-relation mapping; the **physical model** describes record formats and access paths. Models thus give a notation for communication, a basis for schema design and normalisation, and a foundation for the three-level architecture.

Conceptual (ER/EER) models	Object-based models
Describe entities, attributes, relationships	Describe objects with state and behaviour (methods)
Design-time only; no execution semantics	Directly implementable in an OODBMS/ORDBMS
Identity by key values	Identity by system-generated OID
Only attributes; atomic values preferred	Complex data types, inheritance, reference types

Relational model and relational algebra

Q1. Explain the following basic relational operations with the help of an appropriate example for each: SELECTION, PROJECTION, CARTESIAN PRODUCT, JOIN (J23: UNION, SET DIFFERENCE instead of JOIN). [10 marks · asked 2x: J23 Q2(b), J24 Q1(e)]

Base relations:

STUDENT	enrol_no	name	prog	marks
	S01	Amit	MCA	75
	S02	Bina	BCA	62
	S03	Chitra	MCA	55

PROG	prog	title
	MCA	Master of Comp. App.
	BCA	Bachelor of Comp. App.

Selection (σ) — picks tuples satisfying a predicate; degree unchanged, cardinality reduced.

$\sigma(\text{marks} > 60)(\text{STUDENT}) = \{(S01, \text{Amit}, \text{MCA}, 75), (S02, \text{Bina}, \text{BCA}, 62)\}$

Projection (π) — picks columns and removes duplicate tuples.

$\pi(\text{prog})(\text{STUDENT}) = \{(\text{MCA}), (\text{BCA})\}$

Cartesian product ($\times$) — every tuple of R paired with every tuple of S; degree = sum of degrees, cardinality = $3 \times 2 = 6$ tuples; includes meaningless pairs such as (S01, Amit, MCA, 75, BCA, Bachelor...).

Join ($\bowtie$) — a Cartesian product followed by a selection on a comparison; the natural join equates same-named attributes and keeps one copy.

```
STUDENT  $\bowtie$  PROG =  
(S01, Amit, MCA, 75, Master of Comp. App.)  
(S02, Bina, BCA, 62, Bachelor of Comp. App.)  
(S03, Chitra, MCA, 55, Master of Comp. App.)
```

Union ($\cup$) — for union-compatible relations, all tuples of both with duplicates removed: $\pi(\text{prog})(\text{STUDENT}) \cup \{(MBA)\} = \{(MCA), (BCA), (MBA)\}$.

Set difference ($-$) — tuples in R not in S: $\pi(\text{enrol_no})(\text{STUDENT}) - \pi(\text{enrol_no})(\sigma(\text{marks} > 60)(\text{STUDENT})) = \{(S03)\}$.

Q2. Why do we need relational algebra in DBMS ? List the Relational operations and Binary set operations. [5 marks · asked 1×: J26 Q1(d)]

Relational algebra is the formal, procedural query language of the relational model. It is needed because (a) it gives a precise mathematical semantics to queries, (b) a declarative SQL statement is translated by the parser into an algebra expression, which is the object the optimiser manipulates, (c) algebraic equivalence rules (push selections below joins, combine projections) make query optimisation possible, and (d) it is closed — every operation returns a relation, so operations compose.

Relational (unary/relational-specific) operations: SELECT σ , PROJECT π , RENAME ρ , JOIN $\bowtie$ (theta, equi, natural, outer), DIVISION $\div$, and the aggregate/grouping extension.

Binary set operations: UNION $\cup$, INTERSECTION $\cap$, SET DIFFERENCE $-$, and CARTESIAN PRODUCT $\times$. The first three require union-compatibility (same degree, corresponding domains); $\times$ does not.

Q3. Consider relations $P(\text{Pid}, \text{Pname}) = \{001 \text{ abc}, 012 \text{ xyz}, 014 \text{ lmn}, 015 \text{ opq}, 017 \text{ ssd}\}$ and $Q(\text{Pid}, \text{Pname}) = \{012 \text{ xyz}, 014 \text{ lmn}, 016 \text{ sss}, 017 \text{ ssd}\}$. Find: (i) $P \cup Q$ (ii) $P \times Q$.

[5 marks (2+3) · asked 1×: D23 Q1(g)]

(i) $P \cup Q$ — union-compatible, so all tuples of both with duplicates (012, 014, 017) kept once; $5 + 4 - 3 = 6$ tuples.

```
P  $\cup$  Q = { (001, abc), (012, xyz), (014, lmn), (015, opq), (017, ssd), (016, sss) }
```

(ii) $P \times Q$ — degree 4, schema (P.Pid, P.Pname, Q.Pid, Q.Pname), cardinality $5 \times 4 = 20$ tuples.

```
(001,abc,012,xyz) (001,abc,014,lmn) (001,abc,016,sss) (001,abc,017,ssd)  
(012,xyz,012,xyz) (012,xyz,014,lmn) (012,xyz,016,sss) (012,xyz,017,ssd)  
(014,lmn,012,xyz) (014,lmn,014,lmn) (014,lmn,016,sss) (014,lmn,017,ssd)  
(015,opq,012,xyz) (015,opq,014,lmn) (015,opq,016,sss) (015,opq,017,ssd)  
(017,ssd,012,xyz) (017,ssd,014,lmn) (017,ssd,016,sss) (017,ssd,017,ssd)
```

Q4. Explain the Cartesian product and theta join operations in the context of relational algebra, with the help of an example of each. [5 marks · asked 1×: J25 Q2(c)]

EMP	eno	name	dno
	E1	Amit	D1
	E2	Bina	D2

DEPT	dno	dname
	D1	Sales
	D2	Audit

Cartesian product ($R \times S$) concatenates every tuple of R with every tuple of S. Degree = degree(R) + degree(S), cardinality = $|R| \times |S|$. It applies no condition, so most result tuples are semantically meaningless and it is the costliest operator.

```
EMP × DEPT = (E1,Amit,D1,D1,Sales) (E1,Amit,D1,D2,Audit)
              (E2,Bina,D2,D1,Sales) (E2,Bina,D2,D2,Audit)    -- 2×2 = 4 tuples
```

Theta join ($R \bowtie_{\theta} S = \sigma_{\theta}(R \times S)$) keeps only those concatenated tuples that satisfy a condition θ built from =, $\neq$, <, $\leq$, >, $\geq$ and the connectives AND/OR. When θ uses only equality it is an **equi-join**.

```
EMP ⋈ (EMP.dno = DEPT.dno) DEPT = (E1,Amit,D1,D1,Sales) (E2,Bina,D2,D2,Audit)
```

Q5. What is a natural join operation ? Explain any one algorithm that can be used to implement the join operation in an RDBMS. / Join Operation (short note).

[4-6 marks · asked 2×: J22 Q5(a) 4, D22 Q4(c) 6]

A **natural join** $R \bowtie S$ is an equi-join on *all* attributes with the same name in R and S, with the duplicate copy of each join attribute projected away. For STUDENT(enrol_no, name, prog) and PROG(prog, title), STUDENT $\bowtie$ PROG matches on prog and returns (enrol_no, name, prog, title). Tuples with no match are lost (that is what outer joins restore).

Nested-loop join algorithm

```
for each block B_r of r:                -- outer relation
  for each block B_s of s:                -- inner relation
    for each tuple t_r in B_r:
      for each tuple t_s in B_s:
        if t_r.join_attr = t_s.join_attr then output t_r · t_s
```

Cost (block nested loop, one buffer block per relation) = $b_r + b_r \times b_s$ block transfers + $2 \cdot b_r$ seeks; if the smaller relation fits in memory it falls to $b_r + b_s$. It needs no index or sorting and works for any join condition, but is quadratic, so RDBMSs prefer **indexed nested loop** ($b_r + n_r \times c$), **sort-merge** or **hash join** ($3(b_r + b_s)$) when applicable.

Q6. Explain the self join and outer join. [6 marks · asked 1×: D24 Q3(c)]

Self join joins a relation with itself, using two aliases, to relate tuples of the same relation. In EMP(eno, name, mgr_no):

```
SELECT e.name AS employee, m.name AS manager
FROM emp e, emp m
WHERE e.mgr_no = m.eno;
```

Algebraically $\rho_e(EMP) \bowtie (e.mgr_no = m.eno) \rho_m(EMP)$. It is required for recursive/hierarchical relationships (employee-manager, part-subpart).

Outer join extends the natural/theta join by preserving non-matching ("dangling") tuples and padding the missing side with NULLs, so no information is lost. Three forms, with STUDENT(S01 Amit MCA; S04 Deepa NULL) and PROG(MCA ...; BCA ...):

- **LEFT outer join ($\ltimes$)** — all STUDENT tuples; S04 appears with NULL title.
- **RIGHT outer join ($\rtimes$)** — all PROG tuples; BCA appears with NULL enrol_no and name.
- **FULL outer join ($\ltimes\rtimes$)** — the union of both, so both S04 and BCA appear padded.

Outer joins are used in reports that must list every row of a table, e.g. all departments including those with no employees.

Q7. Explain the relational model of database management system / Relational Model (short note). [5–8 marks · asked 2x: D21 Q1(d) 8, D24 Q5(g) 5]

Proposed by E. F. Codd (1970), the relational model represents the database as a collection of **relations** (two-dimensional tables). Terminology: a **relation schema** $R(A_1:D_1, \dots, A_n:D_n)$, e.g. STUDENT(enrol_no, name, prog, fee); a **tuple** is a row; an **attribute** is a column; a **domain** is the set of atomic values an attribute may take; **degree** is the number of attributes (4 here); **cardinality** is the number of tuples; a **relation instance** is the current set of tuples, while the schema is the time-invariant definition.

Structural properties: all values atomic (1NF), no duplicate tuples (a relation is a set), tuple order and attribute order immaterial, each attribute's values drawn from one domain, NULL for unknown/inapplicable.

Constraints: **domain**, **key** (super key, candidate key, primary key — unique and minimal), **entity integrity** (primary key not NULL), **referential integrity** (a foreign key value must match an existing primary key value or be NULL).

Relationships are represented by *values* (foreign keys), not pointers, which is why the model supports declarative querying by relational algebra, relational calculus and SQL, and gives high physical data independence.

Q8. What is a relational schema ? Explain with the help of an example. Define the terms super key, candidate key and primary key with the help of an example. [5 marks · asked 1x: J25 Q2(b)]

A **relational schema** is the time-invariant definition of a relation: its name, its attributes with domains, and its constraints, written $R(\underline{A_1}, A_2, \dots, A_n)$ with the primary key underlined. Example:

```
STUDENT(enrol_no, name, phone, prog_code)    -- enrol_no is PK
PROGRAMME(prog_code, title, fee)             -- prog_code is PK
STUDENT.prog_code -> PROGRAMME.prog_code     -- FK, referential integrity
```

A **database schema** is the set of all such relation schemas; a **relation instance** is the set of tuples currently stored.

Using `STUDENT(enrol_no, name, phone, prog_code)` where `enrol_no` and `phone` are each unique:

- **Super key** — any attribute set whose values uniquely identify a tuple, not necessarily minimal: `{enrol_no}`, `{enrol_no, name}`, `{phone, prog_code}`, `{enrol_no, name, phone, prog_code}`.
- **Candidate key** — a *minimal* super key (no proper subset is a super key): `{enrol_no}` and `{phone}`.
- **Primary key** — the candidate key chosen by the designer to identify tuples, declared NOT NULL and unique: `enrol_no` (chosen because a phone number may change). The remaining candidate key, `phone`, becomes an alternate key.

Q9. Explain the concept of super key, candidate key and primary key in a relational database system, with the help of an example.

[5-6 marks · asked 4x: D22 Q2(b) 6, J25 Q2(b) 5, D25 Q1(b)(iii) ("Candidate keys"), J26 Q1(c) 5 ("Define super key and primary key. Ca

Consider `EMPLOYEE(emp_id, pan_no, name, dept, salary)` with `emp_id` and `pan_no` unique.

- **Super key:** a set of attributes K such that no two tuples have the same value of K. `{emp_id}`, `{pan_no}`, `{emp_id, name}`, `{pan_no, dept, salary}` are super keys. Super keys need not be minimal, so `{emp_id, name}` qualifies although `name` adds nothing.
- **Candidate key:** a minimal super key — removing any attribute destroys uniqueness. Here `{emp_id}` and `{pan_no}`.
- **Primary key:** the one candidate key selected to identify tuples and to be referenced by foreign keys; it is automatically UNIQUE and NOT NULL (entity integrity). Choose `emp_id`; `pan_no` is then an alternate key.

Can a composite key be a primary key ? Yes. If no single attribute is unique, the minimal attribute set that is unique is composite and becomes the primary key. Example: `ISSUE(memberID, book_id, date_of_issue)` — neither `memberID` nor `book_id` is unique on its own because a member borrows many books and a title is borrowed by many members, but `{memberID, book_id}` is unique and minimal, so it is the composite primary key. Both components must then be NOT NULL.

Q10. Explain the following terms in the context of Relations: (i) Domain (ii) Tuple (iii) Candidate keys (iv) Entity Integrity constraint (v) Domain constraints. [5 marks · asked 1x: D25 Q1(b)]

(i) Domain — the named set of atomic values permitted for an attribute, with a data type and format. `grade` has domain `{A, B, C, D, F}`; `fee` has domain `NUMBER(8,2) ≥ 0`.

(ii) Tuple — one row of a relation, an ordered list of values, one per attribute, drawn from the corresponding domains: `(S01, Amit, MCA, 75)` in `STUDENT(enrol_no, name, prog, marks)`. The number of tuples is the relation's cardinality; a relation contains no duplicate tuples.

(iii) Candidate key — a minimal set of attributes that uniquely identifies every tuple. In `STUDENT(enrol_no, name, phone, prog)` with unique phone numbers, `{enrol_no}` and `{phone}` are candidate keys; one is chosen as primary key, the rest are alternate keys.

(iv) Entity integrity constraint — no attribute of a primary key may be NULL (and the key must be unique), because a NULL identifier cannot identify a tuple. Inserting a `STUDENT` row with `enrol_no` NULL is rejected.

(v) Domain constraint — every value of an attribute must belong to that attribute's domain. `CHECK (marks BETWEEN 0 AND 100)` rejects `marks = 150`; inserting 'abc' into a `NUMBER` column is also a domain violation.

File organisation and indexing

Q1. What is a sequential file ? What is the structure of a sequential file ? Explain with the help of a diagram. List the advantages and disadvantages of using sequential files.

[2–10 marks · asked 3x: D23 Q2(a) 10, J24 Q2(c) 2 ("What is the use of sequential file organization ?"), D24 Q2(a) 8]

A **sequential (ordered/sorted) file** stores records physically in ascending order of a chosen *ordering field*, usually the key. Records are placed consecutively in blocks; new records that would break the order are written to an **overflow area** linked to the main area, and the file is periodically reorganised (merged and rewritten).



Access cost. Binary search on the ordering key: $\lceil \log_2 b \rceil$ block transfers for b blocks — e.g. 10 accesses for 1000 blocks instead of 1000. Searching on any *other* attribute, or scanning the overflow area, costs $O(b)$. Ordered retrieval of the whole file is one sequential sweep.

Advantages	Disadvantages
Binary search on the ordering key — fast equality and range search	Insertion and deletion are expensive: records must be shifted
Records retrieved in sorted order without sorting; ideal for reports and batch payroll/billing runs	Overflow area degrades performance until reorganisation
Efficient sequential/batch processing with merge updates	Searching on a non-ordering attribute is a linear scan
Simple, no index overhead, good tape/disk block utilisation	Poor for interactive random access and frequent updates

Q2. What is hash file organisation ? Explain with the help of a diagram.

[5 marks · asked 1x: J25 Q2(d)]

In a **hash (direct) file**, a hash function h is applied to the value of a hash field of a record and returns the address of the bucket (one or more blocks) in which that record is stored, so a record is located by computation rather than search. Example: $h(\text{enrol_no}) = \text{enrol_no} \bmod 5$.



Equality search, insert and delete cost ~ 1 block transfer ($O(1)$) when buckets are not overfull. Collisions (two keys hashing to the same bucket) are handled by overflow chaining, open addressing or multiple hashing; **static hashing** fixes the bucket count and degrades as the file grows, so **dynamic/extendible hashing** splits buckets on demand. Limitation: records are not stored in key order, so range queries and ordered retrieval require a full scan — there a B+-tree index is better.

Q3. Describe Unordered Heap file organization. Give advantages and disadvantages of heap files. [5 marks · asked 1x: J26 Q1(g)]

In a **heap (pile/unordered) file** a new record is simply appended at the end of the last block, or into any block with free space listed in the free-space map; no ordering or hashing is imposed.

```

+-----+ +-----+ +-----+
| Block 1 | | Block 2 | | Block 3 (last) |
| S07 | S01 | S12 | --> | S03 | --- | S09 | --> | S15 | free | free | <-- insert here
+-----+ +-----+ +-----+
free-space list: Block 2 slot 2, Block 3 slots 2-3

```

Deletion marks the record with a deletion bit or adds its slot to the free list, leaving holes until the file is compacted.

Advantages	Disadvantages
Insertion is $O(1)$: append to the last block	Any search is a linear scan: $b/2$ blocks on average, b for a non-key or unsuccessful search
Simple to implement, no reorganisation on insert	Deletion leaves unused holes, wasting space until compaction
Best when the whole file is scanned anyway, or for bulk loading and small files	Records not in sorted order; ordered or range retrieval needs an external sort

Q4. Multi-list file organization (short note). [5 marks · asked 1x: J24 Q5(i)]

A multi-list file supports queries on several secondary keys by threading **linked lists through the data records themselves**. For each secondary key value there is one index entry holding the head pointer and list length, and every record carries one pointer field per secondary key, chaining it to the next record with the same value of that key. Example on `EMPLOYEE(eno, name, dept, grade)`: the `dept` index has entries `Sales → E1 → E4 → E7 → NULL` and `Audit → E2 → E5 → NULL`, while an independent `grade` chain runs through the same records.

Advantages: a query such as "all employees of Sales" follows one chain with no sorting, no separate pointer file is needed, and insertion only relinks pointers. Disadvantages: records are enlarged by a pointer per indexed key, deletion requires updating predecessors in every chain, and lists are traversed one record (one block transfer) at a time, so long chains are slow. A **coral ring** variant makes the chains circular and doubly linked to ease deletion.

Q5. Compare fixed length records with variable length records. How is the storage of these two types of records performed on HDD or SSD ? Discuss. [5 marks · asked 1×: J26 Q1(f)]

Fixed-length records	Variable-length records
Every record occupies the same number of bytes	Record size differs per record
Arises from fixed-size types: CHAR(10) , NUMBER(6)	Arises from VARCHAR , repeating groups, optional/NULL fields, multiple record types in one file
i-th record starts at offset (i-1)×L — direct computation	Offset must be read from a header or directory
Simple deletion: mark slot free, reuse via a free list	Deletion/update may not fit in place; causes fragmentation
Wastes space on padding	Space-efficient but needs length/offset bookkeeping

Storage. Both are stored in fixed-size blocks/pages (e.g. 4 KB). Fixed-length records are packed $\lceil B/L \rceil$ per block, giving a *blocking factor* and leaving unused bytes at the block end (or spanned records if blocks are packed across boundaries). Variable-length records use a **slotted-page** organisation: a block header holds the record count, the end-of-free-space pointer and an array of (offset, length) slots; records grow from the end of the block towards the middle, so a record can be moved within the block without changing its external identifier (block, slot). Each field is delimited by an end-marker or preceded by a length, and long values are stored as overflow/LOB pages.

On an **HDD** the cost is dominated by seek and rotational latency, so records of one relation are clustered into consecutive cylinders and large sequential block transfers are preferred; reorganisation to remove fragmentation pays off. On an **SSD** there is no seek, random reads are nearly as cheap as sequential ones, but writes happen in pages and erases in large blocks, so in-place updates of variable-length records are costly; SSD-aware DBMSs prefer append-only/log-structured writing and leave garbage collection and wear-levelling to the FTL.

Q6. What are primary index and secondary index ? Which of them is more advantageous ? Explain with the help of an example for each.

[4-6 marks · asked 4×: D21 Q2(b) 6, J22 Q5(c) 4, D22 Q2(d) 5, J26 Q3(b)(ii) ("Differentiate between Primary Indexes and Secondary Indexes")]

A **primary index** is built on the *ordering key* of a sequentially organised file. It is normally **sparse**: one entry per block, holding the value of the key in the block's first (anchor) record and that block's address.

Primary index (sparse)	Data file ordered on enrol_no
+-----+-----+	+-----+
S01 ----->	S01 Amit S03 Bina Block 1
S07 ----->	S07 Chand S09 Dev Block 2
S12 ----->	S12 Esha S15 Farah Block 3
+-----+-----+	+-----+

A **secondary index** is built on a non-ordering attribute (e.g. `name`, or `city`). It must be **dense**: one entry per distinct value, and for a non-unique attribute each entry points to a bucket of record pointers, giving an extra level of indirection.

```

Secondary index on city (dense)    -> pointer buckets -> data records anywhere
| Agra | -> [ptr, ptr, ptr] -----> rows scattered over the file
| Delhi | -> [ptr, ptr]

```

Primary index	Secondary index
On ordering key; file physically sorted on it	On any non-ordering attribute
Sparse, one entry per block — small index	Dense, one entry per record/value — large index
Only one per file	Many per file
Range queries very efficient (data contiguous)	Range queries fetch scattered blocks, one per matching record

Which is more advantageous ? The primary index. It is smaller (fewer levels, so fewer block transfers — a 3-level sparse index reaches a record in 4 accesses), it needs no bucket indirection, and because the data is clustered in key order a range query of k matching records costs about k/f blocks instead of k . The secondary index is nevertheless indispensable: it is the only way to avoid a full linear scan on non-key attributes, and a file can carry many of them. Cost: every secondary index must be updated on every insert/delete/update of its attribute.

Q7. Why do we need indexes ? Explain the following: (i) Primary index (ii) Clustering index (iii) Secondary index. [7 marks · asked 1x: J24 Q3(c)]

Why indexes. Without an access path, finding a record means a linear scan of all b blocks of the file. An index is an auxiliary (search-key, pointer) structure, itself much smaller than the data file and usually a multi-level B+-tree, that reduces the search to the index height plus one block transfer. Indexes thus speed up selections, joins, ORDER BY and uniqueness checking; the price is storage and the cost of maintaining them on every update.

(i) Primary index — on the *primary/ordering key* of a file sorted on that key. **Sparse:** one entry (first key in the block, block address) per block. Search: binary search in the index, then one data-block read. Only one per file, because a file can have only one physical order. Example: index on `enrol_no` of `STUDENT` stored in `enrol_no` order.

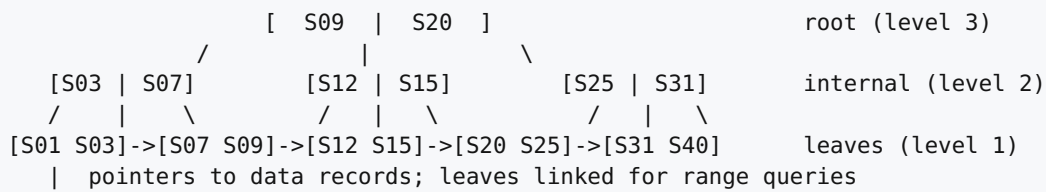
(ii) Clustering index — on an ordering field that is **not** unique, so many records share a value. One entry per *distinct* value, pointing to the first block containing that value; the matching records occupy consecutive blocks. Only one per file. Example: `STUDENT` stored in `prog_code` order with a clustering index on `prog_code`; all MCA records are read as one contiguous run.

(iii) Secondary index — on a non-ordering attribute, unique or not. **Dense,** with one entry per distinct value; non-unique values point to a bucket of record pointers. Many per file. Example: index on `name`. It avoids a full scan, but since the matching records are scattered, each retrieved record may cost a separate block transfer.

Q8. Explain the different types of indexes used in file organization. How do index structures like B-trees or hash indexes enhance data access efficiency ? [10 marks · asked 1x: D25 Q3(a)]

Types of index. - **Primary (sparse)** — on the ordering key; one entry per block; one per file. - **Clustering** — on a non-unique ordering field; one entry per distinct value; one per file. - **Secondary (dense)** — on a non-ordering attribute; one entry per value plus pointer buckets; many per file. - **Dense vs sparse** — dense has an entry for every search-key value (larger index, no data scan needed); sparse has an entry per block (smaller, needs the file to be ordered). - **Single-level vs multi-level** — a single-level index may itself be too large to search in memory, so an index is built on the index, recursively, until the top level fits in one block: this is the B+-tree. - **Ordered (tree) vs hash** — ordered indexes support range and ordered access; hash indexes support equality only.

B+-tree index. A balanced multi-way tree of order n : internal nodes hold up to $n-1$ keys and n child pointers and act as a routing directory; all data pointers sit in the leaves, which are chained left-to-right for range scans. Every node is at least half full, so the height is at most $\lceil \log[n/2] K \rceil$ for K keys, and the tree stays balanced under insertion (node split) and deletion (merge/redistribute).



With $n = 100$ and 1,000,000 keys the height is 3, so a record is found in 3 index block transfers + 1 data transfer $\approx$ **4 block accesses** instead of scanning, say, 50,000 blocks. A range query `WHERE enrol_no BETWEEN 'S07' AND 'S20'` descends once and then walks the leaf chain.

Hash index. $h(\text{key})$ computes the bucket address directly, so an equality search is **1 block transfer** ($O(1)$), independent of file size, and insert/delete are equally cheap; extendible hashing keeps this true as the file grows. But hashing destroys order, so range queries, `ORDER BY` and partial-key searches degenerate to a full scan.

Efficiency summary: use a B+-tree for range, ordered and mixed workloads and a hash index for pure equality lookups and hash joins; both convert an $O(b)$ scan into a small constant number of block transfers, at the cost of extra storage and update overhead.

Query processing and optimisation

Q1. What is query processing ? List the basic steps in query processing.

[4-5 marks · asked 3x: D23 Q5(iv) 5 ("Query processing"), J24 Q1(c) 5, D25 Q4(c) 4 ("List the steps of query processing in the context

Query processing is the set of activities by which a DBMS extracts the data requested by a high-level query: translating the query into an internal low-level form, choosing an efficient strategy for it, and executing that strategy.

```
SQL query
|
(1) PARSING AND TRANSLATION --> checks syntax, and names/types against
|                               the data dictionary; builds a parse tree
(2) relational-algebra expression
|
(3) OPTIMISER <---- statistics from the data dictionary
|                   (relation sizes, index availability, selectivity)
(4) execution plan (annotated algebra tree: which algorithm per operator)
|
(5) EVALUATION ENGINE --> reads the database, materialises or pipelines
|                           intermediate results
query output
```

The steps are therefore: **(i) parsing and translation, (ii) conversion to a relational-algebra expression, (iii) optimisation — generate equivalent expressions and evaluation plans and pick the cheapest by estimated cost, (iv) code generation of the chosen plan, and (v) evaluation/execution** to produce the result.

Q2. What is meant by the term "Query Optimisation" ? Explain with the help of an example. How is query optimisation different from query evaluation ? [6–7 marks · asked 2×: D21 Q4(b) 6, J22 Q4(a) 7]

Query optimisation is the DBMS activity of choosing, among the many equivalent evaluation plans for a query, the one with the least estimated cost (block transfers + seeks). It has two parts: **algebraic (heuristic) optimisation**, which rewrites the expression using equivalence rules — perform selections as early as possible, perform projections early, replace $\times$ followed by σ with a join, combine cascaded selections, order joins so the smallest intermediate result is produced first — and **cost-based optimisation**, which uses dictionary statistics to pick an algorithm per operator (linear scan vs index scan, nested-loop vs hash join).

Example. STUDENT (10,000 tuples) and PROGRAMME (20 tuples):

```
SELECT s.name FROM student s, programme p
WHERE s.prog_code = p.prog_code AND p.title = 'MCA';
```

Unoptimised: $\pi(\text{name})(\sigma(\text{title}='MCA' \wedge \text{s.prog_code}=\text{p.prog_code})(\text{STUDENT} \times \text{PROGRAMME}))$ — the Cartesian product builds $10,000 \times 20 = 200,000$ tuples before any filtering.

Optimised (selection pushed down, product replaced by a join):

```
 $\pi(\text{name})(\text{STUDENT} \bowtie (\text{prog\_code}) \pi(\text{prog\_code})(\sigma(\text{title}='MCA')(\text{PROGRAMME})))$ 
```

The selection leaves 1 tuple, so the join inspects $10,000 \times 1$ pairs and produces only the MCA students — orders of magnitude cheaper, with the same result.

Difference. Optimisation happens *before* execution and *decides* how the query will be run: it generates equivalent expressions, estimates their costs from statistics and selects a plan; it reads only metadata. Query evaluation (execution) *carries out* the chosen plan on the actual data using the implementation algorithms, reading blocks, pipelining or materialising intermediate relations and returning tuples. Optimisation is a planning/search problem; evaluation is the I/O-bound work.

Q3. What is a measure of query cost ? How can you compute the cost of the SELECTION operation on a relational table, where data is not sorted in the order of the attribute on which selection is being performed ? Explain. [5 marks · asked 2×: J23 Q4(b) 5, D22 Q1(c)(v) 2 ("Query cost")]

The cost of a query is the total resource usage of its evaluation plan. Since disk access dominates, cost is measured as **number of block transfers + number of disk seeks**, usually reported as $b \times t_T + S \times t_S$ where t_T is the transfer time per block and t_S the average seek time. CPU time, memory and, for distributed systems, communication cost also contribute but are normally ignored in textbook formulas. Cost is *estimated*, not measured, from data-dictionary statistics: n_r (tuples), b_r (blocks), f_r (blocking factor), $V(A, r)$ (distinct values) and index height.

Selection on an unordered attribute = linear search (file scan). No index and no ordering exists, so every block of the relation must be read and each tuple tested. - Worst case and non-key attribute: **cost = b_r block transfers + 1 seek** (the file is contiguous, so one seek then a sequential sweep). - If the selection is on a *candidate* key, the scan can stop at the match, giving an average of **$b_r/2$ transfers + 1 seek**. - An unsuccessful search always costs the full b_r .

Example: `SELECT * FROM student WHERE city = 'Delhi';` with 10,000 tuples, 50 per block $\rightarrow b_r = 200$, so cost ≈ 200 block transfers + 1 seek. With a secondary B+-tree index on `city` of height 3 and 20 matching records the cost falls to about $3 + 20 = 23$ transfers with 23 seeks — which is why the optimiser prefers the index only when the selectivity is high enough.

Q4. Which of the relational operators has maximum query execution cost ? Explain with the help of an example. [4 marks · asked 1x: D24 Q4(c)]

The **Cartesian product** ($\times$) is the costliest relational operator, followed by the join. Reason: for relations r and s it produces $n_r \times n_s$ tuples each of degree (degree r + degree s), so both the output size and the work are multiplicative, whereas σ and π only shrink their input and the set operations are additive.

Example. STUDENT with $n_r = 10,000$ tuples in $b_r = 200$ blocks and PROGRAMME with $n_s = 20$ tuples in $b_s = 1$ block.

```
STUDENT  $\times$  PROGRAMME  $\rightarrow 10,000 \times 20 = 200,000$  tuples
```

Every output tuple must be built and written, and if the result does not fit in memory it is materialised to disk and re-read by the next operator; the block-nested-loop cost is $b_r + b_r \times b_s$ transfers for the input alone. The natural join STUDENT $\bowtie$ PROGRAMME over the same relations yields only 10,000 tuples, and $\sigma(\text{title}='MCA')(\text{PROGRAMME})$ yields 1. Hence an optimiser never evaluates a Cartesian product followed by a selection: it rewrites $\sigma\theta(r \times s)$ as $r \bowtie\theta s$ so the condition is applied while tuples are generated, and chooses hash or indexed nested-loop join to avoid the full product.

Q5. What is Hash-Join ? Explain the cost calculation for simple Hash-Join.

[7 marks · asked 1x: J24 Q4(c)]

Hash join computes an **equi-join** $r \bowtie(r.A = s.B) s$ by partitioning both relations with the *same* hash function on the join attribute, so that a tuple of r can only match tuples of s in the partition of the same number; the join is then performed partition by partition in memory.

Algorithm (partitioned/grace hash join). 1. **Partition phase.** Choose h with range $0 \dots n-1$. Read s and write each tuple to partition s_i where $i = h(t[B])$; read r and write each tuple to r_i where $i = h(t[A])$. 2. **Build phase.** For each i , read the smaller partition (say s_i) into memory and build an in-memory hash index on B with a different function h_2 . 3. **Probe phase.** Read r_i block by block, probe the in-memory index with $h_2(t[A])$ and output every matching concatenated pair.

```
r ----h---->  r0 r1 r2 ... rn-1
                | | |
s ----h---->  s0 s1 s2 ... sn-1
join only r_i with s_i   (build on s_i, probe with r_i)
```

Cost. Assume no partition overflows (each s_i fits in memory) and no recursive partitioning; let b_r and b_s be the number of blocks. - Partitioning reads both relations once and writes both out: $2(b_r + b_s)$ block transfers. - The build-and-probe pass reads both sets of partitions once: $(b_r + b_s)$.

```
Total cost =  $3(b_r + b_s)$  block transfers
              +  $2([b_r/b_b] + [b_s/b_b])$  seeks      ( $b_b$  = blocks per output buffer)
```

Example: $b_r = 2000$, $b_s = 500 \rightarrow 3 \times 2500 = \mathbf{7500 \text{ block transfers}}$, against $2000 + 2000 \times 500 \approx 1,002,000$ for a block nested-loop join. Memory requirement: $M > \sqrt{b_s}$ blocks for a single partitioning pass; otherwise recursive partitioning adds $2(b_r + b_s)$ per extra pass. Hash join works only for equi-joins and natural joins, not for general theta joins, and skewed data causes partition overflow.

Database security and integrity

Q1. Database Security (short note) / Relationship between Database security and Integrity / Basic model of database access control. [3-5 marks · asked 3x: J22 Q5(e), D24 Q5(e), J26 Q5(c)]

Database security is the protection of the database against accidental or intentional loss, destruction, unauthorised disclosure and unauthorised modification. Threats are loss of availability, loss of confidentiality (unauthorised reading) and loss of integrity (unauthorised writing).

Mechanisms, in layers: **authentication** (user id/password, biometrics) establishes who the user is; **authorisation/access control** grants discretionary privileges on objects —

```
GRANT SELECT, INSERT ON student TO clerk;  
GRANT UPDATE (fee) ON student TO accountant WITH GRANT OPTION;  
REVOKE INSERT ON student FROM clerk;
```

plus role-based grants and mandatory access control with security classes for multilevel security; **views** mask rows and columns a user may not see; **encryption** protects stored and transmitted data; **audit trails** log every access for accountability; and physical/OS security plus backup and recovery protect availability.

Basic access-control model. The DBMS maintains an *access matrix* of (subject, object, privilege) triples in the data dictionary. A subject (user/role) issues an operation on an object (database, relation, attribute, view, tuple); a **reference monitor** inside the database manager intercepts the request, looks up the matrix and permits or rejects it, writing an audit record either way.

```
User --request--> [ Authentication ] --> [ Authorisation / reference monitor ]  
                                     |           ^  
                                     | permit / deny | access matrix,  
                                     v           | privileges  
                                     [ Database ] -----> Data dictionary  
                                     |  
                                     Audit trail
```

Relationship with integrity. Security and integrity are complementary halves of data protection and both are enforced by the database manager: security stops *unauthorised users* from reaching data; integrity stops *authorised users* from putting invalid data in. A security failure is a clerk reading salaries he may not see; an integrity failure is the same clerk legitimately entering marks = 150.

Q2. Differentiate between data security and data integrity. [3 marks · asked 1x: D23 Q1(e)]

Data security	Data integrity
Protects data from unauthorised access, disclosure, modification and destruction	Keeps data accurate, valid and consistent even when accessed legitimately
Concerned with <i>who</i> may do <i>what</i>	Concerned with <i>what values and states are legal</i>
Enforced by authentication, GRANT / REVOKE, roles, views, encryption, audit trails	Enforced by domain/CHECK constraints, PRIMARY KEY, UNIQUE, NOT NULL, FOREIGN KEY, assertions, triggers, and transaction ACID properties
Administered by the DBA/security administrator as a policy	Specified by the designer as part of the schema
Failure example: an outsider reads employee salaries	Failure example: an authorised clerk enters marks = 150, or an account row referring to a non-existent customer

In short, security keeps the wrong people out; integrity keeps the wrong data out. Both are needed: encrypted but inconsistent data is useless, and valid but publicly readable data is unsafe.

Q3. Define and explain the concept of integrity constraints with the help of example(s).

[3-6 marks · asked 4x: J22 Q1(e) 3 ("Explain the concept of referential integrity with the help of an example"), J24 Q3(a) 6 ("entity inte

Integrity constraints are rules declared in the schema that every legal database state must satisfy; the DBMS rejects any insert, delete or update that would violate them, so correctness is enforced once, centrally, rather than in each application.

Schema used:

```
CREATE TABLE customer (
  cust_id  CHAR(5)    PRIMARY KEY,           -- entity integrity
  name     VARCHAR(40) NOT NULL,
  city     VARCHAR(20));

CREATE TABLE account (
  acc_no   CHAR(8)    PRIMARY KEY,
  cust_id  CHAR(5)    REFERENCES customer(cust_id) -- referential integrity
                        ON DELETE CASCADE,
  type     CHAR(7)    CHECK (type IN ('SAVING','CURRENT')), -- domain
  balance  NUMBER(10,2) CHECK (balance >= 0));
```

(i) Domain constraint — every value of an attribute must come from the attribute's domain/data type. type = 'LOAN' or balance = -500 is rejected.

(ii) Entity integrity constraint — no component of a primary key may be NULL, and the key must be unique, since a tuple cannot be identified otherwise. Inserting ('C01','Amit','Agra') twice, or with cust_id NULL, is rejected.

(iii) Referential integrity constraint — a foreign key value must either be NULL or match an existing primary key value in the referenced relation. INSERT INTO account VALUES('A1001','C99','SAVING',5000) fails because customer C99 does not exist. On deletion of a referenced customer the DBMS applies the declared action: CASCADE (delete the accounts), SET NULL, SET DEFAULT, or RESTRICT/NO ACTION (refuse).

(iv) Key constraint — declared candidate keys must stay unique (UNIQUE(pam_no)).

(v) General semantic constraints — arbitrary rules written as assertions or triggers, e.g. "a member may not hold more than 4 issued books".

Q4. Authorisation (as part of a five-term question). [3 marks · asked 1×: J23 Q4(a)(v)]

Authorisation is the process of granting a user or role a specific set of privileges on specific database objects, and of checking those privileges on every request. After authentication identifies the user, the database manager consults the privileges stored in the data dictionary and permits the operation only if it has been granted. The common privileges on data are **READ/SELECT** (retrieve only), **INSERT** (add new tuples), **UPDATE** (modify existing values, possibly limited to named columns) and **DELETE** (remove tuples); schema-level privileges include INDEX, ALTER, REFERENCES and the right to create relations.

```
GRANT SELECT, INSERT ON student TO clerk;
GRANT UPDATE (fee) ON student TO accountant WITH GRANT OPTION;
REVOKE INSERT ON student FROM clerk CASCADE;
```

WITH GRANT OPTION lets the grantee pass the privilege on; REVOKE ... CASCADE withdraws such derived grants too. Privileges may be given to **roles** (GRANT clerk_role TO ravi;) to simplify administration, and views are used to authorise access to a subset of rows and columns.

Asked once

Q1. What are views in DBMS ? How do views differ from tables in DBMS ? Can we perform insert, delete and modify operations, if the view contains a group by clause ? Justify. Also, give advantages of views. [8 marks · J26 Q3(a)]

A **view** is a virtual relation derived from one or more base tables by a stored query; its definition is kept in the data dictionary and the rows are computed when the view is referenced (unless it is a materialised view).

```
CREATE VIEW mca_student AS
  SELECT enrol_no, name FROM student WHERE prog_code = 'MCA';

CREATE VIEW prog_count AS
  SELECT prog_code, COUNT(*) AS total FROM student GROUP BY prog_code;
```

Table (base relation)	View
Physically stored on disk	Not stored; only its definition is stored
Occupies space, has its own indexes	No space for data; cannot normally be indexed
Always fully updatable	Updatable only under restrictions
Independent existence	Dropped/invalidated if its base table is dropped

Updates through a view with GROUP BY: no. DML is permitted only on a *simple/updatable* view — one derived from a single base table, containing its primary key, with no DISTINCT, no aggregate function, no GROUP BY/HAVING, no set operation and no computed-only columns. A view containing GROUP BY fails this because each of its rows summarises **many** base tuples, so there is no one-to-one mapping back: to change total from 40 to 41 the DBMS would have to invent a student row with unknown values for enrol_no and name, and a delete of one group row has no well-defined set of base rows to delete. The mapping is irreversible, hence INSERT, UPDATE and

DELETE are rejected. The same reasoning applies to DISTINCT and aggregate views; such views can only be read, or modified indirectly through INSTEAD OF triggers.

Advantages of views: security (expose only permitted rows/columns, hide salary), simplicity (a complex multi-table join is referenced as one name), logical data independence (base tables can be restructured while the view presents the old interface), reusability and consistency of frequently used queries, and customised presentation for different user groups.

Q2. Differentiate between Stored Procedures and Triggers. [4 marks · J26 Q3(b)(i)]

Stored procedure	Trigger
A named block of SQL/PL-SQL stored in the database and executed explicitly by a user or program	A named block stored in the database and executed implicitly by the DBMS when a specified event occurs
Invoked by CALL / EXECUTE	Fired automatically by INSERT / UPDATE / DELETE on a table; cannot be called
Accepts IN/OUT parameters and can return values	No parameters, no return value; uses :NEW / :OLD row values
Can be scheduled, nested and called from applications	Bound to one table and one event; BEFORE / AFTER, row- or statement-level
Purpose: encapsulate reusable business logic, reduce network traffic	Purpose: enforce complex integrity rules, auditing, derived-column maintenance

```
CREATE PROCEDURE raise_fee(p_code IN CHAR, amt IN NUMBER) AS
BEGIN UPDATE programme SET fee = fee + amt WHERE prog_code = p_code; END;

CREATE TRIGGER audit_fee AFTER UPDATE OF fee ON programme
FOR EACH ROW
BEGIN INSERT INTO fee_log VALUES(:OLD.prog_code, :OLD.fee, :NEW.fee, SYSDATE); END;
```

Q3. Explain the rules of data normalization. [5 marks · J26 Q2(c)]

Normalisation is the step-by-step decomposition of a relation, guided by its functional dependencies, into smaller relations that are free of redundancy and of insertion, deletion and update anomalies, while remaining lossless and (preferably) dependency preserving. The rules are the successive normal forms:

- **1NF** — every attribute is atomic: no repeating groups, no multi-valued or composite attributes. STUDENT(enrol_no, name, {course}) becomes STUDENT(enrol_no, name) and ENROL(enrol_no, course).
- **2NF** — in 1NF and **no non-prime attribute is partially dependent** on a candidate key. In ENROL(enrol_no, course_code, course_name, grade) with course_code → course_name, the partial dependency is removed into COURSE(course_code, course_name).
- **3NF** — in 2NF and **no transitive dependency** of a non-prime attribute on the key: for every FD X → A either X is a super key or A is prime. STUDENT(enrol_no, prog_code, prog_fee) with prog_code → prog_fee splits into STUDENT(enrol_no, prog_code) and PROGRAMME(prog_code, prog_fee).
- **BCNF** — every determinant is a candidate key.
- **4NF** — no non-trivial multivalued dependency X ↔ Y unless X is a super key.

- **5NF** — no relation can be reconstructed by joining smaller projections (no join dependency other than through candidate keys).

Accompanying rules: the decomposition must be a **lossless join** (the common attribute of the parts is a key of one of them) and should be **dependency preserving** (every original FD still enforceable in one relation).

Q4. Briefly discuss the optimistic concurrency control technique. [4 marks · J26 Q4(c)]

Optimistic (validation-based) concurrency control assumes conflicts are rare, so it uses **no locks** during execution and checks for conflicts only at the end. Each transaction runs in three phases:

1. **Read phase** — the transaction reads committed data but writes all updates to a **private local workspace**; the database is untouched.
2. **Validation phase** — at the end, the system compares the transaction's read set and write set with those of concurrently executing transactions, using timestamps ($Start(T_i)$, $Validation(T_i)$, $Finish(T_i)$). T_i passes if, for every T_j that validated earlier, either T_j finished before T_i started, or T_j 's write set does not intersect T_i 's read set and T_j finished writing before T_i 's validation.
3. **Write phase** — if validation succeeds the local updates are copied into the database and the transaction commits; if it fails the transaction is **aborted and restarted**, and because nothing was written, the rollback is free.

Advantages: no locking overhead, no deadlock, no waiting — ideal for read-mostly workloads and low-conflict systems. Disadvantages: in a high-conflict workload transactions are repeatedly rolled back late, wasting all work done, and starvation of long transactions is possible.

Q5. Explain different kinds of failures that a transaction program encounters during execution.

[4 marks · D23 Q1(d)]

- **Transaction failure (logical/local error)** — the transaction cannot complete for reasons internal to it: bad input, data not found, overflow, division by zero, or an explicit `ROLLBACK`. Only that transaction is undone, using its log records; the database and other transactions are unaffected.
- **Transaction failure due to system enforcement** — the concurrency-control manager aborts the transaction to break a deadlock or because it failed validation; it is restarted later.
- **System crash (soft failure)** — a hardware fault, operating-system bug, power failure or DBMS crash that stops processing and loses the **volatile** main-memory contents (buffers), while the disk survives. Recovery reads the log from the last checkpoint, redoes committed transactions and undoes uncommitted ones (the *fail-stop* assumption: non-volatile storage is not corrupted).
- **Disk/media failure (hard failure)** — a head crash or unreadable sector destroys part of non-volatile storage. Recovery restores the most recent **backup dump** and redoes the committed transactions from the archived log.
- **Communication failure** — in distributed or client/server databases, a lost message or network partition leaves a transaction in doubt; handled by two-phase commit with timeouts.

Q6. Explain the one-to-one and many-to-many relationship in an ER-diagram. [4 marks · D21 Q2(d)]

Cardinality states how many instances of one entity may be associated with one instance of the other.

One-to-one (1:1) — one instance of A relates to at most one instance of B and vice versa.

```
[EMPLOYEE] ---1--- < manages > ---1--- [DEPARTMENT]
```

Each department has one manager and an employee manages at most one department. In relations, the primary key of either side is placed as a foreign key in the other, declared UNIQUE: `DEPARTMENT(dno, dname, mgr_eno UNIQUE)`.

Many-to-many (M:N) — one instance of A relates to many instances of B and one instance of B relates to many instances of A.

```
[STUDENT] ---M--- < enrolls > ---N--- [COURSE]
                |
                grade          (relationship attribute)
```

A student takes many courses and a course has many students. An M:N relationship cannot be represented with a foreign key in either entity; it must become a **separate relation** whose primary key is the combination of both foreign keys, carrying any relationship attributes: `ENROLS(enrol_no, course_code, grade)` with `enrol_no → STUDENT` and `course_code → COURSE`. (The third form, 1:N, places the key of the "1" side as a foreign key in the "N" side.)

Q7. Nested queries in SQL (short note). [5 marks · J25 Q5(f)]

A nested query (subquery) is a complete `SELECT` statement written inside the `WHERE`, `HAVING`, `FROM` or `SELECT` clause of another statement; the inner query is evaluated and its result used by the outer one.

Non-correlated — the inner query is evaluated once:

```
SELECT acc_no FROM account
WHERE balance = (SELECT MAX(balance) FROM account);           -- scalar subquery

SELECT name FROM student
WHERE prog_code IN (SELECT prog_code FROM programme WHERE fee > 20000);
```

Correlated — the inner query references a column of the outer query and is re-evaluated for each outer row, usually with `EXISTS`:

```
SELECT c.name FROM customer c
WHERE EXISTS (SELECT 1 FROM account a WHERE a.cust_id = c.cust_id);
```

Operators used with subqueries: `IN/NOT IN`, `= ANY/SOME`, `> ALL`, `EXISTS/NOT EXISTS`, and scalar comparison for single-value results. Subqueries may also appear in `FROM` as derived tables and in `INSERT ... SELECT`, `UPDATE ... SET x = (SELECT ...)`. Advantage: a single declarative statement replaces procedural code, and `NOT EXISTS` expresses division/"for all" queries. Limitation: a correlated subquery can be slow, and the optimiser often rewrites it as a join.

Q8. Knowledge databases (short note). [5 marks · J25 Q5(b)]

A knowledge database (knowledge base, deductive or expert database) stores not only facts but also **rules** from which further facts are inferred, so queries are answered by deduction rather than retrieval alone. It combines database technology (persistence, large volumes, transactions) with logic programming. Facts correspond to tuples and rules are written in Datalog-like form:

```
parent(amit, bina).                % fact
ancestor(X, Y) :- parent(X, Y).    % rule
ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y). % recursive rule
```

A query `ancestor(amit, Q)?` returns descendants that are nowhere stored explicitly. Components: fact base, rule base, and an **inference engine** using forward chaining (data-driven) or backward chaining (goal-driven), plus an explanation facility. Representations include rules, frames, semantic networks and ontologies (RDF/OWL). Uses: expert systems for medical diagnosis and credit approval, recommendation and the semantic web. Advantages: recursion and "intelligent" querying; issues: rule consistency, termination of recursion and efficiency on large fact bases.

Q9. Data fragmentation and replication in the context of distributed databases (short note).

[5 marks · D25 Q5(d)]

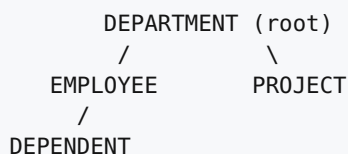
In a distributed database a global relation is divided and placed across sites to improve locality, availability and parallelism.

Fragmentation splits a relation into fragments that can be reconstructed by relational algebra: - **Horizontal** — a subset of tuples by selection; `ACCOUNT_DELHI =  $\sigma(\text{branch}='Delhi')(\text{ACCOUNT})$` , reconstructed by union. Correctness needs *completeness* and *reconstructability*. - **Vertical** — a subset of attributes by projection, each fragment keeping the primary key; `EMP1 =  $\pi(\text{eno}, \text{name}, \text{dept})(\text{EMP})$` , `EMP2 =  $\pi(\text{eno}, \text{salary})(\text{EMP})$` , reconstructed by natural join on `eno`. - **Mixed/hybrid** — horizontal fragments further split vertically.

Replication stores copies of the same fragment or relation at several sites: *full replication* (every site holds the whole database), *partial replication*, or *no replication* (pure fragmentation). Advantages: higher availability (a site failure does not block reads), faster local reads and parallel query processing. Disadvantages: every update must be propagated to all copies, so write cost and the need for concurrency control across sites (distributed 2PL, two-phase commit, primary-copy or quorum protocols) rise, and copies may be temporarily inconsistent under lazy replication.

Q10. Hierarchical model. [2 marks · J23 Q2(a)(v)]

The hierarchical model, used by IBM IMS, organises records into a **tree**: one root record type with child record types linked by parent-child relationships, where each child occurrence has exactly one parent but a parent may have many children.



Access is navigational: an application traverses the tree by pointers from the root downwards (preorder). It represents 1:N relationships naturally and gives fast retrieval along predefined paths, but an M:N relationship requires duplicating records or using virtual/pointer records, deleting a parent deletes its whole subtree, and there is no declarative query language, so the structure is rigid and offers little data independence — the reasons it was superseded by the relational model.

Short notes

N1. Data warehouse / its characteristics / multi-dimensional data / star and snowflake schema / dimensional vs fact table / operational vs data-warehouse data / data-warehousing life cycle

A data warehouse is a **subject-oriented, integrated, time-variant and non-volatile** collection of data supporting management decisions. Operational data is current, detailed, update-intensive and application-oriented; warehouse data is historical, summarised, read-only and subject-oriented. Architecture: sources → **ETL** (extract, transform, clean, load) → staging → warehouse → data marts → OLAP and reporting. Data is modelled **multi-dimensionally** as a cube: measures (sales amount) analysed by dimensions (product, time, branch), supporting roll-up, drill-down, slice, dice and pivot. A **star schema** has one central **fact table** (foreign keys to dimensions + numeric measures, very many rows) surrounded by denormalised **dimension tables** (descriptive attributes, few rows); the **snowflake schema** normalises dimensions into sub-tables, saving space but adding joins. Life cycle: requirements → dimensional modelling → ETL design → physical design → deployment → maintenance and growth.

N2. Classification (and clustering) in the context of data mining

Classification is **supervised** learning: from a training set whose tuples carry a known class label, a model is built that predicts the label of unseen tuples. Steps: training on labelled data, testing on a held-out set, then deployment; accuracy is measured on the test set. Techniques: decision trees (ID3/C4.5, using information gain), naive Bayes, k-nearest neighbour, neural networks, SVM. Example: classifying a loan applicant as **risk = safe/risky** from age, income and credit history. **Clustering** is **unsupervised**: there are no labels, and tuples are grouped so that intra-cluster similarity is high and inter-cluster similarity low, by a distance measure. Techniques: k-means, k-medoids, hierarchical (agglomerative/divisive), DBSCAN. Example: segmenting customers into spending groups for targeted marketing. Classification predicts a predefined class; clustering discovers the groups themselves.

N3. Complex data types / object-relational database system

Relational 1NF demands atomic values, which cannot represent addresses, images, documents or nested lists naturally. An **object-relational DBMS** extends SQL with object features while keeping tables and SQL. **Complex data types** are non-atomic types: structured/composite types, collections (arrays, multisets, nested tables), large objects (BLOB/CLOB), and user-defined types with methods.

```
CREATE TYPE address_t AS OBJECT (house VARCHAR(10), city VARCHAR(20), pin CHAR(6));
CREATE TYPE phone_list AS VARRAY(3) OF VARCHAR(12);
CREATE TABLE person (pid CHAR(5) PRIMARY KEY, name VARCHAR(40),
                     addr address_t, phones phone_list, photo BLOB);
```

Other ORDBMS features: **type inheritance** (`CREATE TYPE student_t UNDER person_t`), **table inheritance**, **object identity** (system-generated OID independent of key values), and **reference types** (`REF person_t`) that replace foreign-key joins by pointer navigation. Benefit: multimedia, CAD, GIS and nested data stored without unnatural decomposition, with SQL still available.

N4. Weak entity (and strong entity)

A **strong (regular) entity** has its own key attribute, so each instance is identified independently — e.g. `CUSTOMER(cust_id, name)`. A **weak entity** has no key attribute of its own; it exists only in relation to an **owner/identifying** strong entity and is identified by the owner's key plus its own **partial (discriminator) key**. Example: `DEPENDENT(name, dob)` of an `EMPLOYEE` — two employees may both have a dependent named "Ravi", so the full key is `(emp_no, name)`. ER notation: double rectangle for the weak entity, double diamond for the identifying relationship, dashed underline for the partial key; participation in that relationship is **total** and the cardinality on the owner side is 1.

```
[EMPLOYEE] ==<< has >>== [[DEPENDENT]]          key: (emp_no, name)
```

On mapping to relations, the weak entity becomes a relation containing the owner's primary key as part of its own key and as a foreign key with `ON DELETE CASCADE`.

N5. NoSQL databases (general / graph-based / vs RDBMS)

NoSQL ("not only SQL") databases are non-relational, schema-flexible stores built for very large volumes of semi-structured data and horizontal scaling across commodity clusters. They relax ACID for **BASE** (Basically Available, Soft state, Eventually consistent) and, by the CAP theorem, usually trade consistency for availability and partition tolerance. Four families: **key-value** (Redis, DynamoDB), **document** (MongoDB, CouchDB — JSON/JSON documents, nested, no fixed schema), **column-family** (Cassandra, HBase), and **graph** (Neo4j, Amazon Neptune). A **graph database** stores nodes, labelled directed relationships and properties on both, and traverses them in constant time per hop, so it is ideal for social networks, fraud rings and recommendations; queries use Cypher: `MATCH (a:Person)-[:FRIEND*2]->(b) RETURN b`. Versus an RDBMS: no fixed schema, no joins or declarative SQL standard, scale-out instead of scale-up, eventual instead of immediate consistency, weaker integrity enforcement.

N6. Database security (basic model of database access control; security vs integrity)

Database security protects the database against unauthorised disclosure, modification and destruction, preserving confidentiality, integrity and availability. Layers: **authentication** of the user, **authorisation** by discretionary privileges (`GRANT SELECT, UPDATE(fee) ON student TO clerk WITH GRANT OPTION; / REVOKE ... CASCADE`), roles, mandatory access control with security classes for multilevel security, **views** to mask rows and columns, **encryption** of stored and transmitted data, and **audit trails** for accountability. The basic access-control model keeps an access matrix of (subject, object, privilege) in the data dictionary; a reference monitor inside the database manager intercepts every request, consults the matrix and permits or denies it, logging the attempt. Security differs from integrity: security keeps unauthorised **users** out, integrity keeps invalid **data** out, the latter enforced by domain, entity and referential constraints.

N7. Data mining (general)

Data mining is the extraction of previously unknown, valid and actionable patterns from large volumes of data, the analysis step of the wider **KDD** process (selection → preprocessing/cleaning → transformation → data mining → interpretation and evaluation). Mining tasks: **descriptive** — association-rule mining (`{bread, butter} → {milk}` with support and confidence), clustering, outlier detection, summarisation; and **predictive** — classification, regression, time-series and sequence prediction. Techniques come from statistics, machine learning and database technology, and run

over a data warehouse or data mart. Business value: market-basket analysis for shelf layout and cross-selling, customer churn prediction, credit-risk scoring, fraud detection, medical diagnosis. Data mining differs from a query: SQL retrieves what you ask for, mining discovers rules you did not know to ask for. Issues: data quality, scalability, overfitting, and privacy.

N8. Use/role of checkpoint in recovery

A **checkpoint** is a point at which the DBMS forces consistency between the buffers and the disk, so that recovery need not scan the whole log. At a checkpoint the system suspends new updates, writes all log records in the buffer to stable storage, writes all modified data blocks to disk, and writes a record `<checkpoint L>` to the log, where L is the list of transactions active at that moment. After a crash the recovery manager scans the log **backwards only to the last checkpoint**: transactions that committed before it need no redo because their effects are already on disk; transactions in L or started later are redone if a `<Ti commit>` record exists and undone otherwise. Role: it bounds recovery time, limits log scanning and permits log truncation/archiving. Variants: fuzzy checkpointing, which avoids a long pause by writing dirty blocks after the checkpoint record.

N9. Deadlock (wound-wait scheme)

A **deadlock** occurs when two or more transactions each hold a lock the other needs, so all wait forever: T1 holds an exclusive lock on A and requests B while T2 holds B and requests A. It is detected by a **wait-for graph** — a cycle means deadlock — after which a victim is chosen and rolled back; alternatives are timeouts and prevention by ordering resources or by pre-declaring all locks. **Timestamp-based prevention** uses the age of transactions: in **wound-wait**, if Ti requests a lock held by Tj and $TS(T_i) < TS(T_j)$ (Ti is older), the older transaction **wounds** the younger — Tj is rolled back and Ti takes the lock; if Ti is younger it simply waits. The complementary **wait-die** makes the older wait and the younger die. Both are deadlock-free because waiting always goes one way in timestamp order, and neither starves transactions, since a rolled-back transaction keeps its original timestamp.

N10. OODBMS vs RDBMS

RDBMS	OODBMS
Data in flat tables of atomic (1NF) values	Data as objects with complex, nested structure
Data and behaviour separate; logic in applications	State and methods encapsulated in the object
Identity by primary-key values	Identity by immutable system-generated OID
Relationships by foreign keys, resolved with joins	Relationships by object references, navigated by pointer
No inheritance; generalisation simulated by extra tables	Class hierarchy with inheritance and polymorphism
Declarative SQL, strong optimiser, mature standards	OQL/ODL, language-integrated persistence, weaker standardisation
Impedance mismatch with OO programming languages	Seamless with C++/Java object models
Excellent for business transaction processing	Better for CAD, GIS, multimedia, engineering design data

An ORDBMS (Oracle, PostgreSQL) adds complex types, inheritance and references to the relational core and is the usual compromise.

N11. Multivalued dependency

A multivalued dependency $X \twoheadrightarrow Y$ holds in $R(X, Y, Z)$ if, for each value of X , the set of Y values associated with it is independent of the Z values — so Y and Z are two independent multi-valued facts about X and the relation stores their Cartesian product. Example: `FACULTY(fname, course, hobby)` where a teacher teaches several courses and has several hobbies, unrelated to each other: `fname  $\twoheadrightarrow$  course` and `fname  $\twoheadrightarrow$  hobby`. With 2 courses and 2 hobbies the relation must store 4 tuples, and adding one hobby forces inserting one tuple per course — redundancy that no FD explains. **Difference from FD:** an FD $X \rightarrow Y$ determines a *single* Y value; an MVD determines a *set* of values; every FD is a trivial MVD. **4NF** removes it: decompose into `TEACHES(fname, course)` and `HOBBIES(fname, hobby)`, a lossless decomposition whose join rebuilds the original.

N12. Data dictionary

The data dictionary (system catalogue) is the repository of **metadata** — data about the database itself. It stores relation and view names, attribute names with data types and lengths, primary-key, foreign-key, unique and check constraints, index definitions and storage structures, user accounts, roles and granted privileges, and statistics such as number of tuples, blocks and distinct values per attribute. It is maintained as ordinary relations and queried with SQL: `SELECT column_name, data_type FROM USER_TAB_COLUMNS WHERE table_name = 'STUDENT';`. Users of the dictionary: the DDL compiler writes into it, the database manager reads it for authorisation and integrity checking, the query optimiser reads its statistics for cost estimation, and the DBA uses it for documentation and impact analysis. An **active** dictionary is updated automatically by the DBMS and so can never drift from reality; a **passive** one is maintained separately.

N13. Query processing

Query processing is the sequence of activities that turns a high-level query into an answer: **parsing and translation** (syntax check, validation of names and types against the data dictionary, construction of a parse tree), conversion into an equivalent **relational-algebra expression**, **optimisation** (generation of equivalent expressions by algebraic rules — push selections and projections down, convert $\sigma(r \times s)$ into a join — and selection of the cheapest annotated evaluation plan using dictionary statistics and the available indexes), **code generation**, and **evaluation** by the execution engine, which runs the chosen algorithm for each operator and either materialises intermediate relations or pipelines tuples to the next operator. Cost is estimated in **block transfers plus seeks**; for example a selection on an unsorted non-key attribute costs b_r transfers + 1 seek, while a simple hash join costs $3(b_r + b_s)$.

N14. Second normal form

A relation is in **2NF** if it is in 1NF and no non-prime attribute is **partially dependent** on a candidate key — that is, every non-key attribute depends on the *whole* key, not on part of a composite key. Example: `ENROL(enrol_no, course_code, course_name, grade)` with key `{enrol_no, course_code}` and FDs `{enrol_no, course_code}  $\rightarrow$  grade`, `course_code  $\rightarrow$  course_name`. The second FD is a partial dependency: `course_name` is repeated for every student taking that course (update anomaly), a new course cannot be recorded until a student enrolls (insertion anomaly), and deleting the last enrolment loses the course name (deletion anomaly). 2NF decomposition:

```

ENROL(enrol_no, course_code, grade)      -- key {enrol_no, course_code}
COURSE(course_code, course_name)         -- key course_code

```

The decomposition is lossless (`course_code` is a key of `COURSE`) and dependency preserving. A relation whose primary key is a single attribute is automatically in 2NF.

N15. Serialisable schedule

A **schedule** is an ordering of the operations of a set of concurrent transactions. A schedule is **serialisable** if it produces exactly the same final database state as *some* serial schedule of the same transactions; serialisability is the accepted criterion of correctness for concurrent execution, because serial execution is correct by definition. A **serial** schedule runs transactions one after another with no interleaving (correct but slow); a serialisable schedule interleaves operations yet remains equivalent to one of them. **Conflict serialisability** is tested with a **precedence (serialisation) graph**: a node per transaction and an edge $T_i \rightarrow T_j$ whenever T_i performs a conflicting operation (read-write, write-read or write-write on the same item) before T_j ; the schedule is conflict-serialisable **iff the graph is acyclic**, and a topological sort gives the equivalent serial order. **View serialisability** is weaker and more general. Two-phase locking guarantees serialisability in practice.

N16. Multi-list file organization

A multi-list file supports retrieval on several secondary keys by threading linked lists through the data records themselves. For each secondary key there is an index whose entries hold a key value, the head pointer of its list and the list length; every record carries one link field per indexed key, pointing to the next record with the same value for that key. On `EMPLOYEE(eno, name, dept, grade)` the `dept` index gives `Sales → E1 → E4 → E7 → NULL` while an independent `grade` chain runs through the same records.

```

dept index: Sales -> [E1] -> [E4] -> [E7] -> NULL
grade index: A    -> [E2] -> [E4] -> NULL

```

Advantages: multi-key queries without sorting or a separate pointer file; insertion only relinks pointers. Disadvantages: records grow by one pointer per key, deletion must fix every chain, and traversal costs one block transfer per record. The **coral ring** variant uses circular doubly linked chains to ease deletion.

N17. Primary and Secondary Index

A **primary index** is built on the ordering key of a file physically sorted on that key. It is **sparse** — one entry (anchor key, block address) per block — so it is small, can be multi-level (a B+-tree), and only **one** can exist per file. A **secondary index** is built on a non-ordering attribute; it must be **dense**, with one entry per distinct value, and for a non-unique attribute each entry points to a bucket of record pointers, adding a level of indirection. **Many** secondary indexes may exist on one file.

Primary	Secondary
Sparse, small, one per file	Dense, large, many per file
Range queries cheap: data contiguous	Range queries fetch scattered blocks
~4 block accesses via a 3-level index	Index height + one transfer per matching record

The primary index is more advantageous; the secondary index is still essential to avoid full scans on non-key attributes, at the cost of maintenance on every update.

N18. Join Operation

A join combines tuples of two relations that satisfy a condition on common attributes; it is a Cartesian product followed by a selection, $R \bowtie S = \sigma_{\theta}(R \times S)$. Kinds: **theta join** (any comparison), **equi-join** (equality only), **natural join** (equality on all same-named attributes with the duplicate column removed), **self join** (a relation with itself via aliases, for employee-manager data), **outer joins** (left $\bowtie$, right $\bowtie$, full $\bowtie$, which retain non-matching tuples padded with NULLs), and **semi/anti joins**.

```
STUDENT(enrol_no, name, prog)  $\bowtie$  PROGRAMME(prog, title)
= (S01, Amit, MCA, Master of Comp. App.)
```

Implementation algorithms and costs: nested-loop $b_r + b_r \cdot b_s$, indexed nested-loop $b_r + n_r \cdot c$, sort-merge $b_r + b_s$ after sorting, hash join $3(b_r + b_s)$. The join is the most expensive frequently used operator, so join order and algorithm choice are the optimiser's main decisions.

N19. Locking in transaction

A **lock** is a variable associated with a data item that controls concurrent access to it, so that interleaved transactions produce serialisable schedules and the lost-update, dirty-read and unrepeatable-read anomalies are prevented. Two modes: **shared (S)**, requested by $\text{lock-S}(X)$ for reading, which several transactions may hold simultaneously; and **exclusive (X)**, requested by $\text{lock-X}(X)$ for writing, which excludes all other locks.

Requested \ Held	S	X
S	yes	no
X	no	no

Example: T1 holds lock-X(A) while transferring money, so T2's lock-S(A) waits until T1 commits and releases it — T2 cannot read the uncommitted value. Locking alone is insufficient; **two-phase locking (2PL)** adds the rule that all locks are acquired in a *growing* phase and released only in a *shrinking* phase, which guarantees serialisability. Strict 2PL holds exclusive locks until commit, avoiding cascading rollback. Costs: waiting, deadlock and starvation.

N20. Functional Dependencies

A functional dependency $X \rightarrow Y$ on relation R means that for any two tuples with the same value of X, the value of Y is also the same: X *determines* Y. It is a semantic constraint derived from the application, not from sample data. Example: in `STUDENT(enrol_no, name, prog_code, prog_fee)`, `enrol_no  $\rightarrow$  name`, `prog_code  $\rightarrow$  prog_fee`, and `enrol_no  $\rightarrow$  prog_code`, so by transitivity `enrol_no  $\rightarrow$  prog_fee`. Kinds: **trivial** ($Y \subseteq X$), **full** (no proper subset of X determines Y), **partial** (part of a composite key determines a non-prime attribute — breaks 2NF) and **transitive** ($X \rightarrow Y \rightarrow Z$ with Y not a key — breaks 3NF). Armstrong's axioms — reflexivity, augmentation and transitivity, with the derived union, decomposition and pseudo-transitivity rules — compute the closure F^+ and attribute closures X^+ , which identify candidate keys and drive normalisation.

N21. Relational Model

Proposed by E. F. Codd in 1970, the relational model represents the whole database as a set of **relations** (tables) of atomic values. Vocabulary: **relation schema** STUDENT(enrol_no, name, prog_code, marks), **tuple** (row), **attribute** (column), **domain** (legal value set), **degree** (number of attributes), **cardinality** (number of tuples), and schema versus instance. Properties: values atomic, no duplicate tuples, tuple and attribute order irrelevant, NULL for unknown. Constraints: domain, key (super key, candidate key, primary key), **entity integrity** (primary key not NULL) and **referential integrity** (a foreign key matches an existing primary key or is NULL). Relationships are represented by shared *values*, not pointers, which allows declarative querying through relational algebra, relational calculus and SQL, and gives strong physical and logical data independence — the reason the model displaced the hierarchical and network models.

N22. Geographic Information Systems

A GIS is a system for capturing, storing, querying, analysing and displaying **spatially referenced** data, built on a spatial database. Two data models: **raster** (a grid of cells, each with a value — satellite images, elevation) and **vector** (points, lines, polygons with coordinates — roads, plot boundaries, cities), organised as thematic **layers** overlaid on a common coordinate reference system, plus attribute tables joined to the geometry. Spatial DBMS support needs geometric data types, spatial operators (contains, intersects, distance, buffer, nearest neighbour) and **spatial indexes** such as R-trees, quad-trees and grid files, since B-trees cannot index two dimensions. Standards: OGC Simple Features, SQL/MM, PostGIS, Oracle Spatial. Applications: land-records and cadastral systems, urban planning, utility and road-network management, vehicle navigation and routing, disaster management, epidemiology and site selection. Challenges: huge volumes, multi-resolution data, topological consistency and spatial join cost.

N23. Multimedia database and challenges in designing it

A multimedia database stores and retrieves media objects — text, images, audio, video, animation — together with their metadata and extracted features. Media are kept as BLOBs, in a file store with database references, or as user-defined complex types; content-based retrieval uses **feature vectors** (colour histograms, texture, shape, audio spectra) and similarity search with nearest-neighbour queries over high-dimensional indexes (R-tree, X-tree, LSH) rather than exact matching. Design challenges: enormous object size and storage/bandwidth needs; **continuous media** requiring guaranteed delivery rates, synchronisation of audio with video and quality-of-service scheduling; the semantic gap between low-level features and human meaning; absence of a natural ordering, so conventional indexes fail; expensive similarity computation; compression format dependence; versioning, copyright and digital-rights management. Applications: digital libraries, video on demand, medical imaging (PACS), e-learning, surveillance, news archives, GIS.

N24. Knowledge databases

A knowledge database stores facts **and** rules, so that answers are deduced rather than merely retrieved; it unites database persistence with logic programming. Facts resemble tuples, and rules are expressed in Datalog form, including recursive rules that relational algebra cannot express directly:

```
parent(amit, bina).
ancestor(X,Y) :- parent(X,Y).
ancestor(X,Y) :- parent(X,Z), ancestor(Z,Y).
```

Components: fact base, rule base, an **inference engine** using forward chaining (data-driven) or backward chaining (goal-driven), and an explanation facility. Knowledge may also be represented as frames, semantic networks or ontologies (RDF/OWL) for the semantic web. Applications: expert systems for medical diagnosis, credit approval and fault diagnosis, intelligent query answering and recommendation. Advantages: recursion, derived knowledge, declarative rules kept separate from code. Issues: rule consistency and conflict resolution, termination of recursion, and efficiency on very large fact bases.

N25. Nested queries in SQL

A nested query is a `SELECT` written inside the `WHERE`, `HAVING`, `FROM` or `SELECT` clause of an outer statement. A **non-correlated** subquery is evaluated once and its result handed to the outer query:

```
SELECT acc_no FROM account WHERE balance = (SELECT MAX(balance) FROM account);
SELECT name FROM student
WHERE prog_code IN (SELECT prog_code FROM programme WHERE fee > 20000);
```

A **correlated** subquery references an outer column and is re-evaluated per outer row:

```
SELECT c.name FROM customer c
WHERE EXISTS (SELECT 1 FROM account a WHERE a.cust_id = c.cust_id);
```

Operators: `IN/NOT IN`, `= ANY/SOME`, `> ALL`, `EXISTS/NOT EXISTS`, and scalar comparison for single-valued subqueries; subqueries also serve as derived tables in `FROM` and inside `INSERT ... SELECT` and `UPDATE ... SET`. Advantage: one declarative statement expresses multi-step logic, and `NOT EXISTS` expresses "for all" (division) queries. Drawback: correlated subqueries can be slow and are usually rewritten as joins by the optimiser.

N26. Enhanced E-R model

The EER (Enhanced/Extended E-R) model adds abstraction concepts to the basic E-R model for complex databases. **Specialisation** is the top-down division of an entity set into subclasses by a distinguishing characteristic — `EMPLOYEE` into `ENGINEER`, `SECRETARY`, `MANAGER` — drawn with a circle and a subset symbol (C) on each subclass. **Generalisation** is the reverse, bottom-up: common attributes of `CAR` and `TRUCK` are abstracted into a superclass `VEHICLE`. Subclasses **inherit** all attributes and relationships of the superclass and add their own, giving an IS-A hierarchy. Constraints: **disjoint (d) vs overlapping (o)** subclasses, and **total vs partial** participation. **Categorisation (union type)** gives a subclass with several possible superclasses: `OWNER` as a union of `PERSON` and `COMPANY`. **Aggregation** treats a relationship as a higher-level entity so that it can participate in another relationship.

N27. Type inheritance / Object definition language

Type inheritance in an object-relational or object-oriented DBMS lets a subtype be defined under a supertype, inheriting its attributes and methods and adding or overriding its own, giving substitutability and polymorphism:

```
CREATE TYPE person_t AS OBJECT (pid CHAR(5), name VARCHAR(40)) NOT FINAL;
CREATE TYPE student_t UNDER person_t (prog CHAR(5), marks NUMBER(3));
```

Table inheritance (`CREATE TABLE student OF student_t UNDER person`) means a query on the supertable also returns subtable rows unless `ONLY` is used; multiple inheritance raises conflict-resolution problems.

ODL (Object Definition Language) is the ODMG standard, language-independent DDL for object databases. It declares interfaces/classes with attributes, relationships with explicit `inverse` clauses, keys, extents and method signatures:

```
class Student extends Person (extent students key enrol_no) {
    attribute string name;
    relationship set<Course> takes inverse Course::taken_by;
    short total_credits(); };
```

OQL is its matching query language; ODL schemas map to C++/Java/Smalltalk bindings.

N28. Data fragmentation and replication in distributed databases

In a distributed database, global relations are divided and distributed to put data near its users. **Horizontal fragmentation** splits tuples by selection — `ACCOUNT_DELHI =  $\sigma(\text{branch}='Delhi')(\text{ACCOUNT})$` — and is reconstructed by union. **Vertical fragmentation** splits attributes by projection, each fragment retaining the primary key — `EMP1 =  $\pi(\text{eno}, \text{name})(\text{EMP})$ , EMP2 =  $\pi(\text{eno}, \text{salary})(\text{EMP})$` — reconstructed by natural join on the key. **Mixed** fragmentation combines both. Correctness requires completeness, reconstructability and disjointness. **Replication** keeps copies of a fragment at several sites: full, partial or none. Benefits: availability despite site failure, faster local reads, parallel query processing. Costs: every update must reach all copies, needing distributed concurrency control (distributed 2PL, primary copy or quorum protocols) and two-phase commit; lazy replication leaves copies temporarily inconsistent. The allocation decision balances read locality against update overhead.